

HemaNet

Technical & Product Blueprint

Version 0.1 — DRAFT for review (not approved)

Date: 30 September 2026

Initial deployment context: Kenya

Reference MVP: HemaNet MVP.pdf

Development-stage prototype. Not clinically validated, medically certified, regulatory compliant, or hospital-ready.
All clinical rules are configurable placeholders pending validation.

HemaNet — Technical & Product Blueprint

Status: DRAFT v0.1 — awaiting review and approval. No implementation has started.

Date: 2026-09-30

Initial deployment context: Kenya

Reference MVP: `HemaNet MVP.pdf` (4 pages; referred to below as "the PDF")

HemaNet is a development-stage prototype. It is **not** clinically validated, medically certified, regulatory compliant, or hospital-ready. Every clinical rule in this document is a configurable placeholder until validated by the appropriate Kenyan authorities and stakeholders.

Label legend

Every feature, rule and design element is tagged with one of these labels:

Label	Meaning
[CONFIRMED]	CONFIRMED REQUIREMENT: explicitly stated in the PDF or in your decisions of 2026-09-30
[PROPOSED]	PROPOSED IMPROVEMENT: added by this blueprint; needs your approval
[FUTURE]	FUTURE ENHANCEMENT: designed for (extension point), not built in the MVP
[VALIDATE]	ASSUMPTION REQUIRING VALIDATION: must be confirmed with stakeholders or authorities before real-world use

A feature can carry two labels, e.g. **[CONFIRMED][VALIDATE]**: required, but the exact rule or workflow still needs validation.

Assumptions are numbered `AS-xx` and listed in full in `11-risks-and-assumptions.md`. Architectural decisions are numbered `ADR-xxx` in `12-architecture-decisions.md`.

Document map

File	Sections
<code>01-product.md</code>	Vision, problem, users, personas, journeys, NFRs, A Product architecture, B Exact MVP scope, C Deferred features
<code>02-data-model.md</code>	D Complete ERD, E Entities & relationships, F Blood-unit/component model
<code>03-workflows.md</code>	G Request lifecycle/state machine, H Donor mobilisation workflow, I Blood inventory workflow
<code>04-matching-and-notifications.md</code>	J Matching engine, K Notification architecture
<code>05-security-and-access.md</code>	L Authentication & RBAC, M Permission matrix, R Security/privacy model
<code>06-api-specification.md</code>	N API specification
<code>07-frontend-and-backend-structure.md</code>	O Frontend route map, P Backend module map
<code>08-audit-and-traceability.md</code>	Q Audit & traceability model
<code>09-quality-and-operations.md</code>	S Testing, T Deployment, U Monitoring/logging, V Disaster recovery
<code>10-future-architecture.md</code>	W Mobile, X FHIR/HL7, Y AI/analytics
<code>11-risks-and-assumptions.md</code>	Z Risks & assumptions, AA Clinical/operational assumptions requiring validation, open questions

File	Sections
12-architecture-decisions.md	AB Architectural decisions & trade-offs (ADR log)
13-delivery-plan.md	AC Feature dependency graph, AD Prioritized MVP backlog, AE Exact development order, AF Definition of Done

Diagrams use Mermaid, which renders on GitHub, GitLab and most Markdown viewers.

Executive summary

1. **What HemaNet is:** a blood-resource coordination network. It is not a donor-finder app. It runs two linked workflows (ADR-006):
 - **Fulfilment:** Hospital request → routed to a blood bank → units allocated from real, per-unit inventory → issued → received.
 - **Mobilisation:** a shortage (low stock or a request shortfall) → explainable donor matching → staff-approved donor appeal → donor response → appointment/collection **recorded by blood-bank staff** → processing → units become available.
2. **Inventory is per-unit** (ADR-004). Totals such as "O+ RBC: 14" are always calculated from unit records, never stored.
3. **Clinical rules are data, not code** (ADR-005). Compatibility, eligibility, component shelf life/storage and urgency levels live in versioned **rule sets** that follow the lifecycle DRAFT → PENDING_VALIDATION → VALIDATED → ACTIVE. The code only implements the *types* of rule; the *values* come from configuration. Pilot and production environments refuse to run clinical logic on unvalidated rule sets.
4. **Architecture:** a modular Flask monolith, a React/TypeScript SPA, and PostgreSQL. A job queue and outbox stored in Postgres mean MVP background work needs no Redis or broker. Deployment uses portable containers, because Kenyan data-residency rules may constrain hosting (AS-40).
5. **Privacy by default:** no patient identifiers in the MVP. Donor identities are hidden from hospitals, and from blood banks until the donor accepts an appeal. The system never stores infectious-disease test results.
6. **Deliberately deferred:** PostGIS, WebSockets, FHIR/HL7, ML, transfers, cold-chain, QR printing, mobile app, blockchain (rejected). Each has a named extension point.

What I need from you

1. Approve, reject or amend the **[PROPOSED]** items. The highest-impact ones are summarised in [13-delivery-plan.md](#) §"Decisions needed from you".
2. Confirm the MVP scope in [01-product.md](#) §B.
3. Read the validation list in [11-risks-and-assumptions.md](#) §AA. It is effectively the agenda for your first stakeholder conversations.

01 — Product: Vision, Users, Architecture, Scope

1. Product vision

[CONFIRMED] HemaNet connects donors, hospitals, blood banks and administrators. Its aims are to reduce delays in getting blood, improve visibility of availability, reduce expiry wastage and encourage regular donation (PDF §Project Description).

[PROPOSED] Long-term identity: a **blood-resource coordination network** for Kenya. It links donor mobilisation, collection, per-unit inventory, hospital requests and (later) inter-facility redistribution, with measurable outcomes.

2. Problem statement

[VALIDATE] The problems below come from the PDF's framing. They have not yet been confirmed with Kenyan hospitals or blood services (AS-01).

#	Problem	Consequence	HemaNet response
P1	Hospitals lack timely visibility of blood availability at supplying blood banks	Phone-call chains; delays in urgent cases	Derived real-time availability plus a structured request workflow
P2	Requests and their status are tracked informally	No accountability or timing data	Request state machine with timestamps and audit
P3	Donor recruitment is reactive and untargeted	Slow replenishment; donor fatigue	Explainable matching and staff-approved appeals
P4	Stock expires unused	Wastage	Per-unit expiry tracking, expiring-soon views, FEFO allocation suggestions
P5	No shared metrics	Improvement can't be measured	Impact metrics captured from day one (§NFR-M)

3. Target users and personas

Persona	Organisation	Primary questions	Label
Wanjiru, regular donor (29, Nairobi, smartphone)	none	Am I eligible? Am I needed? Where and when? What's my impact?	[CONFIRMED]
Otieno, occasional donor (41, rural, basic feature phone)	none	Receives SMS; may not have data access	[PROPOSED] persona — drives AS-22 (feature-phone access)
Hospital requester (ward/lab staff)	Hospital	What do we need, how urgent, who is supplying, when will it arrive?	[CONFIRMED]
Hospital facility admin	Hospital	Which of our staff have access?	[PROPOSED]
Blood bank officer	Blood bank / RBTC / hospital blood bank	What's in stock, what's expiring, what's requested?	[CONFIRMED]
Blood bank manager	Blood bank	Should we launch a donor appeal? Staff access? Stock thresholds?	[PROPOSED] split from officer
Platform administrator	HemaNet operator	Is the network working? Who needs verification? Anomalies?	[CONFIRMED]

Persona	Organisation	Primary questions	Label
Clinical configuration approver	Named, qualified person designated by a partner blood service	Are the active clinical rules the validated ones?	[PROPOSED] (ADR-005)

[VALIDATE] AS-02: we assume Kenyan blood supply involves Regional Blood Transfusion Centres (RBTCs) and satellites supplying hospitals, some of which operate their own hospital blood banks. The facility model (§E, facilities) represents this through *capabilities* rather than fixed types, so that the real structure can be configured once confirmed.

4. User journeys (MVP)

J1 — Donor onboarding [CONFIRMED] Register (name, phone/email, date of birth, self-reported blood group, county/sub-county, optional approximate location) → verify phone by OTP → accept consent notice → dashboard shows eligibility as *Potentially eligible* / *Requires review* / *Potentially ineligible*, with reasons and next possible date.

J2 — Hospital urgent request (fulfilment) [CONFIRMED] Hospital requester opens "New request" → chooses component, ABO/Rh, units, urgency, required-by time → selects a supplying blood bank from the linked list (with live aggregate availability shown) → submits → watches the timeline: *Submitted* → *Accepted* → *Allocated* → *Issued* → *Received/Completed*.

J3 — Blood bank handles a request [CONFIRMED] Officer sees the request in an urgency-sorted queue → accepts or declines (with reason) → opens allocation suggestions (compatible, available, soonest-expiring first, each with an explanation) → reserves units → issues → hospital confirms receipt.

J4 — Shortage → donor appeal (mobilisation) [CONFIRMED core, PROPOSED staff approval] A low-stock threshold is breached, or a request can't be fully allocated → the system creates a *suggested* appeal → the manager reviews the target groups, collection site, time window and candidate preview (with explanations) → launches wave 1 → donors receive SMS/email/in-app alerts → donors accept or decline → accepted donors see where and when to go → staff check them in and **record the donation** → units are registered as QUARANTINED → released to AVAILABLE after local processing.

J5 — Admin verification [CONFIRMED] A facility self-registers (with its KMHFL code) → appears in the admin verification queue → admin verifies out-of-band and records the method → the facility's founding admin activates → that admin invites staff.

J6 — Reporting [CONFIRMED] Admin/manager views donation trends, shortages, donor activity, request performance and expiry wastage, and exports CSV.

5. Functional requirements (summary)

The full, labelled list is §B (MVP scope). Traceability to the PDF:

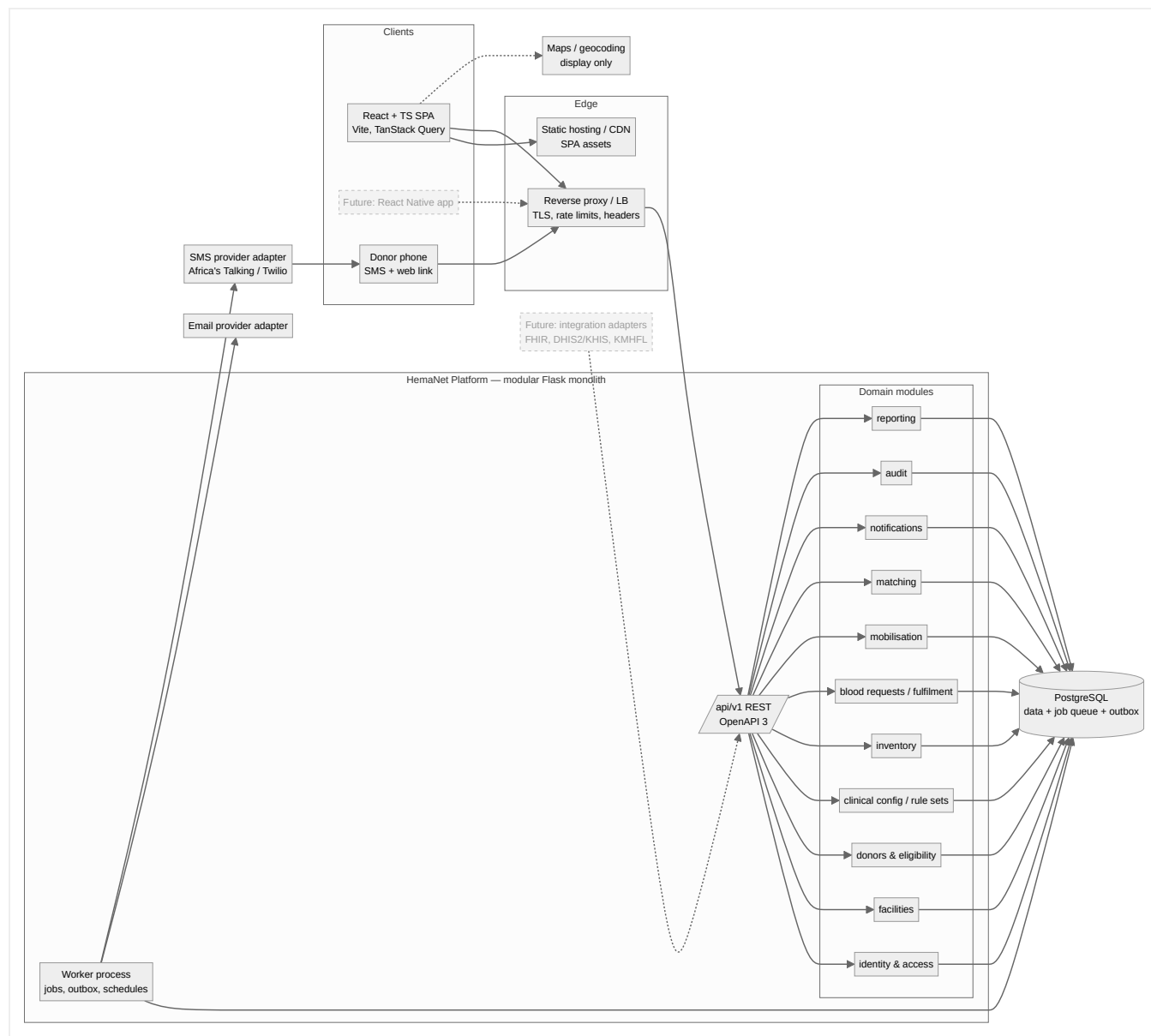
PDF requirement	Blueprint location
Donor register with name, blood group, age, contact, location	J1; <code>donor_profiles</code> (age → date of birth , [PROPOSED])
Donation history & eligibility (time since last donation, health status)	<code>donations</code> , <code>donor_deferrals</code> , eligibility engine (J §J.3); "health status" reduced to staff-recorded deferral categories [PROPOSED][VALIDATE]
Receive emergency alerts; accept/decline	Mobilisation (§H), notifications (§K)
Hospital register & location	<code>facilities</code> + verification
Submit requests (blood type, units, urgency)	<code>blood_requests</code> + component [PROPOSED]

PDF requirement	Blueprint location
View available inventory from blood banks and donors	Aggregate availability (§I); donor availability shown only as banded counts [PROPOSED — privacy]
Track request status (pending, matched, completed)	State machine §G (mapping given there)
Blood bank inventory (type, units, expiry)	Per-unit inventory §F/§I [CONFIRMED by decision 5]
Approve/reject hospital requests	request_routings accept/decline
Notify nearby donors when stock is low	Low-stock → suggested appeal → staff launch §H
Admin verify users	Facility verification + phone OTP for donors [PROPOSED modification: admin does not hand-verify individual donors]
Monitor activity, requests, inventory	Admin dashboard + audit log viewer
Reports: donation trends, shortages, donor activity	reporting module
SMS/email via Twilio/SendGrid	Provider-agnostic adapters; Africa's Talking evaluated for Kenya [PROPOSED]
Google Maps for location-based matching	Maps for display/geocoding only; distance computed in DB [PROPOSED, ADR-009]
S3 for prescriptions/certificates	Excluded from MVP — prescriptions are patient data (§C)

6. Non-functional requirements

ID	Requirement	Label
NFR-S1	All authorization enforced server-side; frontend checks are only for UX	[CONFIRMED] (master instruction)
NFR-S2	OWASP ASVS v4 Level 2 as the security verification reference	[PROPOSED]
NFR-P1	Health-related personal data is "sensitive personal data" under Kenya's Data Protection Act 2019. Apply data minimisation, a DPIA before the pilot, and consent records	[VALIDATE] AS-41 (legal review)
NFR-P2	Data residency may be constrained by the Data Protection (General) Regulations 2021	[VALIDATE] AS-40
NFR-R1	Pilot availability target 99.5% monthly; HemaNet must never be the <i>only</i> path to obtain blood (documented manual fallback)	[PROPOSED]
NFR-R2	RPO ≤ 5 min, RTO ≤ 4 h (pilot)	[PROPOSED]
NFR-PF1	p95 API latency < 500 ms for dashboard/list endpoints at pilot scale (≤ 20 facilities, ≤ 50k donors, ≤ 100k units)	[PROPOSED]
NFR-PF2	Usable on 3G-class connections: initial JS bundle < 250 KB gzipped, route-level code splitting	[PROPOSED]
NFR-A1	WCAG 2.2 AA	[PROPOSED]
NFR-I1	All UI strings externalised from day one; English at MVP, Kiswahili translation before donor-facing pilot	[PROPOSED][VALIDATE] AS-23
NFR-T1	All timestamps stored in UTC (timestamptz); displayed in Africa/Nairobi	[PROPOSED]
NFR-M	Capture timestamps needed for PDF success metrics: request → acceptance, request → issue, request → completion, appeal → first acceptance, acceptance → donation, units expired vs issued	[CONFIRMED] (PDF §Success Metrics)
NFR-O1	Every API response carries X-Request-ID; structured JSON logs without PHI	[PROPOSED]
NFR-C1	The same API serves web now and mobile later; no business logic lives only in React	[CONFIRMED]

A. Product architecture



Key properties

- **One deployable backend, two processes** [PROPOSED, ADR-001/003]: `api` (Gunicorn/Flask) and `worker` (the same codebase, a different entry point). Both are stateless apart from Postgres.
- **Domain modules** communicate through service interfaces and **domain events** written to an `outbox_events` table in the same transaction as the state change. The worker consumes them to send notifications, run matching and update projections. This gives us event-driven behaviour without a message broker.
- **Clinical logic isolation:** only `eligibility`, `matching` and `inventory.allocation` read clinical rule sets, and only through `clinical_config.get_active_rule_set(domain)`. No clinical values appear anywhere else.
- **Integration boundary:** external representations (future FHIR, SMS inbound, provider webhooks) terminate in `integrations/` or `notifications/providers/` adapters. They never reach domain models directly (§X).

B. Exact MVP scope

"MVP" means Phase 1: a production-quality web MVP, fit to enter a controlled pilot **after** the validation work in §AA.

B.1 Identity & access

Feature	Label
Email-or-phone + password login; Argon2id hashing	[CONFIRMED] (login) / [PROPOSED] (details)
Phone OTP verification (donors required; staff required)	[PROPOSED]
Email verification for staff accounts	[PROPOSED]
Password reset by email or SMS OTP	[PROPOSED]
Short-lived access token + rotating, revocable refresh token	[PROPOSED] ADR-007
TOTP MFA mandatory for platform admins and clinical-config approvers ; optional for others	[PROPOSED]
Roles → permissions, facility-scoped memberships	[CONFIRMED] RBAC / [PROPOSED] model ADR-008
Staff invited by their facility admin; memberships revocable	[PROPOSED]
Session list and "sign out other sessions"	[PROPOSED]

B.2 Facilities

Feature	Label
Facility self-registration (name, KMHFL code, type, county/sub-county, address, coordinates, contacts)	[CONFIRMED] (hospital registration) / [PROPOSED] (blood bank self-registration, KMHFL code)
Facility capabilities: <code>can_request_blood</code> , <code>can_hold_inventory</code> , <code>can_collect_donations</code>	[PROPOSED] (supports AS-02)
Admin verification queue; verify/reject/suspend with recorded method and notes	[CONFIRMED]
Supply links (which blood banks serve which hospitals), managed by admin	[PROPOSED][VALIDATE] AS-03
Storage locations (name, type) per facility	[PROPOSED]: cheap now, enables cold chain later

B.3 Donors

Feature	Label
Registration: full name, phone, optional email, date of birth, self-reported ABO/Rh, county, sub-county, optional approximate location	[CONFIRMED] (age → DOB [PROPOSED])
Blood group source: <code>SELF_REPORTED</code> vs <code>LAB_CONFIRMED</code> (confirmed by blood bank staff)	[PROPOSED]
Consent capture (versioned notice, timestamp)	[PROPOSED][VALIDATE] AS-41
Availability toggle (available / unavailable until date / opted out)	[PROPOSED]
Notification preferences: channels, quiet hours, max alerts per 30 days	[PROPOSED]
Donation history (staff-recorded only)	[CONFIRMED]
Eligibility status calculated by the server from validated rule set: 3-state output + reasons + next possible date	[CONFIRMED] (eligibility) / [PROPOSED] (3-state, rule-set driven) / [VALIDATE] all rule values
Staff-recorded deferrals (category code + dates only; no clinical detail)	[PROPOSED][VALIDATE] AS-13
Donor account deactivation and data export request	[PROPOSED][VALIDATE] AS-42

B.4 Clinical configuration (rule sets)

Feature	Label
Versioned rule sets for: <code>COMPATIBILITY</code> , <code>DONOR_ELIGIBILITY</code> , <code>COMPONENT_SPEC</code> , <code>URGENCY</code>	[PROPOSED] ADR-005
Lifecycle DRAFT → PENDING_VALIDATION → VALIDATED → ACTIVE → RETIRED; one ACTIVE per domain	[PROPOSED]

Feature	Label
Validation record: validator name, role, organisation, source document reference, date	[PROPOSED]
Environment guard <code>REQUIRE_VALIDATED_RULES=true</code> in pilot/production	[PROPOSED]
Dev seed rule sets labelled DEVELOPMENT MOCK — NOT CLINICALLY VALIDATED , with a UI banner	[PROPOSED]
Component catalogue: RBC, FFP/plasma, platelets, cryoprecipitate, whole blood (codes only; specs via rule set)	[CONFIRMED] decision 6

B.5 Inventory (per-unit)

Feature	Label
Register unit: identifier, component, ABO/Rh, collection date, expiry, source, storage location, optional link to donation / parent unit	[CONFIRMED] decision 5
Bulk registration: keyboard-wedge barcode scanner input + CSV import (validated, atomic)	[PROPOSED][VALIDATE] AS-30
Unit status machine (\$F) incl. QUARANTINED, AVAILABLE, RESERVED, ISSUED, RECEIVED, EXPIRED, DISCARDED	[PROPOSED]
Release from quarantine by authorised staff (records that local testing/processing is complete; no test results stored)	[PROPOSED][VALIDATE] AS-14
Append-only unit event history (chain of custody within HemaNet)	[CONFIRMED] decision 5 (traceability)
Derived availability (counts by facility × component × ABO × Rh, earliest expiry, last updated)	[CONFIRMED] decision 5
Expiring-soon view (window configurable per component)	[CONFIRMED] (track expiry)
Automatic expiry job and query-time expiry filter	[PROPOSED]
Low-stock thresholds per facility × component × group (values set by the facility, not by HemaNet)	[CONFIRMED] (notify when low) / [PROPOSED] (mechanism)
Discard with controlled reason codes	[PROPOSED][VALIDATE] AS-16

B.6 Blood requests (fulfilment)

Feature	Label
Create request: component, ABO/Rh, units, urgency level, required-by, optional external reference, optional notes (with PHI warning)	[CONFIRMED] (+component [PROPOSED])
No patient identifiers stored	[PROPOSED] ADR-013
Route to one linked blood bank; re-route after decline	[PROPOSED][VALIDATE] AS-03
Blood bank accept/decline with reason code	[CONFIRMED]
Allocation suggestions from the inventory matcher (compatible per active rule set, FEFO, explained)	[PROPOSED]
Reserve units (concurrency-safe), release reservation, reservation timeout	[PROPOSED]
Issue units; hospital confirms receipt	[PROPOSED][VALIDATE] AS-05
Cancel (hospital), close partially fulfilled (hospital), auto-expire past required-by + grace	[PROPOSED]
Status timeline and PDF status mapping	[CONFIRMED]
"Create donor appeal from shortfall" action	[CONFIRMED] decision 4

B.7 Mobilisation

Feature	Label
Appeals (campaigns): trigger type, target component and groups, collection site, time window, donors needed, urgency	[CONFIRMED] decision 4

Feature	Label
Suggested appeals generated from low stock / shortfall; staff launch required	[PROPOSED]
Candidate preview with per-donor explanation; donors masked until they accept	[PROPOSED]
Waves (batch sizes); "send next wave"	[PROPOSED]
Donor response via in-app or signed SMS/email link (accept/decline, optional reason)	[CONFIRMED]
Simple appointment: accepted donor picks an arrival slot within the window	[PROPOSED]
Staff check-in, no-show, and donation recording	[CONFIRMED] decision 4
Contact cooldown and per-donor alert caps	[PROPOSED]

B.8 Notifications

Feature	Label
Notification subsystem: notification + per-channel deliveries, templates, priority, retries, idempotency	[CONFIRMED] (master instruction §17)
Channels: in-app, email, SMS	[CONFIRMED]
Provider adapters; console/mailpit providers in dev labelled DEVELOPMENT MOCK	[PROPOSED]
Delivery-status webhooks (SMS DLR, email events)	[PROPOSED]
Escalation: unacknowledged request after the urgency level's <code>escalation_after_minutes</code> → notify managers + platform admin	[PROPOSED][VALIDATE] AS-10

B.9 Dashboards & reports

Feature	Label
Role dashboards (donor, hospital, blood bank, admin), polling-based refresh	[CONFIRMED] / [PROPOSED] polling ADR-012
Reports: donation trends, shortages, donor activity, request performance, expiry wastage; CSV export	[CONFIRMED] (first three) / [PROPOSED] (last two)
Admin notification-delivery health view	[PROPOSED]

B.10 Cross-cutting

Feature	Label
Append-only, hash-chained audit log; admin viewer	[CONFIRMED] audit / [PROPOSED] hash chain ADR-011
Structured logging, request IDs, health endpoints, error tracking with PII scrubbing	[PROPOSED]
OpenAPI spec generated from code; TS types generated from it	[PROPOSED] ADR-014
i18n scaffolding	[PROPOSED]
Docker-based local environment; CI with tests, lint, type-check, security scans	[PROPOSED]

B.11 Explicitly *not* in MVP

See §C. Also out of scope: patient records, cross-matching workflow, transfusion records at the bedside, payments/billing (e.g., SHA claims), and any hospital-side inventory beyond what the per-unit model already supports.

C. Deferred features

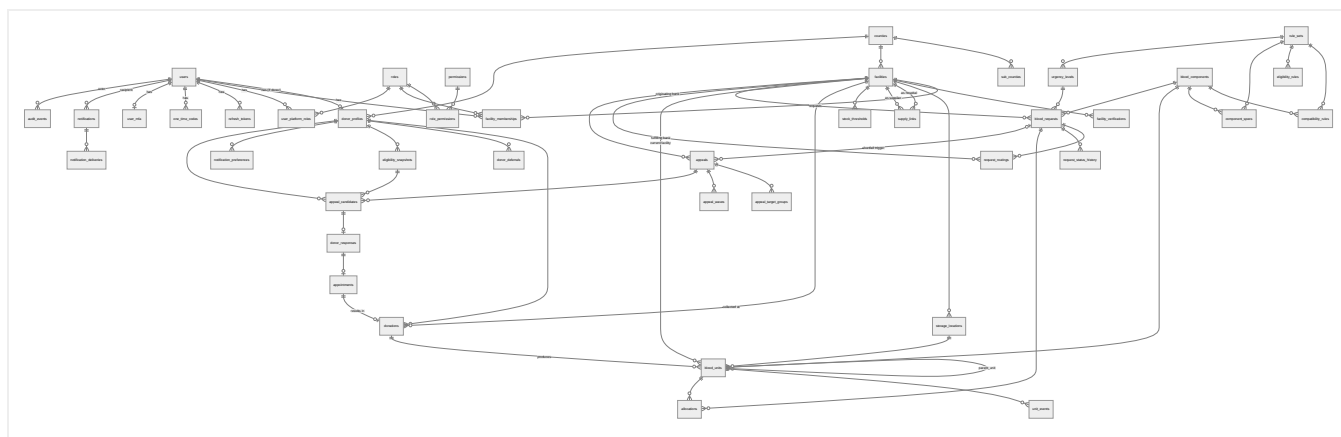
Feature	Phase	Label	Extension point built in MVP (only if cheap and migration-avoiding)
PostGIS spatial queries	V1	[FUTURE]	latitude/longitude numeric columns + a geo module interface <code>find_within_radius()</code> ; swap implementation later (ADR-009)
WebSockets / SSE real-time	V1	[FUTURE]	Domain events in outbox; frontend query keys designed for invalidation
Inter-facility transfers	V1	[FUTURE]	Unit statuses <code>IN_TRANSIT</code> , event types <code>TRANSFER_DISPATCHED/RECEIVED</code> reserved in enums; <code>unit_events.facility_id</code>
QR/barcode label printing	V1	[FUTURE]	<code>unit_identifier</code> accepts scanner input now; format validation pluggable (ISBT 128 AS-31)
Facility document upload (licences) to object storage	V1	[FUTURE]	Verification record has <code>method</code> + <code>evidence_reference</code> text
Real-time aggregate "network map"	V1	[FUTURE]	Availability view already per facility
KMHFL registry lookup for facility verification	V1	[FUTURE]	<code>kmhfl_code</code> column
Donor self-screening questionnaire	V2	[FUTURE]	Eligibility engine already has a 3-state output + rule types
Push notifications, mobile app	V2	[FUTURE]	Channel enum includes <code>PUSH</code> ; token auth usable by non-browser clients
Two-way SMS / USSD donor responses	V1–V2	[FUTURE] [VALIDATE] AS-22	Response service is channel-agnostic (<code>respond(candidate, response, channel)</code>)
Donor impact stories ("your donation contributed...")	V2	[FUTURE] [VALIDATE] AS-43	<code>unit_events</code> link donation → unit → issue
Cold-chain monitoring (manual/IoT readings)	V2	[FUTURE]	<code>storage_locations</code> table
Hospital-side final disposition (transfused/returned/wasted)	V2	[FUTURE] [VALIDATE]	Unit status machine extensible
Recall workflow	V2	[FUTURE]	Unit lineage via <code>donation_id</code> , <code>parent_unit_id</code>
FHIR/HL7, DHIS2/KHIS reporting	V3	[FUTURE]	Clean domain models; <code>integrations/</code> package placeholder only
Demand prediction / ML	V3	[FUTURE]	Event history and reporting schema; baseline heuristics first
Regional coordination / multi-county operations	V3	[FUTURE]	County on all facilities/donors
S3 prescriptions upload (PDF)	—	Not recommended	Patient data; not needed for coordination
Blockchain traceability (PDF)	—	Rejected (ADR-011)	Hash-chained audit log gives tamper-evidence

02 — Data Model

Conventions [PROPOSED]:

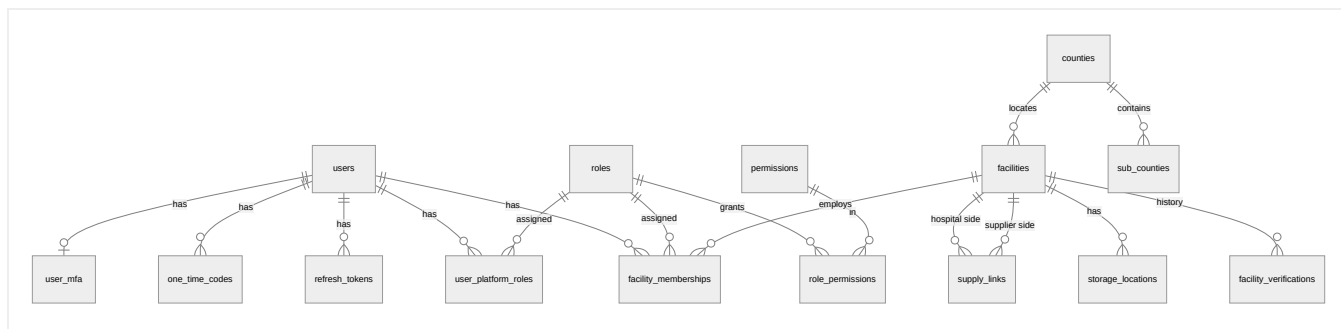
- PostgreSQL 16. SQLAlchemy 2.0 models, Alembic migrations (via Flask-Migrate).
- Primary keys: `uuid` (v7 where available, else v4) — non-enumerable (reduces IDOR guessing), merge-friendly for future multi-region. Append-only high-volume tables (`audit_events`, `unit_events`, `outbox_events`, `jobs`) use `bigint` identity for ordering. Human-facing references use separate `reference_code` columns (e.g., `REQ-2026-000123`) (ADR-016).
- Every mutable table: `created_at` `timestampz` not null default `now()`, `updated_at` `timestampz` not null, and `version` integer not null default `1` where concurrent edits are possible (optimistic locking).
- Soft deletion (`deleted_at`) **only** for: `users`, `facilities`, `donor_profiles`. Clinical/operational records (units, donations, requests, allocations) are never deleted — they move to terminal statuses.
- Controlled values are PostgreSQL enums **for stable state machines** (statuses) and **reference tables** for values that may change by configuration (components, urgency levels, reason codes). Rationale: enum changes need migrations (appropriate for code-coupled states); reference tables can be edited under rule-set control.
- `citext` for emails. Phone numbers stored in E.164 (+2547XXXXXXX), validated server-side.
- `jsonb` is used only where structure is legitimately variable: rule parameters, match explanations, notification template variables, audit diffs. Never for core business state.

D. Complete ERD

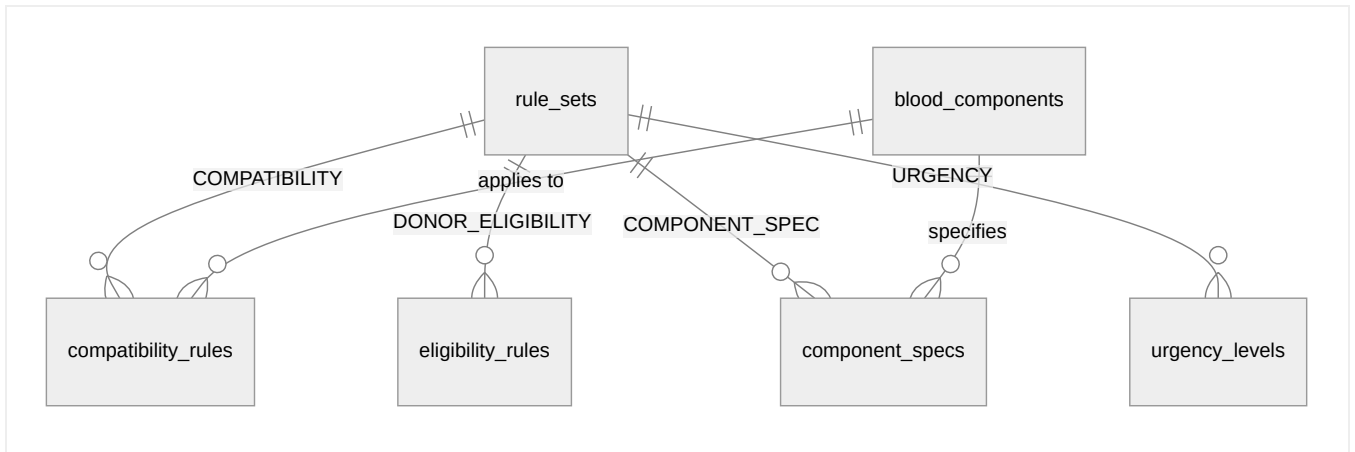


The full diagram above is dense, so the same relationships are repeated below, split by domain for readability.

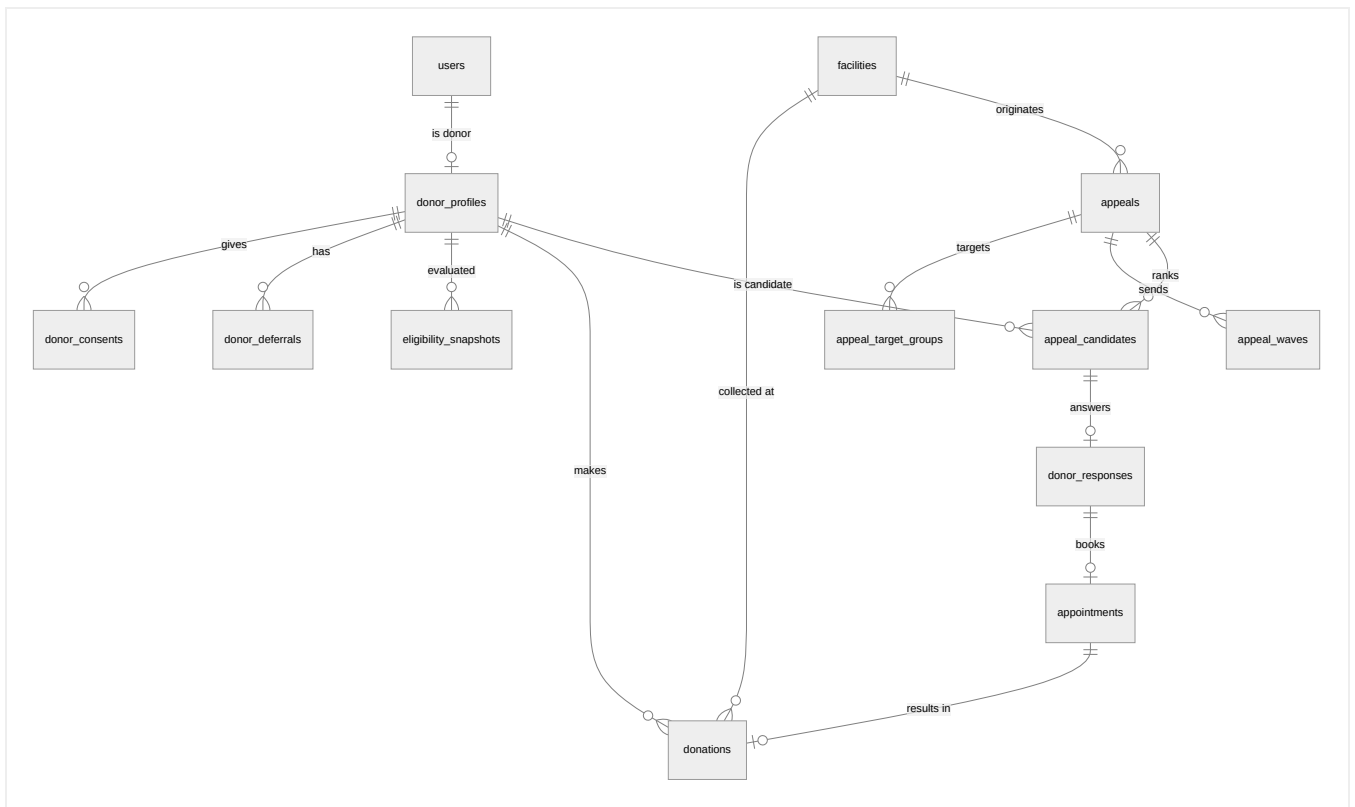
D.1 Identity, access & facilities



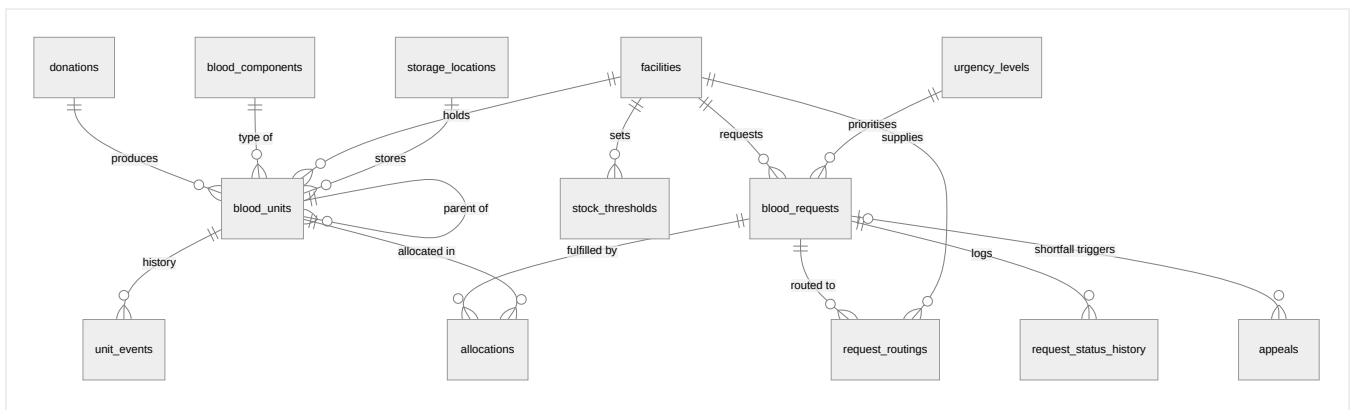
D.2 Clinical configuration (rule sets)



D.3 Donors, donations & mobilisation



D.4 Inventory, requests & fulfilment



Tables outside the diagram (infrastructure): `outbox_events`, `jobs`, `idempotency_keys`, `login_attempts`, `reason_codes`, `consent_notices`, `donor_consents`.

E. Database entities and relationships

Notation: **PK**, **FK** → **table**, **UQ** unique, **IX** index, **CK** check. Types abbreviated.

E.1 Identity & access

users [CONFIRMED core / PROPOSED fields]

Column	Type	Notes
id	uuid PK	
full_name	text not null	
email	citext null UQ (partial, where deleted_at is null)	
phone_e164	text null UQ (partial)	CK regex <code>^\+[1-9]\d{7,14}\$</code>
password_hash	text not null	Argon2id
status	enum <code>user_status</code> (PENDING_VERIFICATION, ACTIVE, SUSPENDED, DEACTIVATED)	
email_verified_at, phone_verified_at	timestampz null	
preferred_language	text not null default 'en'	CK in ('en','sw')
last_login_at	timestampz null	
token_version	int not null default 1	bumped on password change / suspension → invalidates outstanding access tokens
created_at, updated_at, deleted_at		

CK: email is not null or phone_e164 is not null. Note: PDF `role` column on Users is replaced by `user_platform_roles` + `facility_memberships` (a user may be a donor *and* hospital staff).

roles: id, code UQ (DONOR, HOSPITAL_REQUESTER, HOSPITAL_ADMIN, BLOOD_BANK_OFFICER, BLOOD_BANK_MANAGER, PLATFORM_ADMIN, CLINICAL_CONFIG_APPROVER), scope enum (PLATFORM, FACILITY), name, description. Seeded by migration.

permissions: id, code UQ (e.g. `blood_request.create`), description.

role_permissions: role_id FK, permission_id FK, PK(role_id, permission_id).

user_platform_roles: user_id FK, role_id FK (scope PLATFORM), granted_by FK → users, granted_at, revoked_at; UQ active (user_id, role_id) where revoked_at is null.

facility_memberships

Column	Notes
id uuid PK	
user_id FK → users, facility_id FK → facilities, role_id FK → roles (scope FACILITY)	
status enum (INVITED, ACTIVE, SUSPENDED, REVOKED)	
invited_by FK → users, invited_at, activated_at, revoked_at, revoked_by	

UQ (user_id, facility_id, role_id) where status in (INVITED, ACTIVE). IX (facility_id, status).

refresh_tokens: id, user_id FK, token_hash UQ (SHA-256 of opaque token), family_id uuid (rotation family), issued_at, expires_at, last_used_at, revoked_at, revoked_reason, replaced_by_id, user_agent (truncated), client_type enum (WEB, MOBILE). IX (user_id, revoked_at). Reuse of a rotated token revokes the whole family.

one_time_codes: id, user_id FK null, destination_hash, purpose enum (PHONE_VERIFY, EMAIL_VERIFY, PASSWORD_RESET, INVITE_ACCEPT, MFA_RECOVERY), code_hash, expires_at, consumed_at, attempt_count, CK attempt_count ≤ 5.

user_mfa: user_id PK/FK, totp_secret_encrypted bytea (app-level encryption, key from secrets manager), enabled_at, recovery_codes_hashed text[].

login_attempts: id bigint, identifier_hash, success bool, occurred_at, ip_prefix_hash. Used for progressive backoff/lockout. Retention 30 days.

E.2 Geography [PROPOSED]

counties: id smallint PK, code UQ (official county code 001–047), name. Seeded from an authoritative source [VALIDATE AS-44: source list].

sub_counties: id PK, county_id FK, name, UQ(county_id, name).

E.3 Facilities

facilities

Column	Notes
id uuid PK, reference_code UQ (FAC-000123)	
name text not null	
facility_type enum (HOSPITAL, BLOOD_SERVICE_CENTRE, HOSPITAL_BLOOD_BANK, OTHER)	descriptive only
can_request_blood, can_hold_inventory, can_collect_donations bool not null	capabilities drive permissions — [PROPOSED] [VALIDATE AS-02]
kmhfl_code text null UQ	Kenya Master Health Facility List code [PROPOSED]
county_id FK, sub_county_id FK null, address_text	
latitude numeric(9,6) null, longitude numeric(9,6) null	CK ranges; CK both-null-or-both-set
phone_e164, email	
verification_status enum (PENDING, VERIFIED, REJECTED, SUSPENDED)	
created_at, updated_at, deleted_at, version	

IX (verification_status), IX (county_id), IX (latitude, longitude).

facility_verifications (history): id, facility_id FK, decision enum (VERIFIED, REJECTED, SUSPENDED, REINSTATED), method enum (KMHFL_LOOKUP, PHONE_CALLBACK, SITE_VISIT, DOCUMENT_REVIEW, OTHER), evidence_reference text, notes text, decided_by FK → users, decided_at.

supply_links [PROPOSED][VALIDATE AS-03]: id, hospital_facility_id FK, supplier_facility_id FK, priority smallint, active bool, created_by, created_at. UQ(hospital, supplier). CK hospital ≠ supplier.

storage_locations [PROPOSED]: id, facility_id FK, name, storage_type enum (REFRIGERATOR, FREEZER, PLATELET_STORAGE, OTHER), active, UQ(facility_id, name). Temperature requirements are **not** stored here; they belong to component_specs (rule-set controlled).

stock_thresholds [PROPOSED]: id, facility_id FK, component_id FK, abo, rh, minimum_units int CK ≥ 0, updated_by, updated_at. UQ(facility_id, component_id, abo, rh). Values are set by the facility.

E.4 Clinical configuration (ADR-005)

rule_sets

Column	Notes
id uuid PK	
domain enum (COMPATIBILITY, DONOR_ELIGIBILITY, COMPONENT_SPEC, URGENCY)	
version_label text	e.g. 2026.1
status enum (DRAFT, PENDING_VALIDATION, VALIDATED, ACTIVE, RETIRED)	
is_development_mock bool not null	true for seeded placeholders; cannot be activated when REQUIRE_VALIDATED_RULES=true
validated_by_name, validated_by_role, validated_by_organisation, validation_source_reference, validated_at	required to enter VALIDATED
recorded_by FK → users (who entered the validation record), activated_by FK → users, activated_at, retired_at	
notes	

UQ (domain) where status = 'ACTIVE' (partial unique index → exactly one active per domain). Entries of a rule set are immutable once status ≥ PENDING_VALIDATION (enforced by service + DB trigger). Changes = new version.

blood_components [CONFIRMED decision 6]: id smallint, code UQ (WB, RBC, FFP, PLT, CRYO), display_name, active. Codes are domain vocabulary; *properties* are in `component_specs`.

component_specs: id, rule_set_id FK, component_id FK, shelf_life_value int, shelf_life_unit enum (HOURS, DAYS), storage_temp_min_c numeric(4,1), storage_temp_max_c numeric(4,1), expiring_soon_window_hours int, notes. UQ(rule_set_id, component_id). Used to *suggest/validate* expiry on unit registration; the entered label expiry remains authoritative [PROPOSED][VALIDATE AS-15].

compatibility_rules: id, rule_set_id FK, component_id FK, recipient_abo enum, recipient_rh enum, product_abo enum, product_rh enum, compatibility enum (COMPATIBLE, CONDITIONAL, INCOMPATIBLE), preference_rank smallint (1 = preferred), condition_note text. UQ(rule_set_id, component_id, recipient_abo, recipient_rh, product_abo, product_rh). Evaluation is **deny-by-default**: a missing row = incompatible.

eligibility_rules: id, rule_set_id FK, rule_code text, rule_type enum (MIN_AGE_YEARS, MAX_AGE_YEARS, MIN_DAYS_SINCE_LAST_DONATION, ACTIVE_DEFERRAL, BLOOD_GROUP_CONFIRMATION_REQUIRED, ...), parameters jsonb (validated against a per-type JSON schema in code), outcome_if_failed enum (POTENTIALLY_INELIGIBLE, REQUIRES_REVIEW), applies_to jsonb null (e.g., donation type), display_message_key. UQ(rule_set_id, rule_code).

urgency_levels: id, rule_set_id FK, code, display_name, rank smallint (1 = most urgent), target_acknowledge_minutes int, escalation_after_minutes int, reservation_hold_minutes int, bypass_quiet_hours bool not null default false, colour_token text. UQ(rule_set_id, code). Requests store the `urgency_level_id` they were created with (historic accuracy across versions).

reason_codes [PROPOSED]: id, category enum (REQUEST_DECLINE, REQUEST_CANCEL, UNIT_DISCARD, DEFERRAL_CATEGORY, DONOR_DECLINE, DONATION_OUTCOME, STATUS_CORRECTION), code, display_name, active. UQ(category, code). Reason codes replace free text for business-critical states.

E.5 Donors

donor_profiles

Column	Notes
id uuid PK, user_id FK UQ	

Column	Notes
donor_reference text UQ	short human code (e.g., DNR-7K3P9Q) shown on the donor's profile for staff lookup; future QR donor ID (PDF future enhancement) encodes this
date_of_birth date not null	replaces PDF age
sex_at_birth enum null (FEMALE, MALE, INTERSEX, UNDISCLOSED)	collected only if a validated eligibility rule requires it [VALIDATE AS-12]
abo enum (A, B, AB, O, UNKNOWN), rh enum (POS, NEG, UNKNOWN)	
blood_group_source enum (SELF_REPORTED, LAB_CONFIRMED)	
blood_group_confirmed_at, blood_group_confirmed_by_facility_id FK null	
county_id FK, sub_county_id FK null	
approx_latitude numeric(5,2), approx_longitude numeric(6,2) null	rounded server-side to 2 dp (~1.1 km) before storage ; precise home location is never stored [PROPOSED] privacy
availability enum (AVAILABLE, UNAVAILABLE, OPTED_OUT), unavailable_until date null	
last_contacted_at timestamptz null	for cooldown/fairness
created_at, updated_at, deleted_at, version	

IX (abo, rh, availability) where deleted_at is null; IX (county_id); IX (approx_latitude, approx_longitude). Note: PDF last_donation_date and eligibility_status are **derived** (from donations and the eligibility engine), not stored — avoids stale/contradictory state.

consent_notices: id, version UQ, language, content_hash, published_at.

donor_consents: id, donor_id FK, consent_notice_id FK, purpose enum (ACCOUNT, APPEAL_NOTIFICATIONS), granted_at, withdrawn_at.

notification_preferences: id, user_id FK, channel enum, enabled bool, UQ(user_id, channel); plus on donor_profiles-adjacent table donor_contact_limits: donor_id PK, quiet_hours_start time null, quiet_hours_end time null, max_appeals_per_30_days smallint null (null → platform default).

donations [CONFIRMED; staff-recorded only]

Column	Notes
id uuid PK, reference_code UQ	
donor_id FK → donor_profiles	
collection_facility_id FK → facilities (must have can_collect_donations)	
donation_identification_number text null UQ (per issuing system)	the label ID from the blood bank's own system [VALIDATE AS-31]
donation_type code (FK reason-like reference)	e.g. WHOLE_BLOOD; values [VALIDATE]
outcome enum (COLLECTED, DEFERRED_AT_SCREENING, INCOMPLETE)	
collected_at timestamptz	
appointment_id FK null UQ	links mobilisation → donation
recorded_by FK → users, recorded_at	
created_at, updated_at, version	

IX (donor_id, collected_at desc).

donor_deferrals [PROPOSED][VALIDATE AS-13]: id, donor_id FK, deferral_category_code FK → reason_codes (category DEFERRAL_CATEGORY), deferral_type enum (TEMPORARY, INDEFINITE), starts_on, ends_on null (CK temporary ⇒ ends_on not null), recorded_by FK, facility_id FK, lifted_at, lifted_by. **No free-text medical detail.**

eligibility_snapshots: id, donor_id FK, evaluated_at, rule_set_id FK, result enum (POTENTIALLY_ELIGIBLE, POTENTIALLY_INELIGIBLE, REQUIRES_REVIEW), reasons jsonb (list of {rule_code, outcome, message_key}), next_possible_date date null, context enum (DONOR_VIEW, MATCHING, STAFF_VIEW). Written when used for a decision (matching) so decisions are reproducible; not the source of truth.

E.6 Inventory

blood_units — see §F for lifecycle

Column	Notes
id uuid PK	
unit_identifier text not null	label/barcode value; UQ (issuing_system, unit_identifier)
issuing_system text not null default 'LOCAL'	namespace for identifiers from different blood services [VALIDATE AS-31]
component_id FK	
abo enum (A,B,AB,O), rh enum (POS,NEG)	no UNKNOWN allowed for units
volume_ml int null	
collected_at timestamptz null, expires_at timestamptz not null	CK expires_at > collected_at
source enum (DONATION_RECORDED_HERE, RECEIVED_EXTERNAL)	
donation_id FK null, parent_unit_id FK → blood_units null	lineage
batch_reference text null	for per-batch receipts (consignment/delivery note)
status enum unit_status	§F
current_facility_id FK, storage_location_id FK null	
registered_by FK, created_at, updated_at, version	

Indexes:

- IX available_lookup (current_facility_id, component_id, abo, rh, expires_at) WHERE status = 'AVAILABLE'
- IX (expires_at) WHERE status in ('AVAILABLE','QUARANTINED','RESERVED')
- IX (donation_id), IX (parent_unit_id)

unit_events (append-only, chain of custody): id bigint identity PK, unit_id FK, event_type enum, from_status, to_status, facility_id FK, actor_user_id FK null (null ⇒ system), occurred_at, recorded_at default now(), reason_code_id FK null, related_request_id FK null, related_allocation_id FK null, note text null (short, no PHI), metadata jsonb null. IX (unit_id, id). UPDATE/DELETE revoked at DB level (see §Q).

View inventory_availability: SELECT current_facility_id, component_id, abo, rh, count(*) units, min(expires_at) earliest_expiry, max(updated_at) last_change FROM blood_units WHERE status='AVAILABLE' AND expires_at > now() GROUP BY 1,2,3,4. Materialise later only if needed.

E.7 Requests & fulfilment

blood_requests

Column	Notes
id uuid PK, reference_code UQ (REQ-2026-000123)	
requesting_facility_id FK (must have can_request_blood)	

Column	Notes
created_by FK → users	
component_id FK, abo enum, rh enum	patient/recipient group as ordered — no patient identity
units_requested int CK > 0 AND ≤ configured max	
urgency_level_id FK	
required_by timestamptz	
status enum request_status	§G
external_reference varchar(50) null	hospital's own slip number; UI warns "no patient names"
notes varchar(500) null	PHI warning; visible only to requesting + routed facility
cancel_reason_code_id FK null, closed_reason_code_id FK null	
submitted_at, accepted_at, first_allocated_at, issued_at, completed_at, cancelled_at	metric timestamps (NFR-M)
created_at, updated_at, version	

IX (requesting_facility_id, status), IX (status, urgency_level_id, required_by). Derived (not stored): units_reserved, units_issued, units_received — from allocations.

request_routings: id, request_id FK, supplier_facility_id FK, status enum (PENDING, ACCEPTED, DECLINED, WITHDRAWN), routed_by, routed_at, responded_by, responded_at, decline_reason_code_id. UQ (request_id) where status in ('PENDING','ACCEPTED') — only one active supplier at a time in MVP [PROPOSED]; multiple suppliers is [FUTURE].

request_status_history: id bigint, request_id FK, from_status, to_status, actor_user_id null, reason_code_id null, occurred_at. Append-only.

allocations

Column	Notes
id uuid PK	
request_id FK, routing_id FK, unit_id FK	
status enum (RESERVED, RELEASED, ISSUED, RECEIVED, RETURNED)	RETURNED is [FUTURE]
reserved_by, reserved_at, reservation_expires_at	
released_at, released_reason_code_id	
issued_by, issued_at	
received_by, received_at	hospital confirmation

Critical constraint: **UQ (unit_id) WHERE status IN ('RESERVED','ISSUED','RECEIVED')** — a unit can never be held by two allocations. Combined with SELECT ... FOR UPDATE on units during allocation.

E.8 Mobilisation

appeals

Column	Notes
id uuid PK, reference_code UQ (APL-2026-000045)	
originating_facility_id FK (blood bank), collection_facility_id FK (can_collect_donations)	
trigger_type enum (LOW_STOCK, REQUEST_SHORTFALL, MANUAL)	
related_request_id FK null	

Column	Notes
component_id FK	component whose stock is short
donors_needed int CK > 0	
urgency_level_id FK	
window_start, window_end timestamptz	CK end > start
radius_km numeric(5,1)	
status enum (SUGGESTED, DRAFT, ACTIVE, PAUSED, CLOSED, CANCELLED)	
created_by (null if system-suggested), launched_by, launched_at, closed_at	
compatibility_rule_set_id, eligibility_rule_set_id FK	pinned at launch for reproducibility

appeal_target_groups: appeal_id FK, abo, rh, PK(appeal_id, abo, rh). Chosen by staff; system suggests from compatibility rules.

appeal_waves: id, appeal_id FK, wave_number, candidate_count, launched_by, launched_at. UQ(appeal_id, wave_number).

appeal_candidates: id, appeal_id FK, wave_id FK null (null = previewed, not contacted), donor_id FK, eligibility_snapshot_id FK, score numeric(6,2), rank int, distance_km numeric(6,2), explanation jsonb, response_token_hash text UQ null, token_expires_at, contacted_at. UQ(appeal_id, donor_id).

donor_responses: id, candidate_id FK UQ, response enum (ACCEPTED, DECLINED, WITHDRAWN), decline_reason_code_id null, channel enum (IN_APP, SMS_LINK, EMAIL_LINK, STAFF_RECORDED), responded_at.

appointments: id, response_id FK UQ, donor_id FK, facility_id FK, slot_start, slot_end, status enum (SCHEDULED, CHECKED_IN, COMPLETED, NO_SHOW, CANCELLED), checked_in_by, checked_in_at.

E.9 Notifications

notifications: id uuid, recipient_user_id FK, category enum (REQUEST_UPDATE, APPEAL, ACCOUNT_SECURITY, INVENTORY_ALERT, ESCALATION, SYSTEM), priority enum (LOW, NORMAL, HIGH, CRITICAL), template_code, template_version, payload jsonb (template variables; **validated to exclude PHI fields**), related_entity_type, related_entity_id, idempotency_key UQ, created_at, read_at null. IX (recipient_user_id, read_at, created_at desc).

notification_deliveries: id uuid, notification_id FK, channel enum (IN_APP, EMAIL, SMS, PUSH), destination_masked (e.g. +2547*****12), provider, provider_message_id, status enum (PENDING, SENDING, SENT, DELIVERED, FAILED, UNDELIVERABLE, SUPPRESSED), suppression_reason (OPTED_OUT, QUIET_HOURS, RATE_CAP, UNVERIFIED_CONTACT), attempt_count, next_attempt_at, last_error_code, sent_at, delivered_at, cost_minor_units int null. IX (status, next_attempt_at).

E.10 Audit & infrastructure

audit_events: see §Q.

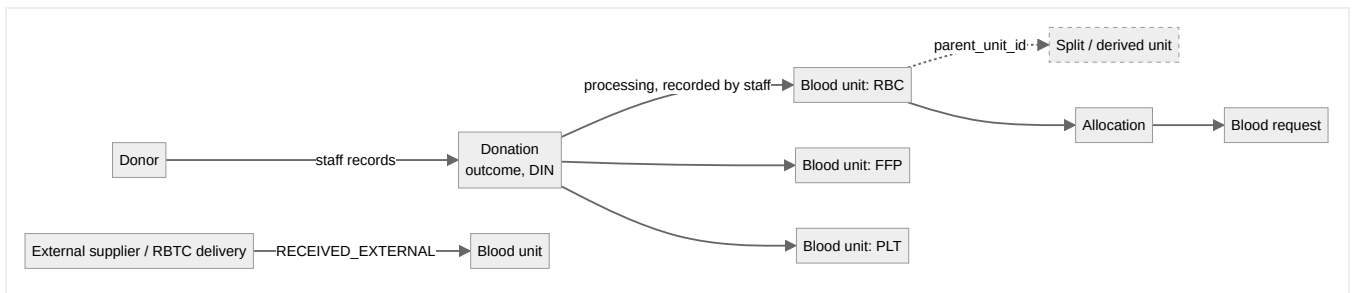
outbox_events: id bigint, event_type, aggregate_type, aggregate_id, payload jsonb, occurred_at, processed_at null, attempts, last_error. IX (processed_at) where processed_at is null.

jobs: id bigint, job_type, payload jsonb, run_at, status (QUEUED, RUNNING, SUCCEEDED, FAILED, DEAD), attempts, max_attempts, locked_by, locked_until, last_error, dedupe_key UQ null. Worker claims with **FOR UPDATE SKIP LOCKED**.

idempotency_keys: key, user_id, endpoint, request_hash, response_status, response_body jsonb, created_at, expires_at; PK(user_id, key).

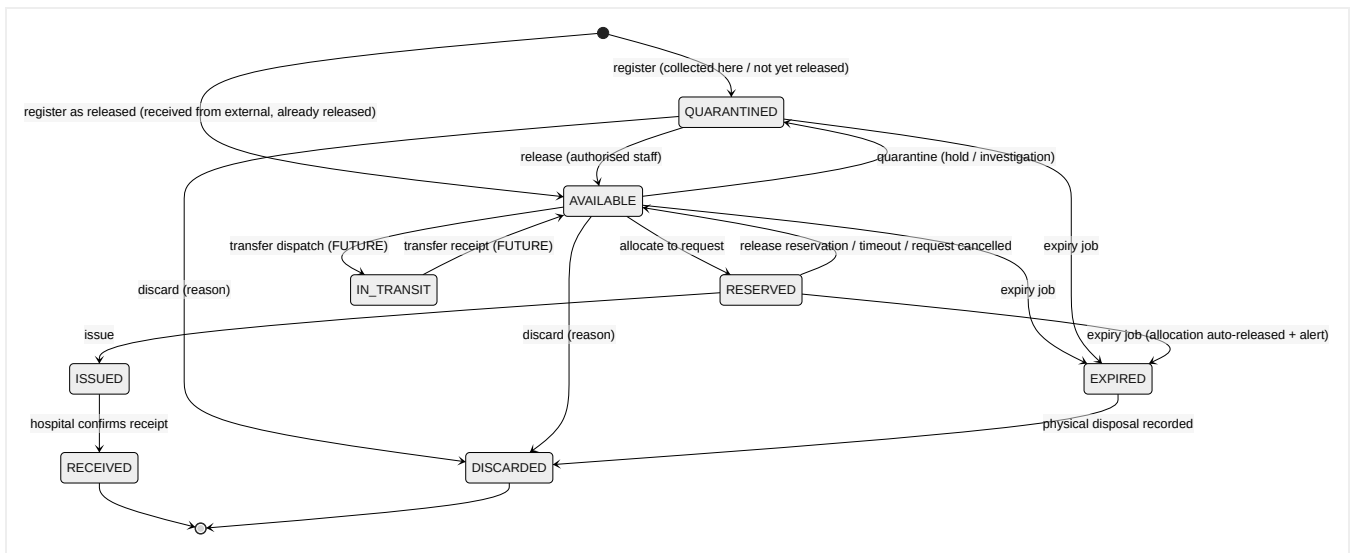
F. Blood-unit / component model

F.1 Concepts



- **Blood unit** = one labelled, physically distinct product bag of a single component with its own expiry. This is the unit of inventory truth [CONFIRMED decision 5].
- **Component** = a catalogue entry (`blood_components`); its properties (shelf life, storage range, expiring-soon window) come from the `ACTIVE COMPONENT_SPEC` rule set [CONFIRMED decision 6][VALIDATE AS-15].
- **Batch** = `batch_reference` groups units received together (a delivery). Batches are a *label on units*, not a separate stock entity, so aggregate counts can never diverge from units [PROPOSED].
- **Lineage**: `donation_id` (unit came from a donation recorded in HemaNet) and `parent_unit_id` (future splits/pooling). Units received from external systems have no donation link but keep their `unit_identifier` + `issuing_system` for traceability.
- **Testing**: HemaNet records only the **release decision** (QUARANTINED → AVAILABLE) by an authorised person. It does **not** store infectious-marker results — they are highly sensitive and remain in the blood service's own laboratory systems [PROPOSED][VALIDATE AS-14].

F.2 Unit status machine



Rules [PROPOSED]:

1. Transitions are implemented in one place: `inventory.domain.unit_state_machine`. Any other transition → 409 `INVALID_STATE_TRANSITION`.
2. Every transition writes a `unit_events` row in the same DB transaction.
3. `expires_at ≤ now()` makes a unit **ineligible for allocation at query time**, regardless of whether the expiry job has run.
4. A unit's `abo/rh/component/expires_at` are immutable after registration except via `STATUS_CORRECTED`-style correction events requiring a reason code and manager permission (audited).
5. `RECEIVED` = custody passed to the hospital. Final disposition (transfused/returned/wasted) is [FUTURE]

[VALIDATE].

6. `IN_TRANSIT` is included in the enum now (cheap), but no endpoint produces it in the MVP.

F.3 Component-aware compatibility (data, not code)

```
is_compatible(component, recipient_abo, recipient_rh, product_abo, product_rh)
:= lookup ACTIVE COMPATIBILITY rule set
  → row exists with compatibility ∈ {COMPATIBLE, CONDITIONAL}    (CONDITIONAL shown with note,
requires explicit staff confirmation)
  → no row ⇒ INCOMPATIBLE (deny by default)
```

Compatibility differs by component (e.g., plasma compatibility is not the same as red-cell compatibility). HemaNet will **not** ship built-in tables. Development seed data is flagged `is_development_mock=true`. Production values must come from Kenyan blood-service guidance [VALIDATE AS-11].

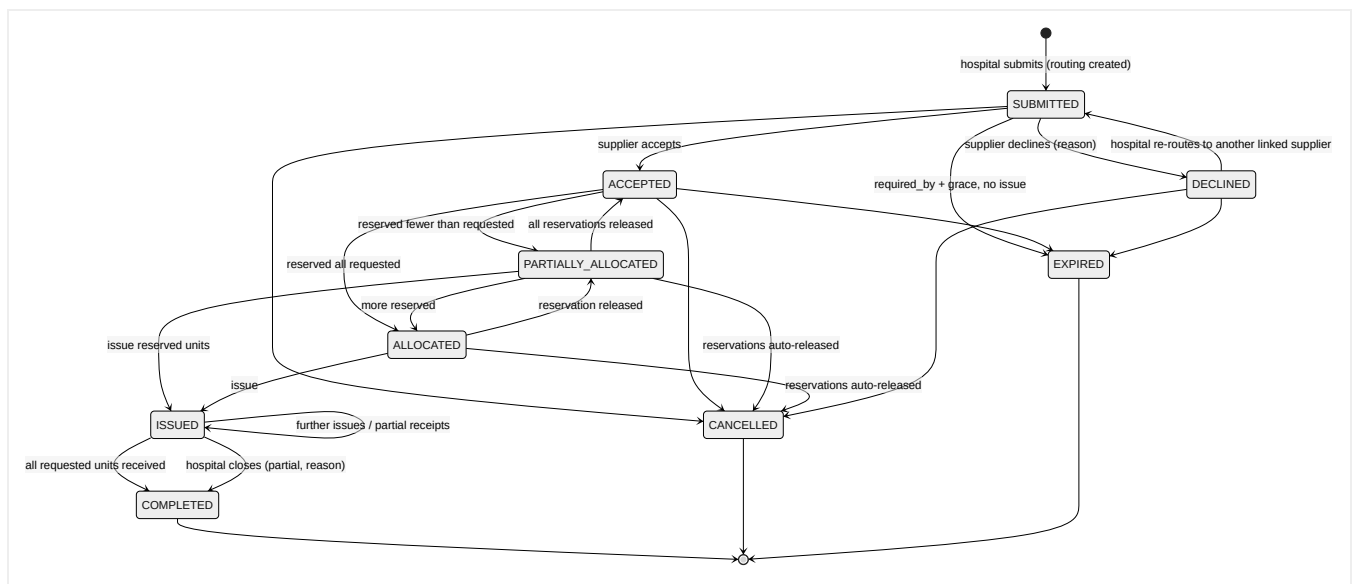
03 — Core Workflows

G. Request lifecycle / state machine

G.1 Request states

State	Meaning	PDF mapping
SUBMITTED	Created and routed to a supplier; awaiting acceptance	pending
ACCEPTED	Supplier accepted responsibility; no units reserved yet	matched
PARTIALLY_ALLOCATED	Some units reserved, fewer than requested	matched
ALLOCATED	Reserved units $\geq$ requested	matched
ISSUED	At least one unit issued; awaiting receipt	matched
COMPLETED	Units received $\geq$ requested, or hospital closed as sufficiently fulfilled	completed
DECLINED	Current supplier declined and not yet re-routed	pending (re-routable)
CANCELLED	Hospital cancelled before completion	—
EXPIRED	required_by + grace passed with no issue; auto-closed	—

Labels: states and transitions [PROPOSED]; the "pending $\rightarrow$ matched $\rightarrow$ completed" semantics [CONFIRMED].



G.2 Transition rules [PROPOSED]

Transition	Who (permission)	Guards	Side effects (same transaction)	Async (outbox)
create $\rightarrow$ SUBMITTED	blood_request. create @ requesting facility	facility VERIFIED & can_request_blood; supplier in active supply_links; supplier VERIFIED; component/urgency from ACTIVE rule sets; units_requested $\leq$ max; Idempotency-Key	routing PENDING; status history; audit; submitted_at	notify supplier staff (priority from urgency); schedule escalation job at escalation_after r_minutes
SUBMITTED $\rightarrow$ ACCEPTED	blood_request. respond @ supplier	routing PENDING	routing ACCEPTED; accepted_at	notify requester; cancel escalation job

Transition	Who (permission)	Guards	Side effects (same transaction)	Async (outbox)
SUBMITTED → DECLINED	<code>blood_request.respond @ supplier</code>	reason code required	routing DECLINED	notify requester (HIGH)
DECLINED → SUBMITTED	<code>blood_request.route @ requester</code>	new supplier ≠ previously declined for this request	new routing PENDING	as create
reserve units	<code>allocation.manage @ supplier</code>	routing ACCEPTED; each unit AVAILABLE, not expired, at supplier facility, compatible per ACTIVE compatibility rule set (CONDITIONAL requires <code>confirm_conditional=true</code>); lock units FOR UPDATE; total reserved ≤ requested	allocation RESERVED; unit RESERVED + unit_event; request status recomputed; <code>first_allocated_at</code>	notify requester
release reservation	<code>allocation.manage @ supplier</code> , or system (timeout/cancel/expiry)	allocation RESERVED	allocation RELEASED; unit AVAILABLE (if not expired)	—
issue	<code>allocation.issue @ supplier</code>	allocation RESERVED; unit not expired at issue time	allocation ISSUED; unit ISSUED; <code>issued_at</code>	notify requester (HIGH)
confirm receipt	<code>blood_request.receive @ requester</code>	allocation ISSUED	allocation RECEIVED; unit RECEIVED, <code>current_facility_id</code> = hospital; request COMPLETED if received ≥ requested	notify supplier
close (partial)	<code>blood_request.close @ requester</code>	status ISSUED; reason code	COMPLETED; <code>completed_at</code>	notify supplier
cancel	<code>blood_request.cancel @ requester</code>	non-terminal; reason code; if any unit is ISSUED, cancel is refused — use close instead	release all RESERVED	notify supplier
expire	system job	non-terminal, no ISSUED allocations, <code>now > required_by + grace</code>	release reservations; EXPIRED	notify both

Request status is **recomputed from allocations** by a single function (`requests.domain.recompute_status`) after allocation changes. This prevents drift between a stored status and the allocations underneath it (addresses master instruction §36).

G.3 Concurrency & integrity [PROPOSED]

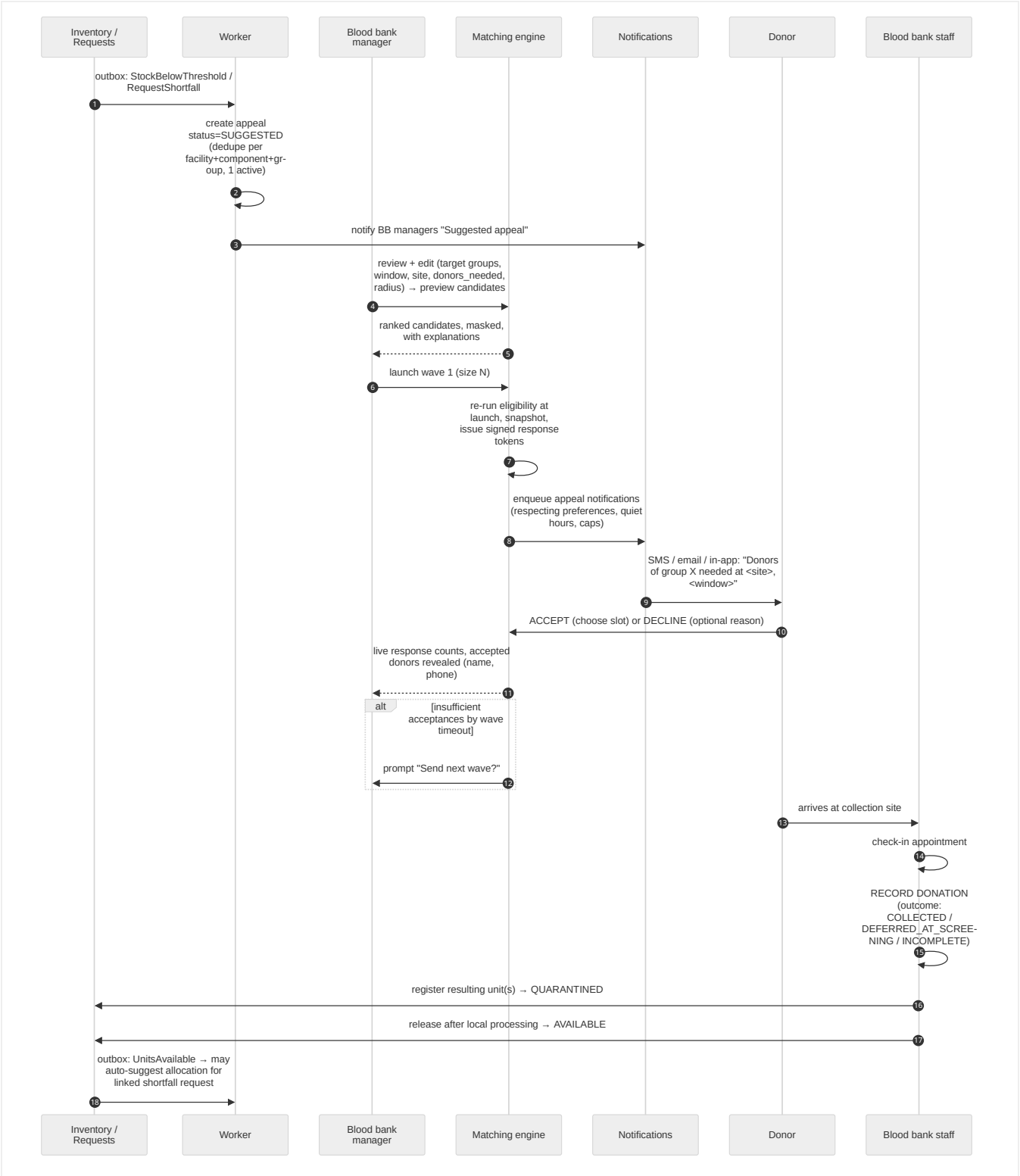
- Allocation runs in one transaction: `SELECT ... FROM blood_units WHERE id = ANY(:ids) FOR UPDATE`, then re-validate every guard, then insert allocations (partial-unique index is the last line of defence) → 409 UNIT_UNAVAILABLE for any failure.
- Request updates use `version` (optimistic locking); a stale version → 409 VERSION_CONFLICT, and the UI refetches.
- The reservation timeout is `urgency_levels.reservation_hold_minutes`, enforced by a scheduled job plus a check at issue time. [VALIDATE AS-06: are time-limited reservations appropriate operationally?]

G.4 Urgency [CONFIRMED configurable][VALIDATE AS-09]

Development placeholders (mock): `ROUTINE`, `PRIORITY`, `URGENT`, `CRITICAL`. Urgency influences: queue sort order, notification priority/channels, escalation timers, reservation hold time and dashboard prominence. **Urgency never bypasses any guard** (compatibility, verification, expiry, authorization).

H. Donor mobilisation workflow [CONFIRMED decision 4]

H.1 Sequence



H.2 Key rules

Rule	Label
A donor's acceptance is an intent , never a donation. Only staff-recorded donations count, and only released units count as supply	[CONFIRMED]

Rule	Label
Appeals are suggested by the system and launched by staff (no automatic mass SMS)	[PROPOSED] [VALIDATE AS-20]
One active or suggested appeal per (facility, component, target group) to prevent alert storms	[PROPOSED]
Target groups are chosen by staff ; the system pre-fills from the ACTIVE compatibility rule set (groups whose products can serve the short group)	[PROPOSED] [VALIDATE AS-11]
Eligibility is re-evaluated at launch and again at check-in (staff view); screening at the collection site remains authoritative	[CONFIRMED] (no bypass)
Donors with <code>REQUIRES_REVIEW</code> may be included only if the appeal sets <code>include_requires_review=true</code> ; the message tells them screening will confirm	[PROPOSED] [VALIDATE AS-21]
Before accepting, donors are masked to staff (initials, group, distance band, explanation). After accepting, staff see name and phone for coordination	[PROPOSED] privacy
Hospitals see only appeal status and counts (contacted / accepted / donated), never donor identities	[PROPOSED]
Response links: single-appeal scope, signed, expire at <code>window_end</code> ; they allow only accept/decline/slot choice for that candidate. No login required, and the page shows no personal data beyond the donor's first name	[PROPOSED]
Cooldown: a donor is not contacted again within <code>APPEAL_CONTACT_COOLDOWN_HOURS</code> (config) or beyond their personal cap	[PROPOSED]
No-show and decline history influence ranking gently (see §J); they never exclude a donor permanently	[PROPOSED]

H.3 Appeal state machine

`SUGGESTED` → `DRAFT` (staff edits) → `ACTIVE` (wave 1 launched) ⇄ `PAUSED` → `CLOSED` (manual close or `window_end` passed); `SUGGESTED/DRAFT` → `CANCELLED`. Suggested appeals auto-dismiss if the stock recovers before launch.

H.4 Appointment [PROPOSED][VALIDATE AS-24]

MVP uses slot choice within the appeal window (e.g., 30-min slots with a capacity per slot configured on the appeal). Walk-ins without appointments are supported at donation recording (appointment optional).

I. Blood inventory workflow

I.1 Registration paths

Path	Initial status	Who	Label
A. Unit from a donation recorded in HemaNet	<code>QUARANTINED</code>	blood bank officer	[CONFIRMED]
B. Unit received from an external supplier (e.g., RBTC delivery) already released	<code>AVAILABLE</code> (requires <code>unit.register_released</code> permission)	officer	[PROPOSED] [VALIDATE AS-04]
C. Bulk: scanner input or CSV of B-type receipts with a <code>batch_reference</code>	as B	officer	[PROPOSED]
D. Pilot opening balance: initial stock take	as B with event <code>OPENING_BALANCE</code>	manager	[PROPOSED]

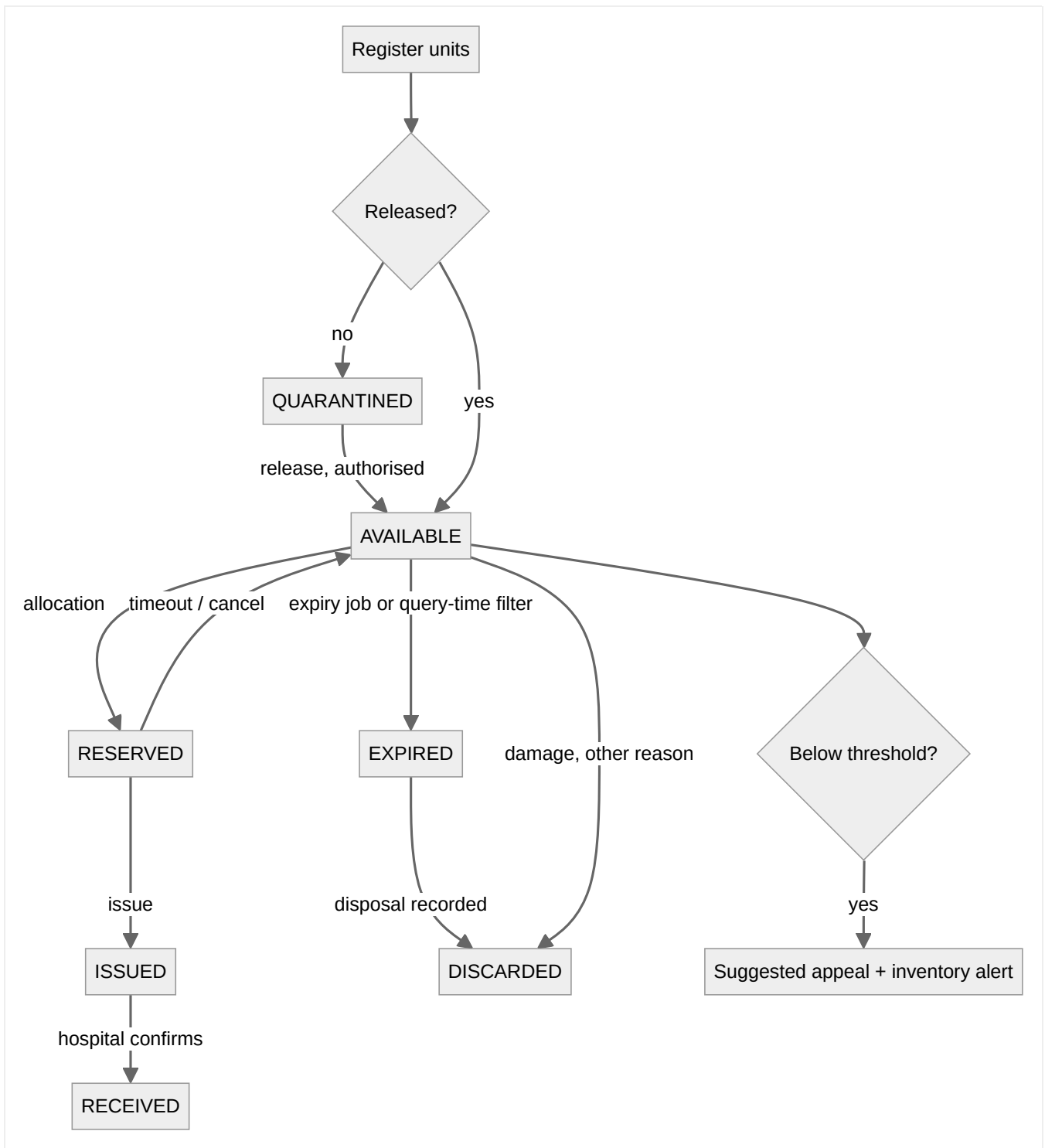
Registration validation [PROPOSED]:

- `unit_identifier` unique within `issuing_system`; duplicate → 409 (critical for double-count prevention).
- `expires_at` must be in the future at registration (unless recording an already-expired unit for disposal).
- Expiry plausibility: if the `ACTIVE COMPONENT_SPEC` gives a shelf life, `expires_at` - `collected_at` beyond that shelf life triggers a **warning requiring confirmation**, not a silent correction. The label is authoritative [VALIDATE

AS-15].

- Bulk import is atomic: all rows valid or none committed, with a per-row error report.

I.2 Daily operations



I.3 Derived views [CONFIRMED decision 5]

- **Facility availability:** the `inventory_availability` view (counts by component × ABO × Rh, earliest expiry, `last_change`).
- **Network availability for hospitals:** aggregates for the hospital's linked suppliers only, showing *counts and last_change*, never unit IDs. The UI shows a freshness indicator and the text "Availability is indicative; confirm via request" [PROPOSED] (availability can be stale if staff update late — risk R-05).

- **Expiring soon:** units with `expires_at < now + component_spec.expiring_soon_window_hours`, sorted ascending, with a quick "prioritise for allocation" signal (FEFO).
- **Low stock:** `inventory_availability` joined to `stock_thresholds`.

I.4 Scheduled jobs [PROPOSED]

Job	Frequency	Action
<code>expire_units</code>	every 5 min	AVAILABLE/QUARANTINED/RESERVED with <code>expires_at ≤ now</code> → EXPIRED; releases affected allocations; alerts
<code>release_stale_reservations</code>	every 1 min	RESERVED past <code>reservation_expires_at</code> → release; notify both sides
<code>check_thresholds</code>	on inventory change events + hourly sweep	create or refresh SUGGESTED appeals and inventory alerts
<code>expiring_soon_digest</code>	daily 07:00 EAT	notify blood bank officers (in-app + email)
<code>expire_requests</code>	every 5 min	per §G.2
<code>close_appeals</code>	every 5 min	past <code>window_end</code> → CLOSED

I.5 Reconciliation [PROPOSED][VALIDATE AS-32]

A stock-take screen lists AVAILABLE/QUARANTINED units per storage location. Staff mark each unit *present* or *missing*. Missing units need a manager-approved correction event with a reason. The result is a reconciliation record that shows how far system stock and physical stock have drifted, which is an important pilot metric. Tables: `stock_takes` (id, facility_id, started_by, started_at, completed_at, status) and `stock_take_items` (stock_take_id, unit_id, result PRESENT|MISSING|UNEXPECTED, resolved_by, correction_event_id). This is backlog priority P1 (late MVP) and is needed before the pilot.

04 — Matching Engine & Notification Architecture

J. Matching-engine design

J.1 Principles

Principle	Label
Deterministic, explainable, rule-driven. No ML in the MVP	[CONFIRMED]
Hard filters (safety) and soft ranking (convenience) are kept strictly separate. Ranking can never override a filter	[PROPOSED]
Every result carries a machine-readable explanation, rendered as human text	[CONFIRMED]
Pinned rule-set versions and eligibility snapshots make every match reproducible	[PROPOSED]
Pure functions over plain data (no DB access inside scoring) so they're unit-testable with golden files	[PROPOSED]
Matching <i>recommends</i> . A human allocates units and launches appeals	[PROPOSED][VALIDATE AS-20]

There are two matchers under one `matching` module:

J.2 Inventory matcher (fulfilment)

Input: request (component, recipient ABO/Rh, units outstanding), supplier facility.

Pipeline:

1. **Candidate query (SQL):** units at the supplier, `status='AVAILABLE'`, `expires_at > now()`, `component_id = requested` (component substitution is out of scope [VALIDATE AS-07]).
2. **Hard filter:** compatibility lookup in the ACTIVE COMPATIBILITY rule set (deny by default). CONDITIONAL rows are returned with a flag and require explicit confirmation.
3. **Order:** `preference_rank` ascending (e.g., identical group first, if the validated table says so), then `expires_at` ascending (FEFO), then `unit_identifier`. [VALIDATE AS-08: FEFO vs. clinical minimum remaining shelf life for certain patients/components]
4. **Explain:** per unit, e.g.

```
{
  "unit_id": "...", "unit_identifier": "...",
  "compatibility": {"result": "COMPATIBLE", "preference_rank": 1, "rule_set": "COMPATIBILITY 2026.1 (VALIDATED)"},
  "expires_in_hours": 52,
  "reasons": ["IDENTICAL_GROUP_PREFERRED", "EARLIEST_EXPIRY_FIRST"]
}
```

5. Returns up to `units_outstanding × 2` suggestions. Staff select, then reserve (§G.2).

If the compatible available count is lower than the units outstanding, the response includes `shortfall: n`, and the UI offers "Create donor appeal from shortfall".

J.3 Eligibility evaluator (used by donor matcher and donor dashboard)

- **Rule types** are implemented in code (`eligibility/rules/*.py`), each with a JSON-schema for its parameters. **Rule values** come from the ACTIVE DONOR_ELIGIBILITY rule set [CONFIRMED principle][VALIDATE AS-12].
- Inputs: DOB, (sex only if a validated rule needs it), donation history, active deferrals, blood-group confirmation

status and the reference date (e.g., appeal window start).

- Output: `POTENTIALLY_ELIGIBLE` | `POTENTIALLY_INELIGIBLE` | `REQUIRES_REVIEW`, a list of reasons and `next_possible_date`.
- **Missing or unknown data** → `REQUIRES_REVIEW`, never `ELIGIBLE`. An unknown rule type or invalid parameters → the evaluator fails closed, logs an error and raises a platform alert.
- Language used everywhere: "potentially eligible — final eligibility is determined at screening".

J.4 Donor matcher (mobilisation)

Input: appeal (target groups, collection facility location, radius, window, component, include_requires_review), wave size.

Stage 1 — Candidate generation (SQL, fast, index-backed)

- `donor_profiles` not deleted; `users.status = ACTIVE`; phone verified
- `abo, rh` ∈ target groups
- `availability = AVAILABLE` or `UNAVAILABLE` with `unavailable_until < window_start`
- has active consent for `APPEAL_NOTIFICATIONS`
- not already a candidate in this appeal; `last_contacted_at` older than the cooldown
- inside a bounding box around the collection facility, then precise haversine distance ≤ radius (SQL function). The implementation lives behind `geo.find_within_radius()` so PostGIS can replace it later (ADR-009)
- Donors without coordinates: included only if `county_id` = the facility's county, with distance marked `UNKNOWN_SAME_COUNTY` [PROPOSED][VALIDATE AS-25]

Stage 2 — Hard filters (Python, rule-driven)

- Eligibility evaluator → exclude `POTENTIALLY_INELIGIBLE`; include `REQUIRES_REVIEW` only if the appeal allows it
- `next_possible_date ≤ window_end`
- Contact caps (personal max per 30 days; platform default)

Stage 3 — Transparent scoring (weights in platform config, not clinical rule sets; they're operational, not clinical)

Factor	Points (default, tunable)	Why
Distance	<code>max(0, 40 × (1 - d/radius))</code>	Closer donors can arrive sooner
Blood group lab-confirmed	+15	Less risk of wasted trips from a wrong self-reported group
Identical group to the shortage (per rule-set <code>preference_rank = 1</code>)	+10	Operational preference
Response history (accept rate over last 5 contacts, Laplace-smoothed)	0–15	Likely to respond
Show-up history (appointments completed / accepted, smoothed)	0–10	Likely to arrive
Rest since last contact (fairness)	0–10, increasing with days since <code>last_contacted_at</code>	Spreads the load and reduces fatigue for top donors
Eligibility <code>REQUIRES_REVIEW</code>	–10	Prefer clearer candidates

Score = sum of factors. Ties are broken by distance, then by a stable per-appeal random hash so no donor is systematically favoured. Every factor's contribution is stored in `appeal_candidates.explanation`:

```

{
  "score": 71.5,
  "hard_filters": ["GROUP_TARGETED:O+", "ELIGIBILITY:POTENTIALLY_ELIGIBLE", "WITHIN_RADIUS",
"CONSENTED", "NOT_IN_COOLDOWN"],
  "factors": [
    {"code": "DISTANCE", "value_km": 4.2, "points": 31.6},
    {"code": "BLOOD_GROUP_LAB_CONFIRMED", "points": 15},
    {"code": "IDENTICAL_GROUP", "points": 10},
    {"code": "RESPONSE_HISTORY", "value": "3/5", "points": 8.9},
    {"code": "REST_SINCE_LAST_CONTACT", "value_days": 61, "points": 6.0}
  ],
  "rule_sets": {"eligibility": "...", "compatibility": "..."},
  "matcher_version": "donor-matcher/1.0.0"
}

```

Rendered for staff as: *"Compatible group (O+, lab-confirmed) · Potentially eligible · 4.2 km away · Responded to 3 of last 5 appeals · Last contacted 61 days ago."*

Stage 4 — Waves: the top N not yet contacted go into the wave. The rest stay as previewed candidates for later waves.

Fairness and risk notes [PROPOSED]: we'll monitor contact distribution across donors and counties in reports. The rest factor exists to prevent "always ping the same 50 donors". Blood groups are never used to prioritise donors beyond what the validated compatibility preference says.

J.5 Performance

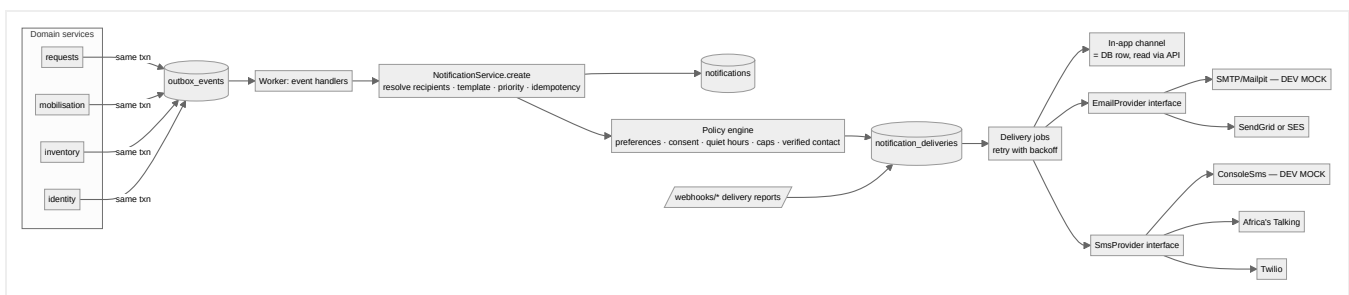
At pilot scale ($\leq 50k$ donors), stage 1 with composite indexes on (abo, rh, availability) plus lat/long bounding-box filtering is milliseconds. Matching runs **as a job** at launch (it's not on the request path) and writes candidates. Preview runs synchronously with a limit.

J.6 Hospital "view donors" (PDF) [CONFIRMED, modified for privacy — PROPOSED]

Hospitals see: *"Registered donors of compatible groups within the supplier's area: 50–100 (potentially eligible)"*, in bands, never lists. Counts under 10 display as "fewer than 10", to avoid re-identification.

K. Notification architecture [CONFIRMED subsystem]

K.1 Components



K.2 Rules

Rule	Label
Route handlers never call providers. They change domain state, and events trigger notifications	[CONFIRMED]
One notification per recipient per event; one delivery per channel	[PROPOSED]

Rule	Label
<code>idempotency_key = event_id + recipient + template</code> ⇒ a retried handler never double-sends	[PROPOSED]
Templates versioned in code (<code>notifications/templates/{code}/{lang}.{channel}.txt</code>), rendered with an allow-listed variable set. Payload validation rejects any field not in the template's schema , which stops PHI leaking into SMS	[PROPOSED]
SMS body ≤ 160 GSM-7 characters where possible; the link uses a short HemaNet domain path	[PROPOSED]
Retries: exponential backoff (30 s, 2 m, 10 m, 30 m, 2 h), max 5; then <code>FAILED</code> + admin dashboard counter. Permanent errors (invalid number) → <code>UNDELIVERABLE</code> straight away	[PROPOSED]
Priority → channels: CRITICAL/HIGH: in-app + SMS + email (if enabled); NORMAL: in-app + email; LOW: in-app only (mapping configurable)	[PROPOSED] [VALIDATE AS-26]
Quiet hours are respected for donor appeals except where the urgency level has <code>bypass_quiet_hours=true</code> and the donor opted in to urgent night alerts	[PROPOSED]
Opt-out is always respected, including for critical alerts. Account-security messages are exempt	[PROPOSED]
Unverified contact → delivery <code>SUPPRESSED(UNVERIFIED_CONTACT)</code>	[PROPOSED]
Provider failover: if the primary SMS provider returns 5xx/timeouts beyond a threshold, route to the secondary if one is configured	[PROPOSED] (config only in MVP)
Dev environment: SMS goes to the console/log, email to Mailpit. UI deliveries show "DEVELOPMENT MOCK — not sent"	[CONFIRMED] mock labelling
Webhook endpoints verify provider signatures or shared secrets and are idempotent	[PROPOSED]
Sender ID / short code registration and bulk-SMS compliance in Kenya	[VALIDATE AS-27]

K.3 Escalation [PROPOSED][VALIDATE AS-10]

<pre> RequestSubmitted └─ schedule job escalate_request(request_id, level=1) at submitted_at + urgency.escalation_after_minutes escalate_request: if routing still PENDING: level 1 → notify all BLOOD_BANK_MANAGER members of supplier (HIGH/CRITICAL) schedule level 2 at + escalation_after_minutes level 2 → notify requester ("no response yet – consider re-routing") + PLATFORM_ADMIN else: no-op AppealWaveLaunched └─ schedule check_wave at + wave_response_window check_wave: if accepted < donors_needed → notify manager "Send next wave?" (no automatic send in MVP) </pre>
--

Escalation never auto-routes a request to another blood bank and never auto-sends new waves. Both are human decisions in the MVP.

K.4 Notification catalogue (MVP)

Template code	Recipients	Priority
<code>request.submitted</code>	supplier officers/managers	from urgency
<code>request.accepted</code> / <code>request.declined</code>	requesting facility members	NORMAL / HIGH
<code>request.units_issued</code>	requester	HIGH
<code>request.escalation</code>	supplier managers, admin	HIGH/CRITICAL
<code>request.expired</code> / <code>request.cancelled</code>	both sides	NORMAL
<code>inventory.low_stock</code>	supplier managers	NORMAL/HIGH
<code>inventory.expiring_digest</code>	supplier officers	LOW
<code>inventory.reservation_expired</code>	both sides	HIGH
<code>appeal.suggested</code>	managers	NORMAL

Template code	Recipients	Priority
appeal.invitation	donors	from urgency
appeal.response_confirmation	donor	NORMAL
appeal.reminder	accepted donors, 2 h before slot	NORMAL
account.* (verify, reset, new sign-in, membership invite)	user	HIGH (security)
facility.verification_decision	facility admin	NORMAL

05 — Authentication, RBAC, Permissions, Security & Privacy

L. Authentication and RBAC

L.1 Authentication [PROPOSED, ADR-007]

Element	Design
Credentials	Email or phone + password. Argon2id (argon2-cffi; memory ≥ 64 MiB, t=3, p=1, tuned on target hardware). Minimum length 10, checked against a breached/common-password list; no composition rules (NIST SP 800-63B)
Access token	JWT, 15 min, HS256 with a key from the secrets manager (RS256 when a second service needs to verify tokens). Claims: <code>sub</code> , <code>sid</code> (refresh family), <code>iat</code> , <code>exp</code> , <code>jti</code> , <code>ver</code> (the user's <code>token_version</code> , bumped on password change/suspension). No roles in the token . Permissions are loaded server-side per request (cached ≤ 60 s), so revocation takes effect immediately
Refresh token	Opaque 256-bit random value, stored hashed. Rotated on every use; reuse detection revokes the family. Lifetime: 12 h idle / 7 days absolute for staff; 30 days for donors [PROPOSED][VALIDATE AS-45]
Web transport	Access token held in memory (not <code>localStorage</code>). Refresh token in a <code>__Host-hemanet_rt</code> cookie: <code>HttpOnly</code> ; <code>Secure</code> ; <code>SameSite=Strict</code> ; <code>Path=/api/v1/auth</code>
CSRF	Only the refresh/logout endpoints use a cookie. They require an <code>X-CSRF-Token</code> header matching a double-submit cookie, and check <code>Origin</code> . All other endpoints use a Bearer header, so they aren't CSRF-exposed
Mobile (future)	The same endpoints return the refresh token in the body when <code>client_type=MOBILE</code> , for storage in secure OS storage
MFA	TOTP (RFC 6238), mandatory for <code>PLATFORM_ADMIN</code> and <code>CLINICAL_CONFIG_APPROVER</code> ; optional for others; 10 single-use recovery codes. Sensitive actions (activating a rule set, verifying a facility, granting platform roles) require an MFA check in the last 15 min (step-up)
Phone verification	6-digit OTP, 10-min expiry, max 5 attempts, resend throttled (60 s, max 5/hour/number)
Brute force	Per-identifier progressive delay after 5 failures; lockout 15 min after 10; per-IP limits at the edge. Uniform error "Invalid credentials"
Account enumeration	Registration and reset responses are identical whether or not the account exists (with the email/SMS side channel handled asynchronously)
Session management	<code>/me/sessions</code> lists refresh families; users can revoke them. Admin suspension revokes all sessions

L.2 Authorization model [CONFIRMED principle "role ≠ permission"; PROPOSED design, ADR-008]

```
User —< user_platform_roles >— Role (scope PLATFORM) —< role_permissions >— Permission
User —< facility_memberships >— Role (scope FACILITY) @ Facility
```

Policy check = `can(user, permission, resource)`:

1. **Permission:** does the user hold the permission through a platform role, or a facility role in the *resource's owning facility*?
2. **Scope/ownership:** the resource's facility (or donor owner) must match the membership (or the user themselves).
3. **State:** the facility is VERIFIED, the membership is ACTIVE, the user is ACTIVE.
4. **Capability:** the facility has the capability that the action needs (e.g., `can_hold_inventory` for unit actions).

Enforcement points:

- A decorator `@require_permission("blood_request.create")` on every route. A CI test fails if any route lacks either this decorator or an explicit `@public` marker.
- **Scoped repositories:** list queries take a `Scope` object built from the caller's memberships. There's no un-scoped query method in facility repositories, so BOLA is structurally hard to introduce.
- Resources outside the caller's scope return **404** (not 403) to avoid revealing that they exist.

L.3 Roles

Role	Scope	Label
DONOR	PLATFORM (self)	[CONFIRMED]
HOSPITAL_REQUESTER	FACILITY	[CONFIRMED] "hospital staff"
HOSPITAL_ADMIN	FACILITY	[PROPOSED]
BLOOD_BANK_OFFICER	FACILITY	[CONFIRMED] "blood bank staff"
BLOOD_BANK_MANAGER	FACILITY	[PROPOSED]
PLATFORM_ADMIN	PLATFORM	[CONFIRMED]
CLINICAL_CONFIG_APPROVER	PLATFORM	[PROPOSED] (separation of duties for clinical rules)
AUDITOR (read-only audit + reports)	PLATFORM	[FUTURE]

Which staff role may approve requests and release units is **[VALIDATE AS-17]**. Because roles map to permissions in data, changing that later is a data migration, not a code change.

L.4 Facility onboarding

1. Self-registration creates the user (PENDING_VERIFICATION), the facility (PENDING) and a membership (HOSPITAL_ADMIN or BLOOD_BANK_MANAGER, status INVITED).
2. Admin verifies the facility, which activates the founding membership.
3. The facility admin/manager invites staff (email/SMS invite code). Staff set passwords and verify their phone, and the membership becomes ACTIVE.
4. Platform admins **cannot** create memberships inside facilities, except to recover a facility whose founding admin has left (audited, MFA step-up).

M. Permission matrix

Legend:  allowed ·  own facility only ·  self only ·  with step-up MFA · — denied

Permission code	Donor	Hosp. Requester	Hosp. Admin	BB Officer	BB Manager	Platform Admin	Clinical Config Approver
profile.read_update							
donor_profile.manage_own		—	—	—	—	—	—
donor.eligibility.view_own		—	—	—	—	—	—
donor.lookup (exact phone/ID match to record donation)	—	—	—			—	—
donor.blood_group.confirm	—	—	—			—	—
donor.deferral.record	—	—	—			—	—
donation.record	—	—	—			—	—

Permission code	Donor	Hosp. Requester	Hosp. Admin	BB Officer	BB Manager	Platform Admin	Clinical Config Approver
donation.view	 own history	—	—	♦	♦	✓ (aggregate + audited detail)	—
facility.update_profile	—	—	♦	—	♦	✓	—
facility.membership.manage	—	—	♦	—	♦	⚠ recovery only	—
facility.verify	—	—	—	—	—	⚠	—
supply_link.manage	—	—	—	—	—	✓	—
storage_location.manage	—	—	—	—	♦	—	—
stock_threshold.manage	—	—	—	—	♦	—	—
inventory.view_units	—	—	—	♦	♦	✓ read	—
inventory.view_network_availability (aggregates of linked suppliers)	—	♦	♦	♦	♦	✓	—
unit.register (QUARANTINED)	—	—	—	♦	♦	—	—
unit.register_release_d	—	—	—	♦ [VALIDATE]	♦	—	—
unit.release (quarantine → available)	—	—	—	♦ [VALIDATE AS-17]	♦	—	—
unit.discard / unit.quarantine	—	—	—	♦	♦	—	—
unit.correct (immutable fields, with reason)	—	—	—	—	♦	—	—
stock_take.perform / .approve	—	—	—	♦ / —	♦ / ♦	—	—
blood_request.create / .cancel / .close / .route	—	♦	♦	—	—	—	—
blood_request.view	—	♦ (requesting)	♦	♦ (routed-to)	♦	✓	—
blood_request.receive	—	♦	♦	—	—	—	—
blood_request.respond (accept/decline)	—	—	—	♦ [VALIDATE AS-17]	♦	—	—
allocation.manage (reserve/release)	—	—	—	♦	♦	—	—
allocation.issue	—	—	—	♦	♦	—	—
appeal.view	—	status + counts for linked requests	same	♦	♦	✓	—
appeal.manage (edit, preview, launch, waves, close)	—	—	—	—	♦	—	—

Permission code	Donor	Hosp. Requester	Hosp. Admin	BB Officer	BB Manager	Platform Admin	Clinical Config Approver
appeal.view_candidates (masked; revealed after acceptance)	—	—	—	♦	♦	—	—
appeal.respond	👤	—	—	—	—	—	—
appointment.check_in / .no_show	—	—	—	♦	♦	—	—
notification.read_own	👤	👤	👤	👤	👤	👤	👤
notification.delivery_admin	—	—	—	—	—	✅	—
rule_set.view	—	✅ (active only)	✅	✅	✅	✅	✅
rule_set.draft_edit	—	—	—	—	—	✅	✅
rule_set.record_validation	—	—	—	—	—	—	⚠️
rule_set.activate	—	—	—	—	—	—	⚠️ (cannot activate a set they drafted, when > 1 approver exists)
user.suspend / .reactivate	—	—	—	—	—	⚠️	—
platform_role.grant	—	—	—	—	—	⚠️	—
report.view	—	♦ own facility	♦	♦	♦	✅ network	—
audit.view	—	—	—	—	♦ own facility events	✅	—

The matrix is also the source for a **parametrised authorization test suite** (§§): every endpoint × every role × in-scope/ out-of-scope resource.

R. Security and privacy model

R.1 Threat model summary (STRIDE-lite) [PROPOSED]

Threat	Example	Controls
Spoofing	Fake hospital registers to see availability / create requests	Facility verification before any operational permission; KMHFL code; out-of-band verification; unverified facilities see nothing operational
Spoofing	Stolen staff credentials	MFA (admins now, staff V1), refresh-token reuse detection, new-sign-in notifications, session revocation
Tampering	Staff alters unit expiry or history	Immutable unit fields + correction events with reason; append-only <code>unit_events</code> / <code>audit_events</code> with DB-level UPDATE/DELETE revocation and a hash chain
Tampering	Client sends <code>eligible: true</code> or <code>status: COMPLETED</code>	Server derives all state; request schemas don't accept computed fields (unknown fields → 400)
Repudiation	"I didn't issue that unit"	Actor + timestamp on every event; audit hash chain
Info disclosure	IDOR on <code>/blood-requests/{id}</code>	Scoped repositories, 404 on out-of-scope, UUIDs, authz test matrix
Info disclosure	PHI in SMS/logs	No patient identifiers collected; template variable allow-lists; log redaction filter; PHI warning on free-text fields
Info disclosure	Donor locations exposed	Coordinates rounded to ~1 km; donors masked until acceptance; banded counts

Threat	Example	Controls
DoS	SMS pumping via OTP endpoint (costly in Kenya)	Per-number/IP/global OTP limits, CAPTCHA fallback after thresholds [PROPOSED], daily SMS spend cap alarm
DoS	Alert storm to donors	Staff-launched appeals, one active appeal per target, contact caps, cooldown
Elevation	Officer grants self manager	Only HOSPITAL_ADMIN/BB_MANAGER manage memberships; users can't modify their own memberships; audited
Elevation	Unvalidated clinical rules activated	Two-role separation, MFA step-up, REQUIRE_VALIDATED_RULES guard, audit

R.2 Application security controls [PROPOSED]

- Input validation at the API boundary (marshmallow schemas, `unknown=RAISE`), plus DB constraints as the final guard.
- SQLAlchemy parameterised queries only. Raw SQL is allowed only in the repository layer with bound parameters (lint rule).
- Response schemas are explicit allow-lists (e.g., the user schema never serialises `password_hash`).
- Security headers: HSTS (1 year, preload once stable), a CSP with no `unsafe-inline` for scripts, `X-Content-Type-Options`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy` (geolocation only on pages that need it), `frame-ancestors 'none'`.
- CORS: explicit allow-list of the web origin(s) per environment; no wildcard with credentials.
- Rate limiting: edge (reverse proxy) plus application-level for auth/OTP/webhooks (Flask-Limiter with Postgres-backed counters in the MVP, Redis if scaling needs it).
- Error responses: generic messages with a `request_id`; stack traces never leave the server.
- Dependency scanning (pip-audit, npm audit), SAST (bandit, semgrep), secret scanning (gitleaks) in CI; Renovate/Dependabot for updates.
- Secrets: `.env` locally (`.env.example` committed); AWS Secrets Manager/SSM in hosted environments; rotation procedure documented.
- Field-level encryption (AES-GCM, app-managed key): TOTP secrets. Future candidates: national ID numbers if ever needed.
- Transport: TLS 1.2+ everywhere, including app ↔ DB (`sslmode=verify-full`).

R.3 Privacy model [PROPOSED][VALIDATE AS-40–AS-43 with legal counsel]

Legal context (to be confirmed by counsel): Kenya Data Protection Act 2019 and its 2021 regulations; health data is *sensitive personal data*. Likely obligations: registration with the Office of the Data Protection Commissioner (ODPC), a DPIA for high-risk processing, a lawful basis and consent records, data-subject rights, breach notification to the ODPC within 72 hours, and possible data-localisation requirements. Kenya's Digital Health Act 2023 may impose further obligations on digital health solutions. **None of this is legal advice. Every item needs confirmation.**

Data class	Examples	Stored?	Who can see
Identity	name, phone, email, DOB	Yes (minimum)	self; blood bank staff only for accepted appeal donors, donation recording and lookups (audited)
Donor health-related	blood group, deferral <i>category</i> , eligibility result, donation dates	Yes (minimum; category codes, not clinical detail)	self; collection-site staff
Laboratory results	infectious markers, haemoglobin values	No	—
Patient data	patient name, ID, diagnosis	No (ADR-013)	—
Operational	requests, units, allocations	Yes	scoped facilities; admin
Location	facility exact; donor rounded ~1 km	Yes	donor: self; others see only distance bands
Audit	actor, action, entity	Yes	admin; facility manager for their own facility's events

Data class	Examples	Stored?	Who can see
Technical	IP address	Truncated/hashed only, 90-day retention	admin

Data-minimisation decisions:

- DOB instead of age; sex collected only if a validated rule needs it; no national ID number in the MVP [VALIDATE AS-33: donor identity verification method].
- Donor precise location is never stored. The browser geolocation result is rounded client-side **and** server-side.
- Free-text fields are limited in length, carry PHI warnings, and are excluded from logs and notifications.

Data-subject rights [PROPOSED][VALIDATE AS-42]:

- Export: `GET /me/data-export` (JSON) is generated as a job.
- Deletion: the donor account is deactivated and identity fields are pseudonymised after a retention period.
Donation and unit records are kept (linked to a pseudonymous ID) because traceability retention obligations may apply. That conflict needs a legal and blood-service ruling.

Retention (placeholders to validate): notifications 12 months; login attempts 30 days; audit events ≥ 7 years [VALIDATE]; unit/donation records per blood-service retention rules [VALIDATE].

Data residency: see AS-40 and ADR-015. **Hosting location must be decided before the pilot, not after.**

06 — N. API Specification (v1)

The machine-readable OpenAPI 3.1 document will be **generated from code** (flask-smorest, ADR-014) and published at `/api/v1/openapi.json` in non-production environments. This file is the design contract that the implementation must satisfy.

N.1 Conventions [PROPOSED]

Topic	Rule
Base path	<code>/api/v1</code> . Breaking changes → <code>/api/v2</code> , with v1 kept for at least 6 months once mobile clients exist
Format	JSON, UTF-8, <code>snake_case</code> fields, ISO-8601 UTC timestamps (<code>2026-09-30T14:05:00Z</code>), UUID string IDs
Auth	Authorization: Bearer <code><access_token></code> unless marked public
Success envelope	Single resource: the object itself. Lists: <code>{"data": [...], "page": {"next_cursor": "...", "limit": 25}}</code>
Pagination	Cursor-based (<code>?limit=25&cursor=...</code>), default 25, max 100
Filtering/sorting	Explicit, allow-listed query params (<code>?status=SUBMITTED&sort=-created_at</code>); unknown params → 400
Concurrency	Mutable resources return <code>version</code> ; updates send <code>If-Match: "<version>"</code> → 409 <code>VERSION_CONFLICT</code> if stale
Idempotency	<code>Idempotency-Key</code> header required on: create request, reserve, issue, record donation, register units, launch appeal/wave, appeal response. Replays return the original response for 24 h
Tracing	Every response has <code>X-Request-ID</code> (echoed if supplied by the client and valid)
Rate limits	429 with <code>Retry-After</code>
Time	Server time is authoritative; clients never send <code>created_at</code> /status timestamps

Error format

```
{
  "error": {
    "code": "INVALID_STATE_TRANSITION",
    "message": "Request cannot be cancelled after units have been issued.",
    "details": [{"field": "status", "issue": "ISSUED"}],
    "request_id": "01J..."
  }
}
```

HTTP	code(s)
400	<code>VALIDATION_ERROR</code> , <code>UNKNOWN_FIELD</code> , <code>MALFORMED_REQUEST</code>
401	<code>UNAUTHENTICATED</code> , <code>TOKEN_EXPIRED</code> , <code>MFA_REQUIRED</code>
403	<code>FORBIDDEN</code> (in-scope resource, missing permission), <code>FACILITY_NOT_VERIFIED</code> , <code>STEP_UP_REQUIRED</code>
404	<code>NOT_FOUND</code> (also used for out-of-scope resources)
409	<code>CONFLICT</code> , <code>VERSION_CONFLICT</code> , <code>INVALID_STATE_TRANSITION</code> , <code>UNIT_UNAVAILABLE</code> , <code>DUPLICATE_UNIT_IDENTIFIER</code> , <code>IDEMPOTENCY_KEY_REUSED</code>
422	<code>BUSINESS_RULE_VIOLATION</code> (e.g., <code>INCOMPATIBLE_UNIT</code> , <code>NO_ACTIVE_VALIDATED_RULE_SET</code> , <code>SUPPLIER_NOT_LINKED</code>)
429	<code>RATE_LIMITED</code>
500/503	<code>INTERNAL_ERROR</code> , <code>SERVICE_UNAVAILABLE</code>

N.2 Endpoint catalogue

Permission codes refer to §M. "Scope" says which facility the permission is checked against.

Auth & account

Method & path	Purpose	Auth / permission
POST /auth/register/donor	Create donor user + profile; sends phone OTP	public; rate-limited
POST /auth/register/facility	Create user + facility (PENDING) + founding membership	public; rate-limited
POST /auth/login	→ {access_token, expires_in, mfa_required?} + refresh cookie	public
POST /auth/mfa/verify	Complete login with TOTP/recovery code	partial token
POST /auth/token/refresh	Rotate refresh token, new access token	refresh cookie + CSRF header
POST /auth/logout	Revoke the current refresh family	refresh cookie + CSRF
POST /auth/phone/send-code, /auth/phone/verify	Phone OTP	authenticated (or registration context)
POST /auth/email/verify	Consume email token	public (token)
POST /auth/password/forgot, /auth/password/reset	Reset flow	public; enumeration-safe
POST /auth/invitations/accept	Staff accepts invite code, sets password	public (code)
POST /auth/mfa/totp/enroll, /auth/mfa/totp/confirm, DELETE /auth/mfa/totp	MFA management	self (+step-up for removal)
POST /auth/step-up	Re-verify MFA for sensitive actions	self
GET /me · PATCH /me	Profile (name, language, email change → re-verify)	self
GET /me/memberships	Facilities + roles + permissions (for UI rendering only)	self
GET /me/sessions · DELETE /me/sessions/{session_id}	Session management	self
POST /me/data-export · POST /me/deactivate	Data-subject rights	self [VALIDATE AS-42]

Reference data

Method & path	Purpose	Auth
GET /reference/counties (? include=sub_counties)	Counties/sub-counties	public
GET /reference/blood-components	Active components	authenticated
GET /reference/urgency-levels	From ACTIVE URGENCY rule set	authenticated
GET /reference/reason-codes? category=	Controlled reasons	authenticated
GET /reference/rule-sets/active	Active rule-set IDs, versions, validation status (drives the "unvalidated rules" banner)	authenticated

Donors (self-service)

Method & path	Purpose	Permission
GET/PATCH /me/donor-profile	View/update profile (blood group editable only while SELF_REPORTED)	donor_profile.manage_own

Method & path	Purpose	Permission
PUT /me/donor-profile/availability	{availability, unavailable_until}	same
GET/PUT /me/notification-preferences	Channels, quiet hours, cap	same
GET /me/eligibility	Evaluated now → 3-state + reasons + next date + rule-set status	donor.eligibility.view_own
GET /me/donations	Own donation history (date, site, outcome)	same
GET /me/appeals	Appeals the donor was invited to (active first)	appeal.respond
GET /me/appeals/{appeal_id}	Details: site, window, slots, group needed	same
POST /me/appeals/{appeal_id}/response	{response: ACCEPTED DECLINED, slot_start?, decline_reason_code?}	same; Idempotency-Key
GET /me/appointments · POST /me/appointments/{id}/cancel	Own appointments	same
GET /appeal-links/{token}	Resolve a signed link → minimal appeal info + donor first name	public (token)
POST /appeal-links/{token}/response	Respond without login	public (token); single-use per response type

Donors (staff)

Method & path	Purpose	Permission (scope)
POST /donors/lookup	Body {phone_e164} or {donor_reference} → exact match only, minimal fields; audited (POST keeps identifiers out of URLs/logs)	donor.lookup (acting facility)
POST /donors/{donor_id}/blood-group-confirmation	Set LAB_CONFIRMED ABO/Rh	donor.blood_group.confirm
POST /donors/{donor_id}/deferrals · POST /deferrals/{id}/lift	Record/lift a deferral (category code only)	donor.deferral.record
GET /donors/{donor_id}/eligibility?facility_id=	Staff view at check-in (snapshot saved)	donation.record

Donations

Method & path	Purpose	Permission
POST /donations	{facility_id, donor_id, collected_at, outcome, donation_type, donation_identification_number?, appointment_id?}	donation.record @ facility; Idempotency-Key
GET /donations?facility_id=&from=&to=&outcome=	Facility donation list	donation.view @ facility
GET /donations/{id}	Detail incl. derived units	same

Facilities & admin

Method & path	Purpose	Permission
GET /facilities?capability=&county_id=&q=	Directory of VERIFIED facilities (public fields)	authenticated
GET /facilities/{id} · PATCH /facilities/{id}	Detail / update profile	read: authenticated; write facility.update_profile

Method & path	Purpose	Permission
GET /facilities/{id}/memberships · POST (invite) · PATCH /facilities/{id}/memberships/{mid} (role/status)	Staff management	facility.membership.manage
GET/POST /facilities/{id}/storage-locations · PATCH .../{loc_id}	Storage locations	storage_location.manage
GET /facilities/{id}/supply-links	Suppliers of a hospital / hospitals served by a bank	member of facility
GET /admin/facilities?verification_status=	Verification queue	PLATFORM_ADMIN
POST /admin/facilities/{id}/verification	{decision, method, evidence_reference, notes}	facility.verify (step-up)
POST /admin/supply-links · PATCH /admin/supply-links/{id}	Manage links	supply_link.manage
GET /admin/users?q=&status= · POST /admin/users/{id}/suspend · /reactivate	User administration	user.suspend (step-up)
POST /admin/users/{id}/platform-roles · DELETE .../{role_code}	Grant/revoke platform roles	platform_role.grant (step-up)

Clinical configuration

Method & path	Purpose	Permission
GET /admin/rule-sets?domain=&status= · GET /admin/rule-sets/{id}	List/detail with entries	rule_set.view
POST /admin/rule-sets	New DRAFT (optionally cloned from an existing set)	rule_set.draft_edit
PUT /admin/rule-sets/{id}/entries	Replace entries (DRAFT only; validated per domain schema)	rule_set.draft_edit
POST /admin/rule-sets/{id}/submit	DRAFT → PENDING_VALIDATION (entries frozen)	rule_set.draft_edit
POST /admin/rule-sets/{id}/validation	Record validator name/role/org/source/date → VALIDATED	rule_set.record_validation (step-up)
POST /admin/rule-sets/{id}/activate	VALIDATED → ACTIVE; previous ACTIVE → RETIRED (one transaction)	rule_set.activate (step-up)
GET /admin/rule-sets/{id}/diff?against=	Entry-level diff between versions	rule_set.view

Inventory

Method & path	Purpose	Permission
GET /inventory/units?facility_id=&status=&component=&abo=&rh=&expires_before=&q=	Unit list	inventory.view_units @ facility
POST /inventory/units	Register one unit	unit.register / unit.register_released; Idempotency-Key
POST /inventory/units/bulk	≤ 200 units, atomic; response has per-row errors on failure	same
POST /inventory/units/import/validate	CSV dry-run → row report	same
GET /inventory/units/{id} · GET /inventory/units/{id}/events	Detail + chain of custody	inventory.view_units

Method & path	Purpose	Permission
POST /inventory/units/{id}/release · /quarantine · /discard · /correct	Status actions (reason codes where required)	unit.release / unit.quarantine / unit.discard / unit.correct
GET /inventory/availability?facility_id=	Derived counts	inventory.view_units
GET /inventory/network-availability?requesting_facility_id=&component=&abo=&rh=	Aggregates across linked suppliers, with last_change	inventory.view_network_availability
GET /inventory/expiring?facility_id=&within_hours=	Expiring soon	inventory.view_units
GET/PUT /inventory/thresholds?facility_id=	Low-stock thresholds	view: inventory.view_units; write: stock_threshold.manage
POST /inventory/stock-takes · PUT /inventory/stock-takes/{id}/items · POST /inventory/stock-takes/{id}/complete	Reconciliation	stock_take.perform / .approve

Blood requests & fulfilment

Method & path	Purpose	Permission
POST /blood-requests	Create + route (body below)	blood_request.create @ requesting facility; Idempotency-Key
GET /blood-requests?facility_id=&perspective=requesting supplying&status=&urgency=&sort=	Queue/list	blood_request.view
GET /blood-requests/{id}	Detail with derived counts, current routing, allowed actions	blood_request.view
GET /blood-requests/{id}/timeline	Status history + key events	same
POST /blood-requests/{id}/routing	Re-route after decline {supplier_facility_id}	blood_request.route
POST /blood-requests/{id}/routing/accept · /routing/decline	Supplier response	blood_request.respond @ supplier
GET /blood-requests/{id}/allocation-suggestions	Inventory matcher output with explanations and shortfall	allocation.manage @ supplier
POST /blood-requests/{id}/allocations	Reserve {unit_ids[], confirm_conditional?}	allocation.manage; Idempotency-Key
POST /allocations/{id}/release	Release a reservation {reason_code}	allocation.manage
POST /blood-requests/{id}/issue	Issue {allocation_ids[]}	allocation.issue; Idempotency-Key
POST /blood-requests/{id}/receipts	Hospital confirms receipt {allocation_ids[]}	blood_request.receive
POST /blood-requests/{id}/cancel · /close	{reason_code, note?}	blood_request.cancel / .close
POST /blood-requests/{id}/appeal	Create a DRAFT appeal prefilled from the shortfall	appeal.manage @ supplier

Example: create request

```

POST /api/v1/blood-requests
Idempotency-Key: 7c1e...
{
  "requesting_facility_id": "...",
  "supplier_facility_id": "...",
  "component_code": "RBC",
  "abo": "O", "rh": "POS",
  "units_requested": 2,
  "urgency_code": "URGENT",
  "required_by": "2026-09-30T18:00:00Z",
  "external_reference": "LAB-4471",
  "notes": null
}
→ 201
{
  "id": "...", "reference_code": "REQ-2026-000123", "status": "SUBMITTED", "version": 1,
  "units": {"requested": 2, "reserved": 0, "issued": 0, "received": 0},
  "urgency": {"code": "URGENT", "display_name": "Urgent", "rule_set_status": "VALIDATED"},
  "routing": {"supplier": {"id": "...", "name": "..."}, "status": "PENDING", "routed_at": "..."},
  "allowed_actions": ["cancel"],
  "created_at": "..."
}

```

`allowed_actions` is computed server-side from permissions and state. The UI uses it to show buttons, and the server still re-checks every action.

Mobilisation

Method & path	Purpose	Permission
GET /appeals? facility_id=&status=	List (incl. SUGGESTED)	appeal.view
POST /appeals	Create DRAFT (manual)	appeal.manage
GET /appeals/{id} · PATCH / appeals/{id}	Detail / edit DRAFT or SUGGESTED	appeal.view / appeal.manage
GET /appeals/{id}/target- group-suggestions	Groups suggested from ACTIVE compatibility rule set	appeal.manage
POST /appeals/{id}/preview	Run matcher (no contact) → masked ranked candidates + explanations + counts	appeal.manage
POST /appeals/{id}/waves	Launch next wave {size}	appeal.manage ; step-up not required; Idempotency-Key
POST /appeals/{id}/pause · / resume · /close · /cancel	Lifecycle	appeal.manage
GET /appeals/{id}/candidates? wave=&response=	Masked, or revealed if ACCEPTED	appeal.view_candidates
GET /appeals/{id}/summary	Counts only (hospital view)	appeal.view
GET /appointments? facility_id=&date=	Day list at collection site	appointment.check_in
POST /appointments/{id}/check- in · /no-show	Staff actions	appointment.check_in

Notifications

Method & path	Purpose	Permission
GET /me/notifications?unread=true	In-app inbox	self

Method & path	Purpose	Permission
POST /me/notifications/{id}/read · POST /me/notifications/read-all	Mark read	self
GET /me/notifications/unread-count	Cheap polling endpoint	self
GET /admin/notification-deliveries?status=&channel=&from=	Delivery health	notification.delivery_admin
POST /admin/notification-deliveries/{id}/retry	Manual retry	same
POST /webhooks/sms/{provider} · /webhooks/email/{provider}	Delivery reports	provider signature

Reports, audit, ops

Method & path	Purpose	Permission
GET /reports/donation-trends?facility_id=&from=&to=&granularity=week	[CONFIRMED]	report.view
GET /reports/shortages?...	Threshold breaches, unfulfilled/partial requests	report.view
GET /reports/donor-activity?...	Appeals, contacts, acceptance/show-up/donation conversion (aggregate)	report.view
GET /reports/request-performance?...	Time to accept/issue/complete (median, p90), fulfilment rate	report.view
GET /reports/wastage?...	Expired/discarded vs issued, by component	report.view
GET /reports/{name}.csv?...	CSV export (streamed; audited)	report.view
GET /admin/audit-events?entity_type=&entity_id=&actor_id=&action=&from=&to=	Audit viewer	audit.view
GET /admin/audit-events/verify-chain?from_id=&to_id=	Hash-chain verification	PLATFORM_ADMIN
GET /healthz (liveness) · GET /readyz (DB + migrations at head + worker heartbeat fresh)	Ops	public (no details) / internal network

N.3 Real-time strategy (MVP) [PROPOSED, ADR-012]

Polling with TanStack Query `refetchInterval`: 15 s for request queues and request detail when non-terminal; 30 s for notification unread count; 60 s for inventory dashboards; paused when the tab is hidden. The endpoints are cheap, indexed queries. Upgrade path: Server-Sent Events fed by outbox events (V1), with the same query keys invalidated on push.

07 — Frontend Route Map & Backend Module Map

O. Frontend route map

O.1 Frontend stack [PROPOSED; each dependency justified]

Dependency	Reason
React 18+ with TypeScript (strict)	[CONFIRMED]
Vite	Fast builds, simple config, first-class TS
React Router (data routers)	Nested role layouts, route-level code splitting, loaders for guards
TanStack Query	Server-state cache, polling (ADR-012), invalidation, retry; avoids a global state library
react-hook-form + zod	Accessible, performant forms; zod mirrors server validation for UX only
Tailwind CSS	Consistent design tokens; small CSS output
Radix UI primitives (or shadcn/ui built on them)	Accessible dialogs, menus and tabs without writing ARIA by hand
openapi-typescript + openapi-fetch	Types generated from the backend OpenAPI spec, so no hand-written DTO drift
react-i18next	Externalised strings from day one (NFR-I1)
date-fns-tz	Display in Africa/Nairobi
Vitest, Testing Library, MSW, Playwright, axe-core	Tests (\$\$)

Not added: Redux/Zustand (server state covers our needs), chart libraries until the reports need them (then one small library), a map SDK in the MVP (a static link to Maps for directions suffices; a picker can come later) [PROPOSED].

O.2 Routes

Layouts: `PublicLayout`, `DonorLayout`, `FacilityLayout` (facility switcher when the user has more than one membership), `AdminLayout`. Guards check authentication and permissions from `/me/memberships`, for UX only.

Path	Page	Role	Label
<code>/</code>	Landing: what HemaNet is, prototype disclaimer	public	[PROPOSED]
<code>/login</code> , <code>/login/mfa</code>	Sign in	public	[CONFIRMED]
<code>/register/donor</code>	Donor registration (multi-step: account → profile → consent → phone OTP)	public	[CONFIRMED]
<code>/register/facility</code>	Facility registration	public	[CONFIRMED]
<code>/verify-phone</code> , <code>/verify-email/:token</code>	Verification	public/auth	[PROPOSED]
<code>/forgot-password</code> , <code>/reset-password/:token</code>	Reset	public	[PROPOSED]
<code>/invite/:code</code>	Staff invite acceptance	public	[PROPOSED]
<code>/r/:token</code>	Donor appeal response via SMS/email link (no login)	public	[PROPOSED]
<code>/privacy</code> , <code>/terms</code>	Legal notices	public	[PROPOSED] [VALIDATE]
<code>/select-facility</code>	Choose the active facility context	staff	[PROPOSED]
<code>/account</code> , <code>/account/security</code>	Profile, password, MFA, sessions	all	[PROPOSED]
<code>/notifications</code>	Inbox	all	[CONFIRMED]

Path	Page	Role	Label
Donor			
/donor	Dashboard: eligibility card (3-state + next date), active appeals, upcoming appointment, impact summary (donations count)	donor	[CONFIRMED]
/donor/appeals, /donor/appeals/:appealId	Alerts and accept/decline + slot	donor	[CONFIRMED]
/donor/appointments	Upcoming/past	donor	[PROPOSED]
/donor/donations	History	donor	[CONFIRMED]
/donor/profile	Profile, blood group (source badge), location, availability	donor	[CONFIRMED]
/donor/preferences	Channels, quiet hours, cap, consents	donor	[PROPOSED]
Hospital (/f/:facilityId/... when the facility can request blood)			
/f/:fid/hospital	Dashboard: open requests by urgency with age timers, recent updates, supplier availability snapshot	requester	[CONFIRMED]
/f/:fid/requests	List with filters	requester	[CONFIRMED]
/f/:fid/requests/new	Request form with live availability panel	requester	[CONFIRMED]
/f/:fid/requests/:rid	Detail: status stepper, timeline, units issued/received, appeal summary, actions	requester	[CONFIRMED]
/f/:fid/availability	Network availability of linked suppliers	requester	[CONFIRMED]
Blood bank (when the facility can hold inventory)			
/f/:fid/blood-bank	Dashboard: incoming queue by urgency, low-stock alerts, expiring within window, suggested appeals, today's appointments	officer/manager	[CONFIRMED]
/f/:fid/incoming	Incoming request queue	officer	[CONFIRMED]
/f/:fid/incoming/:rid	Accept/decline, allocation suggestions, reserve, issue	officer	[CONFIRMED]
/f/:fid/inventory	Availability matrix (component × group), drill-down to units	officer	[CONFIRMED]
/f/:fid/inventory/units	Unit list + filters	officer	[CONFIRMED]
/f/:fid/inventory/units/new	Register (single/scan mode)	officer	[CONFIRMED]
/f/:fid/inventory/import	CSV import with dry-run	officer	[PROPOSED]
/f/:fid/inventory/units/:uid	Unit detail + chain of custody	officer	[CONFIRMED]
/f/:fid/inventory/expiring	Expiring soon	officer	[CONFIRMED]
/f/:fid/inventory/stock-takes, /.../:stid	Reconciliation	officer/manager	[PROPOSED]
/f/:fid/thresholds	Low-stock thresholds	manager	[PROPOSED]
/f/:fid/donations, /f/:fid/donations/new	Record donation (lookup donor → eligibility check → outcome → register units)	officer	[CONFIRMED]
/f/:fid/appointments	Today's check-in list	officer	[PROPOSED]
/f/:fid/appeals, /f/:fid/appeals/new, /f/:fid/appeals/:aid	Appeals: edit, preview with explanations, waves, responses	manager	[CONFIRMED]
Any facility			

Path	Page	Role	Label
/f/:fid/staff	Memberships & invites	hospital admin / BB manager	[PROPOSED]
/f/:fid/settings	Facility profile, storage locations, supply links (read-only)	admin/manager	[PROPOSED]
/f/:fid/reports	Facility reports	staff	[CONFIRMED]
Admin			
/admin	Network overview: open requests by urgency, unacknowledged escalations, shortages, delivery failures, pending verifications	admin	[CONFIRMED]
/admin/facilities, /admin/facilities/:id	Verification queue / detail	admin	[CONFIRMED]
/admin/supply-links	Manage links	admin	[PROPOSED]
/admin/users, /admin/users/:id	Users	admin	[CONFIRMED]
/admin/rule-sets, /admin/rule-sets/:id	Clinical configuration lifecycle, diff, validation record	admin / approver	[PROPOSED]
/admin/notifications	Delivery health	admin	[PROPOSED]
/admin/audit	Audit viewer + chain verification	admin	[CONFIRMED]
/admin/reports	Network reports	admin	[CONFIRMED]

O.3 UX rules [PROPOSED]

- **Urgency visibility:** each urgency has a colour **plus** an icon and text label (never colour alone), and the elapsed time since submission is shown next to it. CRITICAL items pin to the top.
- **Global banners:** "Development system: clinical rules are DEVELOPMENT MOCK / PENDING VALIDATION" whenever any active rule set isn't validated; "Offline / data may be stale" when polling fails.
- **Freshness:** availability shows "updated 3 min ago".
- **Empty states** explain what to do next ("No units registered yet. Register units or import a delivery").
- **Error states** show the `request_id` for support.
- **Forms:** labels always visible; errors in text linked by `aria-describedby`; focus moves to the error summary on submit; an `aria-live` region announces async status.
- **Mobile-first layouts:** donors mainly use phones. Staff dashboards must work on a tablet.

O.4 Frontend structure

```

frontend/src/
  app/           # providers (QueryClient, i18n, router), error boundary
  routes/       # route tree, guards
  layouts/
  features/
    auth/ donor/ hospital/ blood-bank/ admin/ inventory/ requests/ appeals/ notifications/ reports/
    facilities/ rule-sets/
      api.ts (typed hooks) components/ pages/ schemas.ts (zod) __tests__/
    components/ # shared design-system components (Button, StatusBadge, UrgencyBadge, DataTable,
    Stepper...)
  services/     # api client (openapi-fetch), auth token manager, refresh logic
  types/        # generated openapi types (do not edit)
  constants/ utils/ hooks/ i18n/ (en.json, sw.json)

```

P. Backend module map

P.1 Structure change vs the suggested layout (per master instruction §39)

- **Current (suggested):** layer-first top-level folders (`models/`, `routes/`, `services/`, `repositories/`) *and* domain folders (`donors/`, `inventory/` ...) side by side.
- **Problem:** mixing the two means every feature touches 5 top-level folders, and domain boundaries blur because any service can import any model.
- **Proposed: domain-first modules** with the same internal layers inside each module, plus a thin shared core.
- **Impact:** only folder organisation changes; the layers still exist. **Migration:** none (no code exists yet).

```
backend/
├── app/
│   ├── __init__.py          # create_app() factory
│   ├── config.py           # typed settings from env (pydantic-settings), per-environment
│   ├── extensions.py       # db, migrate, api (flask-smorest), limiter
│   ├── core/              # shared kernel (no domain logic)
│   │   ├── errors.py      # exception types → error envelope
│   │   ├── authz.py       # require_permission, Scope, policy registry
│   │   ├── idempotency.py
│   │   ├── pagination.py
│   │   ├── outbox.py      # publish_event() within current txn
│   │   ├── clock.py       # injectable now() for tests
│   │   ├── logging.py     # structlog JSON, redaction filter, request id
│   │   └── security_headers.py
│   └── modules/
│       ├── identity/      # users, auth, tokens, MFA, OTP, roles/permissions, memberships
│       ├── facilities/    # facilities, verification, supply links, storage locations
│       ├── geo/           # counties, distance (haversine now, PostGIS later)
│       ├── clinical_config/ # rule sets, components, compatibility/eligibility/spec/urgency entries
│       ├── donors/        # profiles, consents, preferences, deferrals, blood-group confirmation
│       ├── eligibility/   # evaluator + rule-type implementations (pure)
│       ├── donations/     # donation recording
│       └── inventory/     # units, unit state machine, events, availability, thresholds, stock
├── takes
│   ├── requests/         # blood requests, routing, allocations, request state machine
│   ├── matching/         # inventory matcher, donor matcher, scoring, explanations (pure core)
│   ├── mobilisation/     # appeals, waves, candidates, responses, appointments, response links
│   ├── notifications/    # service, policy, templates, providers/, webhooks
│   ├── audit/            # audit writer, hash chain, viewer, verifier
│   └── reporting/        # read-only report queries
│   ├── jobs/             # job registry, scheduler definitions, worker loop
│   └── integrations/      # placeholder README only (FHIR/DHIS2/KMHFL later)
├── migrations/           # Alembic
├── seeds/
│   └── reference/        # counties, components, reason codes, roles/permissions (safe for all
├── envs)
│   └── development_mock/ # MOCK rule sets, demo facilities/donors – refused outside dev
├── tests/               # unit/ integration/ api/ authz/ concurrency/ migrations/
├── wsgi.py              # api entry
├── worker.py            # worker entry
├── pyproject.toml       # (dependencies pinned via lock file)
└── Dockerfile
```

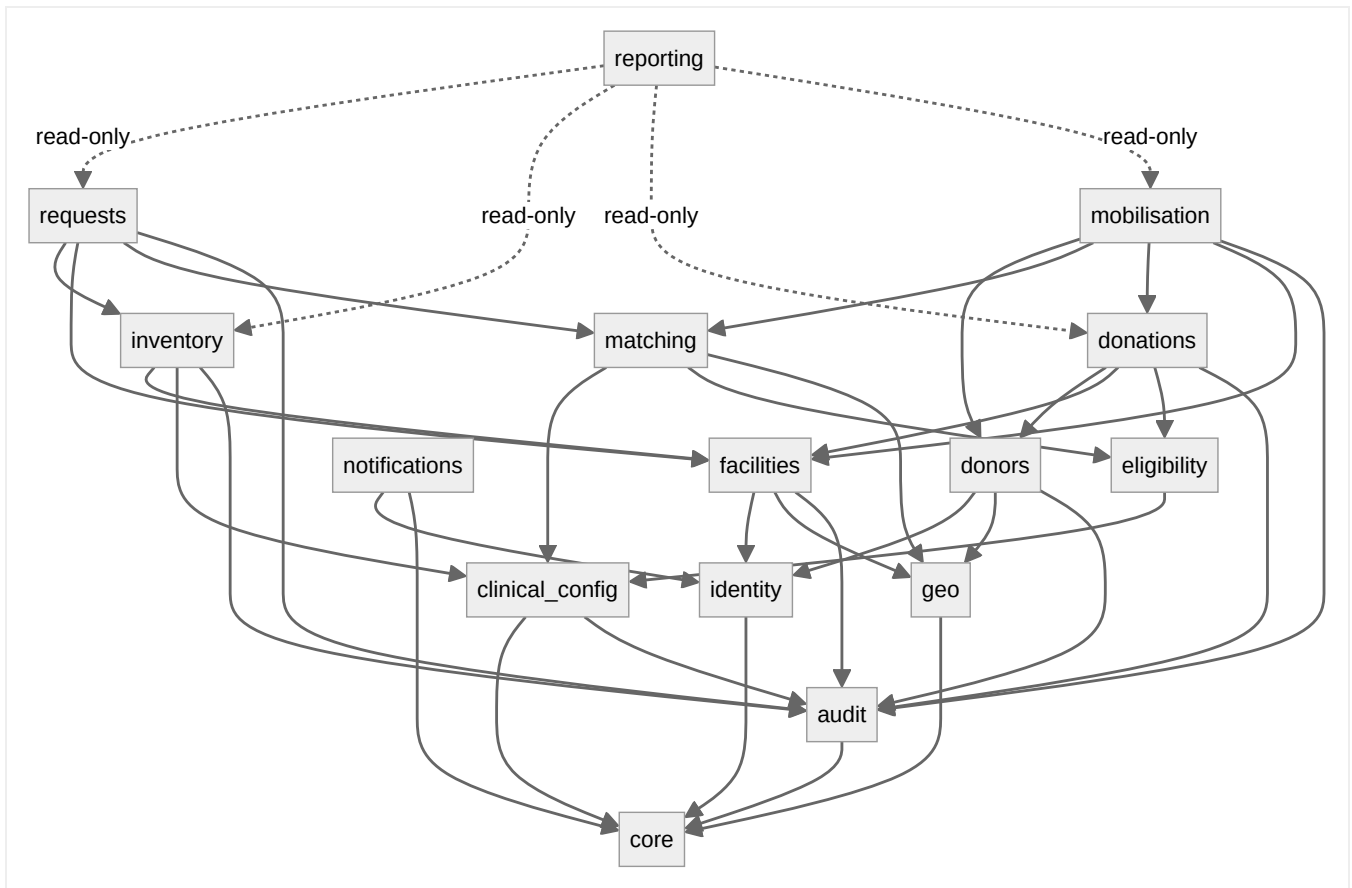
Inside each module:

```

modules/requests/
models.py          # SQLAlchemy models (owned by this module only)
schemas.py         # marshmallow request/response schemas (API boundary)
repository.py      # scoped queries; no business rules
domain/           # pure logic: state_machine.py, recompute_status.py (no Flask, no DB)
service.py         # use cases: transaction boundary, authz-aware, emits events, writes audit
policies.py        # permission + scope rules for this module's resources
routes.py          # thin HTTP layer: parse → service → serialize
events.py          # event type definitions
handlers.py        # outbox event handlers (worker side)

```

P.2 Module dependency rules [PROPOSED; enforced with `import-linter` in CI]



- `routes` → `service` → `repository/domain`. Routes never touch models directly.
- Modules call each other **only through `service.py` public functions** or through events. They never import another module's models or repositories.
- `matching` and `eligibility` domain code is pure (inputs in, results out), which makes it testable and reusable in future batch analytics.
- Notifications are triggered only by outbox events handled in `notifications/handlers.py`, so there's no import from domain modules into providers.

P.3 Backend dependencies [PROPOSED; each with a reason]

Package	Reason
Flask 3, flask-smorest, marshmallow	[CONFIRMED] Flask; smorest provides blueprints, validation and OpenAPI generation
SQLAlchemy 2, Alembic (Flask-Migrate), psycopg 3	ORM + migrations + Postgres driver
argon2-cffi	Password hashing
PyJWT	Access tokens

Package	Reason
pyotp	TOTP
pydantic-settings	Typed env configuration
structlog	Structured logs
Flask-Limiter	Application rate limits
phonenumbers	E.164 validation
sentry-sdk (with scrubbing) or a self-hosted equivalent	Error tracking [VALIDATE AS-40 data transfer]
gunicorn	WSGI server
pytest, pytest-cov, factory-boy, hypothesis, freezegun (or core.clock)	Tests
ruff, mypy, bandit, pip-audit, import-linter	Quality gates
Provider SDKs (Africa's Talking / Twilio / SendGrid)	Only inside notifications/providers/ ; or plain HTTPS via httpx to limit dependencies

08 — Q. Audit and Traceability Model

HemaNet keeps two complementary append-only records. Both are [CONFIRMED] requirements (master instruction §24, decision 5); their design is [PROPOSED].

	audit_events	unit_events
Question answered	<i>Who did what, when, from where, and was it allowed?</i>	<i>Where has this unit been, and what happened to it?</i>
Scope	All security-relevant and business-significant actions, including denied attempts	Blood-unit lifecycle only (chain of custody)
Consumers	Admins, investigations, compliance	Blood bank staff, recalls (future), traceability
Written by	<code>audit.record()</code> in services and the authz layer	Inventory state machine

Q.1 audit_events schema

Column	Notes
id bigint identity PK	monotonic order
occurred_at timestamptz	server clock
actor_type enum (USER, SYSTEM, WORKER, PROVIDER_WEBHOOK)	
actor_user_id uuid null	
actor_facility_id uuid null	acting facility context
action text	controlled vocabulary, e.g. <code>blood_request.created</code> , <code>unit.released</code> , <code>rule_set.activated</code> , <code>auth.login_failed</code> , <code>authz.denied</code>
outcome enum (SUCCESS, DENIED, FAILED)	
entity_type, entity_id	
request_id text	correlates with logs
ip_prefix_hash text, user_agent_family text	minimal technical context
changes jsonb null	field-level <code>{field: [old, new]}</code> for allow-listed non-sensitive fields; sensitive fields are recorded as <code>"<changed>"</code>
metadata jsonb null	e.g. reason code, rule-set version
prev_hash bytea, hash bytea	<code>hash = SHA-256(prev_hash canonical_json(row without hash))</code>

Indexes: (entity_type, entity_id, id), (actor_user_id, id), (action, occurred_at), (occurred_at).

Q.2 Immutability [PROPOSED, ADR-011]

1. The application DB role has `INSERT`, `SELECT` only on `audit_events` and `unit_events`. Migrations run as a separate owner role.
2. A `BEFORE UPDATE OR DELETE` trigger raises an exception, as a backstop against accidental grants.
3. Hash chain: inserts are serialised through a single-row chain-head lock (`SELECT ... FOR UPDATE` on `audit_chain_head`). That's acceptable at pilot write volumes; if contention appears, move to per-partition chains.
4. The `verify-chain` endpoint and a nightly job recompute hashes and alert on mismatch.
5. [FUTURE] Periodically anchor the latest hash externally (e.g., write it to write-once object storage, or email a digest to a designated officer). This gives tamper-evidence without blockchain.
6. Retention: ≥ 7 years placeholder [VALIDATE AS-41]. Archive older partitions to encrypted object storage; the chain is preserved across archives.

Q.3 Mandatory audit actions (MVP)

Domain	Actions
Auth	login success/failure, MFA enroll/disable, password change/reset, token-family reuse detected, session revoked
Authz	every DENIED decision on a protected endpoint (sampled if volume is abusive)
Identity	user suspended/reactivated, platform role granted/revoked, membership invited/activated/role-changed/revoked
Facilities	registered, verification decision (method, evidence ref), supply link changed, threshold changed
Clinical config	rule set created, entries changed, submitted, validation recorded, activated, retired
Donors (staff actions)	donor lookup (who looked up whom), blood group confirmed, deferral recorded/lifted, donation recorded/corrected
Donors (self)	consent granted/withdrawn, profile blood group changed, data export requested, deactivation
Inventory	unit registered (single/bulk with count), released, quarantined, discarded, corrected, stock-take completed
Requests	created, routed, accepted, declined, allocation reserved/released, issued, received, cancelled, closed, expired
Mobilisation	appeal created/edited/launched/wave sent/paused/closed, candidate revealed (identity viewed), check-in, no-show
Notifications	manual retry; provider switch
Reports	CSV export (report name, filters)

Q.4 Traceability chain



From a unit, staff can trace forward (which request/facility received it) and backward (which donation and donor, visible only to collection-site staff with permission). That's the foundation for recalls [FUTURE] and donor-impact messaging [FUTURE][VALIDATE AS-43].

The request side stops at the hospital's receipt. Patient-level linkage (which patient was transfused) is **deliberately outside HemaNet** in the MVP. It belongs in hospital systems, and any future integration must be validated for privacy and legal basis.

Q.5 `unit_events` event types

REGISTERED, OPENING_BALANCE, RELEASED, QUARANTINED, RESERVED, RESERVATION_RELEASED, ISSUED, RECEIVED, EXPIRED, DISCARDED, CORRECTED, STOCK_TAKE_CONFIRMED, STOCK_TAKE_MISSING, and reserved for the future: TRANSFER_DISPATCHED, TRANSFER_RECEIVED, LABEL_PRINTED, TEMPERATURE_EXCURSION, RECALLED, FINAL_DISPOSITION_RECORDED.

09 — Testing, Deployment, Monitoring, Disaster Recovery

S. Testing strategy [CONFIRMED mandatory; PROPOSED design]

S.1 Layers

Layer	Tooling	What	Gate
Pure domain unit tests	pytest, hypothesis	State machines (request, unit, appeal), eligibility evaluator per rule type, compatibility lookup, scoring, explanation rendering, status recomputation	Every PR
Service/ integration tests	pytest + real PostgreSQL (Docker/ Testcontainers; not SQLite, because constraints, locking and partial indexes matter) with transaction rollback per test	Use cases end-to-end within the backend, including outbox events and audit writes	Every PR
API tests	Flask test client	Status codes, error envelope, validation, pagination, idempotency, If-Match	Every PR
Authorization matrix	parametrised from a YAML copy of SM	Every endpoint × every role × {own facility, other facility, unverified facility, suspended membership, anonymous}	Every PR; CI fails if an endpoint isn't in the matrix
Concurrency	pytest with threads/processes against Postgres	Double allocation of one unit; simultaneous issue and expiry; concurrent response-link submissions; idempotency races	Every PR (fast subset), nightly (full)
Migration tests	Alembic	upgrade head from empty; downgrade/ upgrade the last migration; upgrade on a seeded snapshot; model/migration drift check	Every PR
Job tests	injectable clock	expiry, reservation timeout, escalation, threshold → suggested appeal	Every PR
Notification tests	fake providers	template variable allow-list (PHI guard), retries/backoff, permanent failure, suppression reasons, webhook signature failures	Every PR
Frontend component	Vitest + Testing Library + MSW	forms (validation, error announcements), status badges, guards, token refresh	Every PR
Accessibility	vitest-axe on components; @axe-core/ playwright on pages	zero serious/critical violations	Every PR
E2E	Playwright against docker-compose stack with dev mock providers	Critical workflows below	Every PR (smoke), nightly (full)
Security	bandit, semgrep, pip-audit, npm audit, gitleaks; OWASP ZAP baseline on staging		PR + weekly
Performance	k6	request queue, availability, matching preview at pilot data volumes (NFR-PF1)	Before pilot, then monthly
Restore test	scripted	restore the latest backup into an isolated environment, run smoke tests	Quarterly (SV)

Coverage: ≥ 90% line/branch on domain/ packages of inventory, requests, matching, eligibility, mobilisation, identity; no vanity global target.

S.2 Critical E2E workflows

1. Donor registers → verifies phone → sees eligibility "Requires review" (no history, self-reported group) with an explanation.
2. Facility registers → admin verifies → manager invites officer → officer activates.
3. Hospital creates URGENT RBC request → supplier accepts → allocation suggestions (explained) → reserve 2 → issue → hospital confirms receipt → COMPLETED; timeline and audit correct.
4. Request with a shortfall → appeal from shortfall → preview → launch wave → donor (via link) accepts slot → staff check-in → record donation → register unit (QUARANTINED) → release → allocate to the original request.
5. Donor declines; donor cooldown prevents re-contact; next wave excludes them.
6. Low-stock threshold breach → suggested appeal visible to manager, no SMS sent automatically.
7. Rule set lifecycle: draft → submit → validation recorded → activate (MFA step-up) → banner disappears.

S.3 Dangerous cases (explicit test list)

Case	Expected
Incompatible unit selected (per active rule set)	422 INCOMPATIBLE_UNIT ; nothing reserved
No row in compatibility table	Treated as incompatible
No ACTIVE rule set / ACTIVE is a dev mock in pilot env	422 NO_ACTIVE_VALIDATED_RULE_SET ; matching/eligibility disabled with a clear UI
Ineligible donor	Not in candidates; explanation available to staff on lookup
Unknown DOB / blood group	REQUIRES_REVIEW, never ELIGIBLE
Expired unit (job not yet run)	Excluded from suggestions; reserve/issue refused
Unit expires while RESERVED	Job releases the allocation; request recomputed; both sides alerted
Insufficient inventory	shortfall reported; partial allocation allowed; request PARTIALLY_ALLOCATED
Duplicate request submit (double click / retry)	Idempotency returns the same 201 body
Duplicate unit identifier	409
Duplicate donation record (same appointment/DIN)	409
Two officers reserve the same unit concurrently	Exactly one succeeds; the other gets 409 UNIT_UNAVAILABLE
Cancel after issue	409; must use close
Officer at bank B views a request routed to bank A	404
Hospital user reads unit list	403/404
Donor opens another donor's appeal response link token	Token bound to candidate; invalid → 404
Suspended user with a valid access token	Rejected on the next request (token_version check)
Revoked membership	Loses facility access immediately
Refresh token reuse	Family revoked; security notification
Expired/invalid JWT, tampered signature, alg:none	401
Malicious input (SQLi strings, XSS payloads in notes, oversized bodies, unicode tricks in phone numbers)	Rejected or safely stored and escaped on render
Unknown JSON fields (e.g., "eligible": true, "status": "COMPLETED")	400 UNKNOWN_FIELD
SMS provider timeout / 5xx	Retries with backoff → FAILED after max; in-app still delivered; admin counter increments
Email provider down	Same
DB unavailable	503 with request_id ; readiness probe fails; no partial writes (transactions)
Worker down	/readyz reports stale heartbeat; alert; jobs resume without loss

Case	Expected
Clock skew / timezone	All comparisons in UTC; display tests for EAT
Appeal launched twice (double click)	Idempotency; single wave
Donor opted out	Delivery SUPPRESSED, including CRITICAL
PHI accidentally in notes	Excluded from notifications and logs (tested redaction)

T. Deployment architecture [PROPOSED; hosting location pending AS-40]

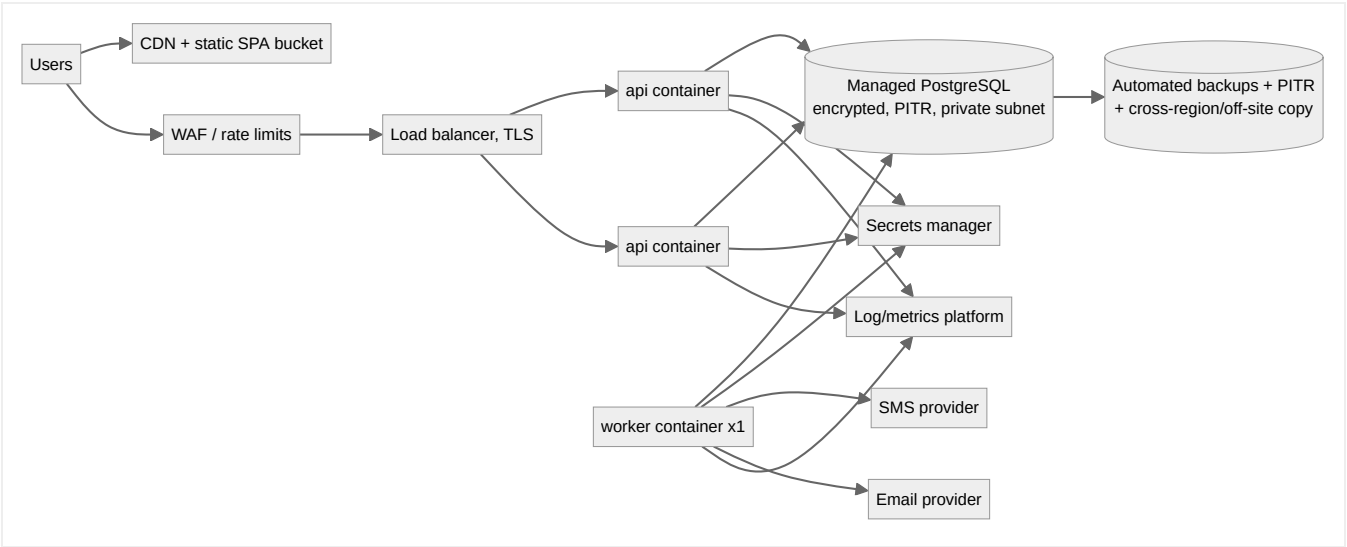
T.1 Environments

Env	Purpose	Data	Providers
local	development	development-mock seeds only	Mailpit, console SMS (DEVELOPMENT MOCK)
ci	automated tests	ephemeral	fakes
staging	pre-release verification, stakeholder demos	synthetic data only; never real donor data	sandbox providers
pilot/production	real use	real	real providers; REQUIRE_VALIDATED_RULES=true

T.2 Local

`docker compose up` → `postgres:16`, `api`, `worker`, `frontend` (Vite dev server), `mailpit`. The `make seed-dev` command loads reference and development-mock seeds; it refuses to run if `APP_ENV` isn't `local`.

T.3 Hosted (pilot) — portable container design (ADR-015)



- The option in the PDF (AWS):** ECS Fargate (or EC2 running containers, per the PDF), RDS PostgreSQL, S3 + CloudFront for the SPA, Secrets Manager, CloudWatch. **Region:** AWS has no Kenyan region. The nearest is `af-south-1` (Cape Town). If Kenyan law requires in-country storage or processing of this health data, AWS alone may not comply [VALIDATE AS-40].
- The in-Kenya option:** a Kenyan data centre or cloud provider running the same containers, managed or self-operated PostgreSQL, and an S3-compatible object store. The application uses **no provider-proprietary services in its core** (no SQS, Lambda or DynamoDB), so switching between these options is a deployment change only.
- Pilot sizing: 2 small API instances, 1 worker, a DB with 2 vCPU / 4–8 GB RAM, Multi-AZ/HA if the budget allows

(otherwise single instance + PITR, with an RTO accepted by stakeholders).

- **Migrations** run as a one-off release task before new API containers start. They must be backward-compatible (expand → migrate → contract) so a rollback doesn't need a DB rollback.

T.4 CI/CD (GitHub Actions or equivalent)

1. PR: ruff, mypy, eslint, tsc, unit/integration/API/authz tests with a Postgres service, frontend tests, axe, migration tests, security scans, OpenAPI diff (breaking-change detection), build images.
2. Merge to `main` → deploy to staging automatically → E2E smoke suite.
3. Production/pilot → manual approval, tagged release, migration task, rolling deploy, post-deploy smoke tests, automatic rollback on failed health checks.
4. Images are signed and scanned; SBOM generated.

U. Monitoring and logging [PROPOSED]

Signal	Implementation	Alert
Structured logs	structlog JSON: timestamp, level, request_id, user_id (UUID only), facility_id, route, status, latency. Redaction filter drops names, phones, emails, notes, tokens	—
Errors	Sentry-compatible error tracking with PII scrubbing (self-hosted or region-appropriate) [VALIDATE AS-40]	New error types; error-rate spike
API metrics	p50/p95/p99 latency, 4xx/5xx rates per route (Prometheus/OpenTelemetry or cloud metrics)	p95 > 1 s for 10 min; 5xx > 1%
Worker	heartbeat row updated every 30 s; job queue depth, oldest queued age, failures	heartbeat stale > 2 min; oldest job > 5 min
Notifications	sent/delivered/failed per channel and provider; SMS spend per day; provider account balance (prepaid SMS in Kenya can run out)	failure rate > 5%/15 min; balance below threshold; daily spend cap
Domain health	unacknowledged CRITICAL/URGENT requests past target; units expiring within 24 h with no plan; audit chain verification result	immediate page/SMS to on-call
Uptime	external probe on <code>/healthz</code> and the SPA	2 consecutive failures
Security	login failures spike, authz denials spike, refresh-token reuse events	threshold alerts
DB	connections, CPU, storage, replication lag, backup success	standard thresholds

Impact metrics (PDF success metrics) come from the reporting module, not the logs: request → acceptance, request → issue, request → completion, appeal → first acceptance, acceptance → donation conversion, donations per appeal, expired-vs-issued ratio, donor engagement rate. A quarterly "Is HemaNet improving coordination?" review compares these against the pre-pilot baseline, which must be measured before go-live [VALIDATE AS-50].

V. Disaster recovery considerations [PROPOSED]

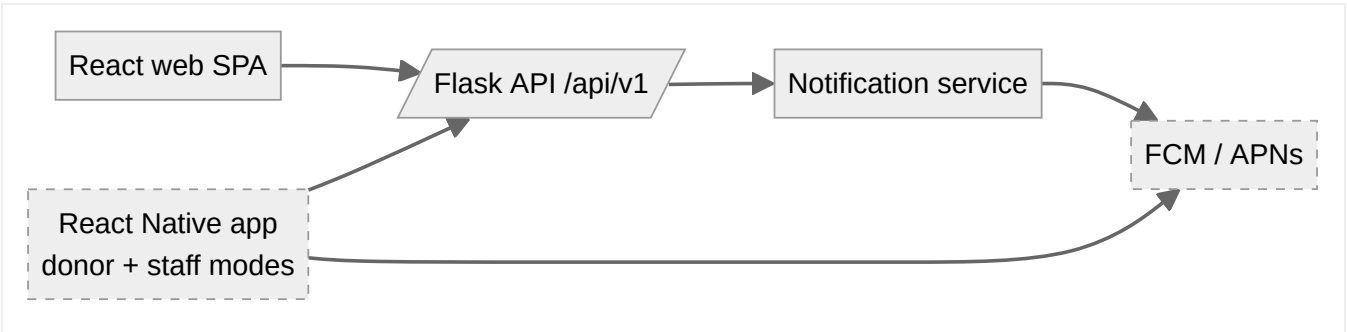
Item	Target / approach
RPO	≤ 5 min (continuous WAL archiving / PITR)
RTO	≤ 4 h pilot; ≤ 1 h at scale (HA DB + multi-instance)
Backups	Daily snapshots kept 35 days; monthly kept 12 months; one encrypted copy in a separate account/region (subject to residency)
Restore drills	Quarterly, timed and documented
Runbooks	DB restore, bad-deploy rollback, SMS provider outage (switch to secondary), credential compromise (key rotation + session revocation), data-breach response (72-hour notification path [VALIDATE AS-41])

Item	Target / approach
Operational fallback	HemaNet must never be the only way to get blood. Every pilot facility keeps its existing phone/paper process. HemaNet provides a printable/CSV current stock and open-request snapshot (generated every few hours and on demand) for use during outages. After an outage, staff back-enter events with actual <code>occurred_at</code> (recorded time vs occurred time are kept separately in <code>unit_events</code>) [VALIDATE AS-34]
Secrets	Rotation playbook; JWT key rotation using <code>kid</code> with an overlap window
Dependency failure	SMS/email down → in-app continues, staff dashboards show "notifications degraded"; maps down → no impact on core logic (distance computed in DB)

10 — Future Architecture: Mobile, FHIR/HL7, AI/Analytics

None of this is built in the MVP. Each section lists what is **built now** (a foundation that avoids expensive migration later) and what is **deferred**.

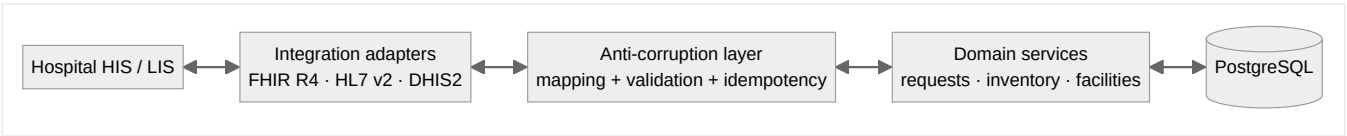
W. Future mobile architecture [FUTURE]



Concern	Built now (MVP)	Deferred
Auth	Token-based API; <code>client_type=MOBILE</code> returns the refresh token in the body; no cookie-only flows	Secure-storage integration, biometric unlock
Business logic	Entirely server-side; <code>allowed_actions</code> in responses so clients don't re-implement rules	—
Networking	Idempotency keys on all critical POSTs (flaky mobile networks); cursor pagination; compact responses	Offline queue for staff actions [VALIDATE: would offline work be acceptable operationally?]
Notifications	PUSH channel in the enum; channel-agnostic notification service	<code>device_tokens</code> table, push provider adapter
Deep links	Appeal response links use path <code>/r/:token</code> , which can become a universal/app link	App link association files
Types	OpenAPI → TS types shared by web and RN	Shared <code>@hemanet/api-client</code> package
Location	Donor location rounded server-side regardless of client	Background location is not planned (privacy); geofencing deferred and questionable
QR donor ID	<code>donor_reference</code> exists	QR rendering in app; staff scanning
API versioning	<code>/api/v1</code> + OpenAPI diff in CI	v2 only when a breaking change is unavoidable

X. Future FHIR / HL7 integration strategy [FUTURE]

Principle: HemaNet's domain model is independent of any external standard. Integration happens through adapters at the edge.



Candidate mappings (indicative; to be validated against the Kenyan national profiles published under the Digital Health Act framework [VALIDATE AS-46]):

HemaNet	FHIR R4 resource
---------	------------------

HemaNet	FHIR R4 resource
Facility	Organization + Location
Staff member	Practitioner / PractitionerRole
Blood request	ServiceRequest (clinical order) or SupplyRequest (logistics); the choice depends on the partner system
Blood unit	BiologicallyDerivedProduct
Issue/receipt	SupplyDelivery
Donor (if ever exchanged)	Patient/Person, only with an explicit legal basis

Other Kenya-relevant integrations to evaluate [VALIDATE AS-46]:

- **KMHFL** (Kenya Master Health Facility List) lookup for facility verification (V1, the cheapest win).
- **KHIS/DHIS2** aggregate reporting export (monthly indicators) (V2–V3).
- National health information exchange / client registry, subject to Digital Health Agency requirements (V3).
- **ISBT 128** labelling if Kenyan blood services use it (AS-31). The unit-identifier design (`issuing_system` + `unit_identifier`) already allows parsing it later.

Built now: clean domain models, reference codes, `integrations/` placeholder, OpenAPI docs, idempotency, UUIDs.
Deferred: every adapter, terminology mapping and message queue.

Y. Future AI / analytics architecture [FUTURE]

Principle: measure → baseline → simple heuristics → evaluated ML. AI recommendations are always explainable and human-approved, and never make clinical decisions [CONFIRMED].

Stage	When	What
0. Data foundation	MVP (built now)	Timestamped events (<code>unit_events</code> , <code>request_status_history</code> , appeals/responses), controlled reason codes (clean labels), consistent facility/component/group dimensions, reporting module
1. Descriptive	MVP/V1	Reports: wastage, fulfilment times, shortages, donor conversion
2. Heuristic intelligence	V1	Days of cover = available units ÷ trailing average daily issues (per facility × component × group); expiry-risk list (units unlikely to be used before expiry, given issue rates); transfer <i>suggestions</i> (V1+ with transfers). All transparent formulas
3. Forecasting	V3, after ≥ 12 months of reliable pilot data	Demand forecasting per facility × component × group (seasonal baselines first, e.g. seasonal naive/ETS; ML only if it beats the baseline on back-tests); donor response-propensity models (fairness-audited)
4. Anomaly detection	V3	Unusual discard rates, stock drift in stock-takes, suspicious access patterns

Architecture when needed: a read replica or nightly export to an analytics schema (`analytics.*` star schema: `fact_issues`, `fact_requests`, `fact_units_daily`, `fact_appeals`), batch jobs in the worker or a separate analytics service, and model outputs written back as **recommendations** with an explanation and model version. Evaluation reports (MAPE vs baseline, calibration) are required before any model is surfaced to users.

Explicitly avoided: LLM-generated clinical guidance, automated donor exclusion by opaque models, and "AI" features without an evaluation objective.

11 — Risks, Assumptions, Clinical/Operational Validation

Z. Risks

ID	Risk	Likelihood	Impact	Mitigation
R-01	No confirmed stakeholder access: workflows are built on assumptions and don't match real practice	High	High	Validation list AA is the agenda for the first stakeholder meetings; rules and roles are configurable; build the foundation first, pilot-specific details last
R-02	Unvalidated clinical rules used in a real setting	Medium	Critical	Rule-set lifecycle; REQUIRE_VALIDATED_RULES guard; UI banner; separation of duties; audit
R-03	Data-entry burden: per-unit tracking costs staff time, so adoption fails or data goes stale	High	High	Scanner-friendly entry, bulk/CSV import, minimal required fields, measure time per task in usability tests; later integration with the blood service's existing systems
R-04	Duplicate entry with an existing blood-establishment system (if one exists)	Medium	High	Ask early (AS-36); design import/export; favour integration over parallel entry in V1+
R-05	Stale availability misleads hospitals	Medium	High	Freshness indicators, "indicative" wording, requests still confirmed by the blood bank, stock-take reconciliation metrics
R-06	Donor data breach (sensitive personal data)	Low–Medium	Critical	Minimisation, masking, RBAC, MFA, encryption, audit, DPIA, incident runbook
R-07	Alert fatigue / donor burnout	Medium	Medium	Staff-launched appeals, caps, cooldown, fairness factor, opt-out respected
R-08	SMS costs and delivery reliability in Kenya	Medium	Medium	Spend caps/alerts, provider abstraction, secondary provider, in-app fallback, delivery metrics
R-09	Feature-phone donors can't use web links	High	Medium	Two-way SMS/USSD as a V1–V2 option; channel-agnostic response service; STAFF_RECORDED response channel (phone calls)
R-10	Data-residency non-compliance from choice of hosting	Medium	High	Decide hosting after legal review (AS-40) and before pilot; portable containers
R-11	Fake facilities or impersonation	Low	High	Verification workflow, KMHFL code, out-of-band confirmation, no operational access until verified
R-12	Over-scoping: building V1–V3 features too early	Medium	High	MVP scope §B locked; each deferred item has an explicit extension point only
R-13	Single-developer bus factor	Medium	Medium	ADRs, docs, tests, conventional structure
R-14	Over-claiming (e.g., "clinically validated")	Medium	High	Language policy (master §48), prototype disclaimers in UI and docs
R-15	Liability if HemaNet contributes to a delay or wrong product	Low	Critical	HemaNet never replaces the clinical cross-match/issue checks; it supports coordination only; legal review of terms; the pilot runs in parallel with existing processes
R-16	Worker/job failure silently stops escalations or expiry	Low	High	Heartbeat alerts, query-time expiry filters (correctness doesn't depend on jobs), readiness checks

Assumptions (general)

- The PDF reflects the intended product. Your decisions of 2026-09-30 supersede it where they differ.
- Pilot scale: ≤ 20 facilities, ≤ 50k registered donors, ≤ 100k units per year (sizing only).
- Staff have at least intermittent internet access and a browser (desktop, tablet or phone) [VALIDATE AS-49].

- English is acceptable for staff UIs at MVP [VALIDATE AS-23].

AA. Clinical and operational assumptions requiring validation

Each item names what we're assuming, why it matters, who should validate it, and how the design contains the risk. "Kenyan blood-service authority" means the national blood transfusion service or its current successor body. **Which institution now holds that mandate should itself be confirmed**, because we understand there have been recent institutional changes (AS-00).

ID	Assumption	Why it matters	Validate with	Design containment
AS-00	Identity of the current national authority for blood transfusion services and its guidelines	Every clinical rule depends on it	Ministry of Health / national blood service	All rules are configurable and cite a <code>validation_source_reference</code>
AS-01	The problems P1–P5 exist and are priorities	Product-market fit	Hospital labs, blood banks	Discovery interviews before pilot
AS-02	Supply structure: regional centres/satellites supply hospitals; some hospitals have their own blood banks	Facility model, routing	National blood service, hospital labs	Capability flags, not fixed types
AS-03	Hospitals request from a known, limited set of suppliers; one supplier at a time	<code>supply_links</code> , routing	Same	Links are admin-managed data; multi-supplier routing is a future extension
AS-04	Units received from an external centre arrive already tested and released, and may be registered as AVAILABLE	Registration path B	Blood bank managers	Separate permission <code>unit.register_released</code> ; can be restricted
AS-05	The workflow is request → accept → reserve → issue → receipt confirmation	State machine	Hospital labs + blood banks	States are code, but guards and permissions are configurable; receipt step can be made optional by config
AS-06	Time-limited reservations are appropriate	Stock tied up vs availability	Blood bank managers	Hold time per urgency level (config)
AS-07	No automatic component substitution	Allocation logic	Clinical/lab experts	Substitution is a future rule type
AS-08	FEFO ordering is acceptable (no minimum remaining shelf life for certain patients)	Allocation suggestions	Clinical/lab experts	Suggestions only; staff choose; a min-shelf-life rule type can be added
AS-09	Urgency levels, names, target and escalation times	Priority behaviour	Hospitals + blood service	URGENCY rule set
AS-10	Escalation path: supplier manager → requester + platform admin	Response times	Same	Config + templates
AS-11	Blood group compatibility by component (red cells, plasma, platelets, cryo) and preference ordering	Patient safety	National guidelines, transfusion medicine specialist	COMPATIBILITY rule set; deny-by-default; no built-in tables
AS-12	Donor eligibility criteria: age limits, minimum inter-donation interval (possibly sex-specific), other criteria HemaNet should <i>not</i> assess	Donor safety; who gets contacted	National donor selection guidelines	DONOR_ELIGIBILITY rule set; 3-state output; screening at site is authoritative
AS-13	Staff may record deferral <i>categories</i> in HemaNet, and doing so is appropriate and lawful	Sensitive data	Blood service + legal	Category codes only; can be disabled
AS-14	HemaNet records only the release decision, not test results, and the right role releases units	Safety + privacy	Blood bank quality managers	Separate permission; results never stored
AS-15	Component shelf life and storage temperature ranges	Expiry validation, future cold chain	National guidelines	COMPONENT_SPEC rule set; label expiry authoritative; mismatch = warning

ID	Assumption	Why it matters	Validate with	Design containment
AS-16	Discard reason categories	Wastage analytics	Blood bank managers	<code>reason_codes</code> table
AS-17	Which roles accept/decline requests, release units, issue units	RBAC	Facilities	Role → permission mapping is data
AS-18	Whether family/replacement donation practices exist at pilot sites and how HemaNet should treat them	Mobilisation design; ethics	Hospitals + blood service	Appeals are facility-level (not patient-directed); patient-directed appeals are not supported until validated
AS-19	Whether donor appeals linked to a specific hospital request are appropriate to disclose to donors (e.g., "needed for a request at Hospital X")	Privacy, coercion risk	Blood service + ethics	Default message names only the collection site and group needed
AS-20	Appeals should be launched by staff, not automatically	Alert storms, cost	Blood bank managers	Suggested → staff launch; auto-launch can be a future opt-in
AS-21	Donors whose status is "Requires review" may be invited with a caveat	Wasted trips vs reach	Blood bank managers	Per-appeal toggle, default off
AS-22	Donors can respond via smartphone links; the feature-phone share is unknown	Reach	Donor survey/pilot data	<code>STAFF_RECORDED</code> channel; two-way SMS/USSD future
AS-23	Language needs (English/Kiswahili)	Accessibility	Users	i18n scaffolding in MVP
AS-24	Slot-based appointments suit collection sites (vs walk-in only)	Mobilisation	Collection sites	Slots optional; walk-ins supported
AS-25	County-level fallback for donors without coordinates is useful	Reach	Blood banks	Toggle per appeal
AS-26	Mapping from priority to channels, and whether night-time alerts are acceptable	Donor experience	Donors + blood service	Config
AS-27	SMS sender ID registration, bulk-SMS and consent rules (Communications Authority; DPA direct-marketing rules)	Legal delivery	Legal + provider	Provider adapters; consent records
AS-30	Facilities can realistically record per-unit data (barcode scanners available? label formats?)	Adoption (R-03)	Blood bank staff	Scanner/CSV entry; usability testing
AS-31	Unit/donation identifier format (e.g., whether ISBT 128 is used) and uniqueness across issuing centres	Duplicate prevention, traceability	Blood service	<code>issuing_system</code> namespace; pluggable format validators
AS-32	Stock-take practice and frequency	Reconciliation	Blood banks	Optional feature
AS-33	How donors are identified at collection (national ID?) and whether HemaNet should store it	Duplicates vs privacy	Blood service + legal	Not stored in MVP; phone + DOB + <code>donor_reference</code>
AS-34	Back-entry after downtime is acceptable (recorded vs occurred time)	Data integrity	Blood banks	Separate timestamps
AS-35	Approvals needed before a pilot (MoH, blood service, county health departments, ethics review if research data is collected)	Legality of pilot	MoH / county / ethics committee	Pilot readiness checklist (AE step 22)
AS-36	Existing systems at blood banks/hospitals (lab/blood-establishment systems, KHIS reporting duties)	Duplicate entry (R-04)	Facilities	Import/export first; integration later
AS-37	Cost-recovery/fees for blood processing and how they affect the request workflow	Workflow, scope	Hospitals + blood service	Out of MVP scope; no billing entities
AS-40	Data residency: whether Kenyan regulations require HemaNet's health-related data to be processed/stored in Kenya	Hosting choice	Data-protection counsel	Portable deployment (ADR-015); decide before pilot

ID	Assumption	Why it matters	Validate with	Design containment
AS-41	DPA obligations: ODPC registration, DPIA, lawful basis/consent wording, retention periods, 72-hour breach notification	Legal operation	Counsel / ODPC guidance	Consent versioning, retention config, runbooks
AS-42	Donor erasure vs traceability retention	Rights vs safety	Counsel + blood service	Pseudonymisation design
AS-43	Whether and how donors may be told their donation was used	Retention, privacy	Blood service + ethics	Future; no patient data ever
AS-44	Authoritative county/sub-county list and codes	Reference data	Official publications	Seed with source citation
AS-45	Session lifetimes acceptable for shared ward computers	Security vs usability	Hospitals	Config per role
AS-46	Kenyan interoperability standards (national FHIR profiles, KHIS/DHIS2, facility registry APIs)	Future integration	Digital Health Agency / MoH	Adapter layer only
AS-47	Whether HemaNet must register or be certified under the Digital Health Act before handling health data	Legal operation	Counsel / Digital Health Agency	Pilot readiness checklist
AS-48	Blood service centres have KMHFL codes	Facility verification	KMHFL	kmhf1_code nullable
AS-49	Connectivity and devices at facilities	UX, offline needs	Site visits	Lightweight SPA; outage snapshot
AS-50	A pre-pilot baseline (current response times, wastage) can be measured	Impact measurement	Pilot facilities	Baseline data collection plan

Open questions for you

1. Who will run HemaNet during the pilot (you, a university, an NGO, a partner facility)? This determines the data-controller role under the DPA.
2. Do you have a target county or region for the first pilot?
3. Budget constraints for hosting and SMS (they affect the hosting option and SMS strategy)?
4. Team: are you developing alone, or will others contribute (affects process and code-review setup)?
5. Should HemaNet eventually handle hospital-internal inventory (hospital blood banks), or only supplier inventory? The model supports both; the MVP UI assumes supplier-side blood banks and hospital blood banks behave the same.

12 — AB. Architectural Decisions and Trade-offs (ADR log)

Status for all: **Proposed** (pending your approval). Once approved, each moves to `docs/adr/ADR-xxx-*.md` as an individual record.

ADR-001 — Modular monolith

- **Decision:** A single Flask codebase with strict domain modules, deployed as `api` + `worker` processes.
- **Context:** One team, early product, domain boundaries still being learned.
- **Alternatives:** Microservices (premature: operational cost and distributed transactions around inventory); a single flat Flask app (becomes unmaintainable).
- **Reason:** Fast iteration, one transaction boundary for inventory integrity, and module boundaries enforced by import-linter so services can be extracted later.
- **Consequences:** Scaling is vertical plus horizontal replicas; extraction candidates later: notifications, matching, integrations.

ADR-002 — Domain-first module layout

- **Decision:** `app/modules/<domain>/{models,schemas,repository,domain,service,policies,routes}` instead of layer-first top-level folders.
- **Alternatives:** The suggested hybrid layout (layers + domains side by side).
- **Reason:** Each feature lives in one place; ownership of models is clear; cross-module access only through services and events.
- **Consequences:** Some shared utilities live in `core/`; discipline is needed so `core` doesn't grow domain logic.

ADR-003 — PostgreSQL as system of record, job queue and outbox

- **Decision:** Use Postgres tables (`jobs`, `outbox_events`) with `FOR UPDATE SKIP LOCKED` for background work and events. No Redis/Celery/SQS in the MVP.
- **Alternatives:** Celery + Redis (more moving parts, and no transactional guarantee between the DB write and enqueue); AWS SQS (vendor lock-in conflicts with the residency uncertainty).
- **Reason:** Events are enqueued atomically with state changes (transactional outbox), one fewer service to operate and secure, and ample throughput at pilot scale (thousands of jobs/minute).
- **Consequences:** Must monitor queue depth; if throughput needs grow, swap the job backend behind the `jobs` interface.

ADR-004 — Per-unit inventory as source of truth

- **Decision:** `blood_units` rows are the truth; totals are derived (view). Batches are labels on units.
- **Alternatives:** Aggregate counts (PDF), which can't track expiry per unit; separate batch-quantity stock (two sources of truth).
- **Reason:** Correct expiry handling, traceability, prevention of double allocation, a foundation for QR, recall and transfers. Confirmed by your decision 5.
- **Consequences:** More data entry (R-03), so we invest in scanner/CSV entry. Counts are queries (indexed, cheap at pilot scale).

ADR-005 — Clinical rules as versioned, validated configuration

- **Decision:** Compatibility, eligibility, component specs and urgency live in `rule_sets` with the lifecycle DRAFT →

PENDING_VALIDATION → VALIDATED → ACTIVE → RETIRED. Code implements rule *types*; data supplies *values*. Separation of duties and an environment guard apply.

- **Alternatives:** Hardcoded tables (violates the master instruction; unsafe); a free-form rules engine/DSL (over-engineered, and hard to validate for non-engineers).
- **Reason:** Validated rules can change without code deploys; every decision is reproducible (pinned rule-set IDs); there's a clear accountability trail.
- **Consequences:** New *types* of rule still need code; the admin UI must present tables clearly for clinical reviewers.

ADR-006 — Separate fulfilment and mobilisation

- **Decision:** Requests are fulfilled only from released inventory. Donor appeals replenish inventory and are linked to, but not equal to, requests.
- **Reason:** Donated blood must be collected and processed before use; a donor's "accept" isn't supply. Confirmed by your decision 4.
- **Consequences:** Two state machines; the link is `appeals.related_request_id`. Direct patient-directed donation isn't modelled (AS-18).

ADR-007 — Short-lived JWT access + rotating opaque refresh tokens

- **Decision:** 15-min JWT in memory; DB-stored, hashed, rotating refresh tokens (cookie for web, body for mobile); permissions resolved server-side per request.
- **Alternatives:** Server-side sessions only (simple for web, awkward for mobile); long-lived JWTs (no revocation).
- **Reason:** One mechanism for web and mobile, immediate revocation via `token_version` + refresh revocation, and no permissions baked into tokens.
- **Consequences:** Refresh logic in the frontend client; CSRF protection on refresh endpoints.

ADR-008 — Facility-scoped RBAC with permission codes

- **Decision:** Roles map to permissions (data); memberships bind user + role + facility; policy checks combine permission, scope, state and capability.
- **Reason:** "Role ≠ permission" (master §25); users can belong to multiple facilities; validated role responsibilities (AS-17) can be applied without code changes.
- **Consequences:** Needs an exhaustive authz test matrix (planned).

ADR-009 — Coordinates + haversine now, PostGIS in V1

- **Decision:** Store `latitude/longitude` numerics; compute distance with bounding box + haversine SQL behind `geo.find_within_radius()`. Introduce PostGIS (geography column + GiST index, backfilled by migration) when radius queries or transfer planning need it.
- **Alternatives:** PostGIS from day one (small cost, but adds an extension dependency to every environment and more to learn); external maps APIs for distance (cost, latency, dependency, privacy).
- **Reason:** Adequate at pilot scale; the migration path is trivial because the interface is stable.
- **Consequences:** Straight-line distance only (not travel time); acceptable for ranking.

ADR-010 — Provider-agnostic notification adapters

- **Decision:** `EmailProvider/SmsProvider` interfaces; dev mocks; providers selected by config; secondary SMS provider supported.
- **Reason:** Kenyan SMS market specifics (e.g., Africa's Talking vs Twilio: cost, sender IDs, delivery), failover, testability.

ADR-011 — Append-only hash-chained audit; no blockchain

- **Decision:** DB-level append-only audit and unit events with hash chaining and optional external anchoring.
- **Alternatives:** Blockchain (PDF future enhancement): no trust boundary between mutually distrusting writers that needs it, plus high operational complexity and privacy risk from immutable personal data on a ledger.
- **Reason:** Tamper-evidence at a fraction of the cost; personal data stays deletable/pseudonymisable where the law requires.

ADR-012 — Polling instead of WebSockets in the MVP

- **Decision:** TanStack Query polling at 15–60 s; SSE in V1 fed by outbox events.
- **Reason:** Simpler infrastructure (no sticky sessions or connection management); the latency needs of the MVP workflows are measured in minutes, not milliseconds.
- **Consequences:** Slightly higher request volume, mitigated by cheap endpoints and pausing when the tab is hidden.

ADR-013 — No patient identifiers in the MVP

- **Decision:** Requests carry the component, recipient ABO/Rh, quantity, urgency and an optional hospital reference, never patient name/ID/diagnosis. No prescription uploads.
- **Reason:** Data minimisation; coordination doesn't need patient identity; it greatly reduces regulatory burden and breach impact.
- **Consequences:** Patient-level traceability stays in hospital systems; future integration requires a legal basis.

ADR-014 — flask-smorest + marshmallow; OpenAPI-generated TypeScript types

- **Decision:** API schemas defined once in marshmallow; the OpenAPI spec is generated; the frontend uses `openapi-typescript`.
- **Alternatives:** Pydantic + flask-openapi3/spectree (good typing, less mature Flask integration); hand-written TS types (drift).
- **Reason:** Mature Flask integration, a single source of truth, and CI can detect breaking API changes, which protects future mobile clients.

ADR-015 — Portable container deployment; hosting decided after legal review

- **Decision:** Docker images; only standard Postgres and S3-compatible storage; no proprietary managed services in core logic. AWS (PDF) or an in-Kenya provider is chosen after AS-40 is answered.
- **Reason:** Data-residency uncertainty; avoiding lock-in.

ADR-016 — UUID primary keys + human reference codes

- **Decision:** UUIDs for entity IDs; `reference_code` (e.g., `REQ-2026-000123`) for humans and phone calls; bigint identities for append-only logs.
- **Reason:** Non-enumerable IDs (defence in depth against IDOR), merge-friendly, readable references for operations.

ADR-017 — Eligibility is a three-state *indication*, never a clearance

- **Decision:** Output is always `POTENTIALLY_ELIGIBLE` / `POTENTIALLY_INELIGIBLE` / `REQUIRES_REVIEW`; missing data → `REQUIRES_REVIEW`; screening at the collection site is authoritative.
- **Reason:** Master §16/§49; HemaNet must not make medical decisions.

ADR-018 — Staff-in-the-loop for high-impact actions

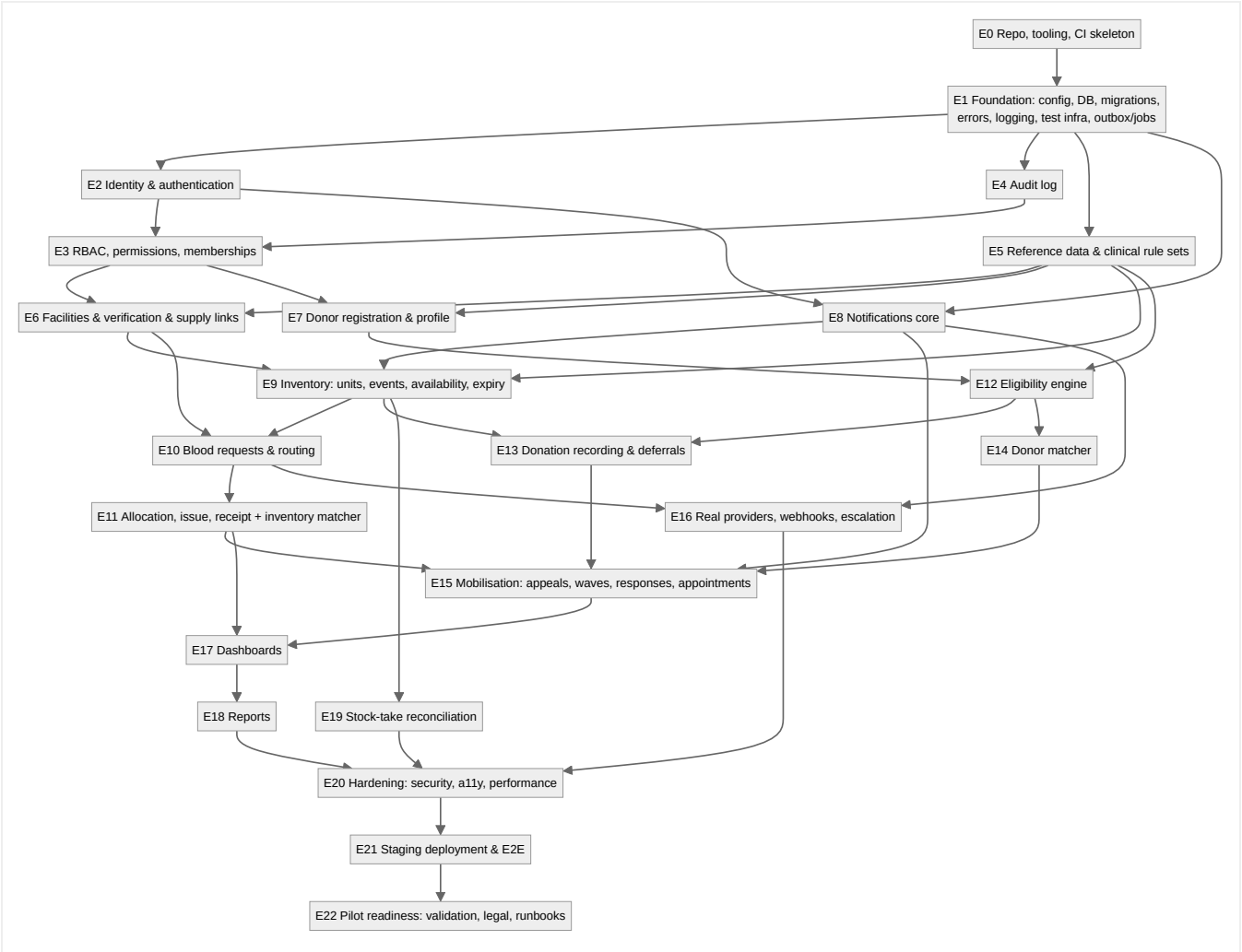
- **Decision:** The system *suggests* allocations and appeals; humans reserve, issue and launch. No automatic re-

routing, auto-waves or transfers.

- **Reason:** Safety, cost control, validation pending; automation can be enabled per facility later once it's trusted.

13 — Dependencies, Backlog, Development Order, Definition of Done

AC. Feature dependency graph



The frontend for each epic is built together with its backend (vertical slices) from E6 onward. E0–E5 include the frontend shell: app skeleton, auth pages, layouts and the design-system basics.

AD. Prioritised MVP backlog

Priority: **P0** = required for the MVP to be meaningful; **P1** = required before the pilot; **P2** = nice to have within Phase 1.
Size: S (≤ 1 day), M (2–3 days), L (4–5 days). Estimates are for one experienced developer, including tests; for planning only.

#	Epic / item	Pri	Size	Depends on	Label
E0 Repository & tooling					
0.1	Monorepo layout (backend/, frontend/, docs/, infra/), README, CONTRIBUTING, .editorconfig, .gitignore	P0	S	—	PROPOSED
0.2	Docker compose (postgres, api, worker, frontend, mailpit); .env.example	P0	S	0.1	PROPOSED

#	Epic / item	Pri	Size	Depends on	Label
0.3	CI: lint, type-check, test jobs (placeholders), secret scan	P0	S	0.1	PROPOSED
0.4	ADR folder seeded from \$AB	P0	S	approval	CONFIRMED (master \$56)
E1 Backend & frontend foundation					
1.1	App factory, typed config per env, extensions	P0	S	E0	PROPOSED
1.2	SQLAlchemy/Alembic setup, naming conventions, base model mixins (timestamps, version)	P0	S	1.1	CONFIRMED (migrations)
1.3	Error envelope, exception mapping, request ID, structured logging + redaction	P0	M	1.1	PROPOSED
1.4	Pagination, idempotency-key middleware, If-Match helper	P0	M	1.2	PROPOSED
1.5	Outbox + jobs tables, worker loop, scheduler, heartbeat	P0	M	1.2	PROPOSED
1.6	Test infrastructure: Postgres fixtures, factories, clock injection, API client helpers	P0	M	1.2	CONFIRMED (testing)
1.7	Health endpoints, security headers, CORS config	P0	S	1.1	PROPOSED
1.8	Frontend skeleton: Vite/TS strict, router, QueryClient, i18n, Tailwind tokens, layouts, error boundary, API client from OpenAPI	P0	M	0.1	CONFIRMED stack
1.9	Design-system basics: Button, Field, Select, Dialog, Table, StatusBadge, UrgencyBadge, EmptyState, ErrorState, Banner	P0	M	1.8	PROPOSED
E2 Identity & authentication					
2.1	Users model, Argon2id, registration primitives	P0	M	E1	CONFIRMED
2.2	Login, access/refresh tokens, rotation, reuse detection, logout, CSRF on refresh	P0	L	2.1	PROPOSED
2.3	Phone OTP + email verification, OTP abuse limits	P0	M	2.1, 1.5	PROPOSED
2.4	Password reset (enumeration-safe)	P0	S	2.3	PROPOSED
2.5	Brute-force protection, login attempts	P0	S	2.2	PROPOSED
2.6	TOTP MFA + recovery codes + step-up	P1	M	2.2	PROPOSED
2.7	Session list/revoke	P1	S	2.2	PROPOSED
2.8	Frontend: login, register shells, token manager, refresh, guards	P0	M	2.2, 1.8	CONFIRMED
E3 RBAC					
3.1	Roles, permissions, seeds; require_permission; Scope; policy registry; 404-on-out-of-scope	P0	L	E2	CONFIRMED principle
3.2	Facility memberships, invitations, acceptance	P0	M	3.1, E6.1	PROPOSED
3.3	Authz matrix test harness (YAML from \$M) + "every route declared" CI check	P0	M	3.1	PROPOSED
E4 Audit					
4.1	audit_events, writer, hash chain, DB grants + trigger	P0	M	E1	CONFIRMED
4.2	Audit viewer API + chain verification job	P1	M	4.1	PROPOSED
E5 Reference data & clinical configuration					
5.1	Counties/sub-counties, components, reason codes seeds (with source citations)	P0	S	E1	PROPOSED
5.2	Rule sets + entries models; lifecycle service; one-ACTIVE constraint; freeze trigger	P0	L	4.1, 3.1	PROPOSED
5.3	Development-mock rule sets (flagged) + environment guard + banner endpoint	P0	S	5.2	PROPOSED
5.4	Admin UI: rule-set list/detail/edit draft/submit/record validation/activate/diff	P1	L	5.2, 2.6	PROPOSED

#	Epic / item	Pri	Size	Depends on	Label
E6	Facilities				
6.1	Facility model, self-registration (API + UI)	P0	M	E3.1	CONFIRMED
6.2	Admin verification queue + decision (API + UI)	P0	M	6.1	CONFIRMED
6.3	Supply links (admin)	P0	S	6.2	PROPOSED
6.4	Storage locations; facility settings UI; staff management UI	P1	M	3.2	PROPOSED
E7	Donors				
7.1	Donor registration flow (API + multi-step UI), consent capture, rounding of location	P0	L	E2, 5.1	CONFIRMED
7.2	Profile, availability, notification preferences (API + UI)	P0	M	7.1	CONFIRMED/ PROPOSED
7.3	Data export/deactivation	P2	M	7.1	PROPOSED [VALIDATE]
E8	Notifications core				
8.1	Notification + delivery models; service; policy (prefs, consent, quiet hours, caps, verified contact)	P0	L	1.5, E2	CONFIRMED
8.2	Template system with variable allow-lists (PHI guard)	P0	M	8.1	PROPOSED
8.3	In-app inbox API + UI; unread count polling	P0	M	8.1	CONFIRMED
8.4	Dev mock providers (console SMS, Mailpit) with MOCK labelling	P0	S	8.1	CONFIRMED
E9	Inventory				
9.1	Units model, unit state machine (pure), unit events	P0	L	E6, 5.2	CONFIRMED
9.2	Register unit (single), scan-mode UI	P0	M	9.1	CONFIRMED
9.3	Bulk register + CSV dry-run import	P1	M	9.2	PROPOSED
9.4	Release/quarantine/discard/correct actions (API + UI)	P0	M	9.1	CONFIRMED/ PROPOSED
9.5	Availability view, facility inventory matrix UI, unit detail + history UI	P0	M	9.1	CONFIRMED
9.6	Expiry job + expiring-soon view + digest	P0	M	9.1, 8.1	CONFIRMED
9.7	Thresholds + low-stock detection + inventory alert	P0	M	9.5	CONFIRMED
9.8	Network availability (hospital view)	P0	S	9.5, 6.3	CONFIRMED
E10	Blood requests				
10.1	Request model, state machine (pure), status recompute, history	P0	L	E9, 5.2	CONFIRMED
10.2	Create + route (API + form UI with availability panel)	P0	M	10.1, 9.8	CONFIRMED
10.3	Supplier accept/decline; re-route; cancel; expire job	P0	M	10.1	CONFIRMED
10.4	Hospital request list/detail/timeline UI; supplier incoming queue UI	P0	M	10.2	CONFIRMED
E11	Allocation & fulfilment				
11.1	Inventory matcher (pure) + suggestions endpoint with explanations/ shortfall	P0	M	10.1	PROPOSED
11.2	Reserve (row locks + partial unique index), release, reservation timeout job	P0	L	11.1	PROPOSED
11.3	Issue + receipt + close (API + UI)	P0	M	11.2	PROPOSED [VALIDATE AS-05]
11.4	Concurrency test suite	P0	M	11.2	CONFIRMED (master S33)
E12	Eligibility				
12.1	Rule-type implementations + parameter schemas + evaluator (fail closed)	P0	L	5.2	CONFIRMED principle
12.2	Donor eligibility endpoint + dashboard card (3-state, reasons, next date)	P0	M	12.1, 7.1	CONFIRMED

#	Epic / item	Pri	Size	Depends on	Label
E13 Donations					
13.1	Staff donor lookup (audited), blood-group confirmation	P0	M	7.1, E3	PROPOSED
13.2	Record donation (API + UI) → register resulting units (QUARANTINED)	P0	M	13.1, 9.2	CONFIRMED
13.3	Deferral category recording/lifting	P1	S	13.1	PROPOSED [VALIDATE AS-13]
13.4	Donor donation history UI	P0	S	13.2	CONFIRMED
E14 Donor matcher					
14.1	Candidate generation (SQL, geo interface), hard filters, scoring, explanations (pure + golden tests)	P0	L	12.1	CONFIRMED
E15 Mobilisation					
15.1	Appeals model + lifecycle; suggested appeals from threshold/shortfall events	P0	L	9.7, 11.1	CONFIRMED
15.2	Appeal editor UI + target-group suggestions + preview (masked, explained)	P0	M	15.1, 14.1	CONFIRMED
15.3	Waves: launch, snapshots, signed response tokens, notifications	P0	M	15.2, 8.2	CONFIRMED
15.4	Donor response (in-app + link page / ɿ / : token), slots, appointments	P0	M	15.3	CONFIRMED
15.5	Check-in / no-show; donation linkage; reveal-on-accept for staff	P0	M	15.4, 13.2	CONFIRMED
15.6	Hospital appeal summary (counts only)	P1	S	15.1	PROPOSED
E16 Real providers & escalation					
16.1	Email provider adapter (SendGrid or SES)	P1	S	8.1	CONFIRMED
16.2	SMS provider adapter (Africa's Talking and/or Twilio) + cost tracking	P1	M	8.1	CONFIRMED
16.3	Delivery webhooks with signature verification	P1	M	16.1, 16.2	PROPOSED
16.4	Request escalation jobs; wave check prompts	P1	M	10.3, 15.3	PROPOSED
16.5	Admin delivery health view + manual retry	P1	S	16.3	PROPOSED
E17 Dashboards					
17.1	Donor, hospital, blood-bank, admin dashboards (polling)	P0	L	E11, E15	CONFIRMED
E18 Reports					
18.1	Donation trends, shortages, donor activity	P0	M	17.1	CONFIRMED
18.2	Request performance, wastage; CSV export (audited)	P1	M	18.1	PROPOSED
E19 Reconciliation					
19.1	Stock-takes	P1	M	E9	PROPOSED
E20 Hardening					
20.1	Security review against ASVS L2 checklist; ZAP baseline; fix findings	P0	L	all P0	PROPOSED
20.2	Accessibility audit (axe + manual keyboard + screen reader pass)	P0	M	all UI	PROPOSED
20.3	Performance test at pilot volumes; index tuning	P1	M	all	PROPOSED
20.4	Kiswahili translation of donor-facing strings	P1	M	1.8	PROPOSED [VALIDATE AS-23]
E21 Staging					
21.1	Infrastructure as code for staging; CI/CD deploy; backups; monitoring & alerts	P1	L	20.1	PROPOSED
21.2	Full E2E suite in CI against staging-like stack	P0	M	E17	CONFIRMED
E22 Pilot readiness (non-code + code)					
22.1	Stakeholder validation of AA items; record results; configure validated rule sets	P0-pilot	—	stakeholders	CONFIRMED (your decision 2)

#	Epic / item	Pri	Size	Depends on	Label
22.2	Legal: DPIA, ODPC registration, privacy notice, terms, residency decision, Digital Health Act check	P0-pilot	—	counsel	VALIDATE
22.3	Runbooks, downtime snapshot export, restore drill, on-call	P0-pilot	M	21.1	PROPOSED
22.4	Production hosting per residency decision	P0-pilot	L	22.2	PROPOSED
22.5	Baseline measurement plan at pilot sites	P0-pilot	—	22.1	CONFIRMED (PDF pilot plan)

AE. Exact development order

Each step ends with a **gate**: its tests pass in CI, docs are updated, and you review before the next step starts.

Step	Work	Gate / demo
1	E0: repo, compose, CI skeleton, ADRs	<code>docker compose up</code> runs empty api/frontend; CI green
2	E1.1–1.7: backend foundation	<code>/healthz</code> , <code>/readyz</code> ; error envelope tests; worker processes a no-op job
3	E1.8–1.9: frontend shell & design system	Storybook-free component tests + axe pass; layouts render
4	E4.1: audit	Append-only proven by tests (UPDATE/DELETE fail)
5	E2.1–2.5, 2.8: authentication	Register/login/refresh/logout E2E; brute-force tests
6	E3.1, 3.3: RBAC + authz harness	Harness runs (initially few routes); "undeclared route" check enforced
7	E5.1–5.3: reference data, rule sets, mock guard	Pilot-mode config refuses mock activation (test)
8	E6.1–6.3 + E3.2: facilities, verification, memberships	J5 E2E (facility onboarding)
9	E7.1–7.2: donor onboarding	J1 E2E minus eligibility
10	E8.1–8.4: notifications core	Events → in-app + mock email/SMS; PHI guard tests
11	E9.1–9.2, 9.4–9.8: inventory	Register/release/discard; availability; expiry job; low-stock alert
12	E10: requests	Create/route/accept/decline/cancel/expire
13	E11: allocation & fulfilment + concurrency suite	J2+J3 E2E: first complete fulfilment slice ← key milestone
14	E12: eligibility engine	Golden tests per rule type; donor dashboard card
15	E13.1–13.2, 13.4: donation recording	Donation → quarantined unit → release
16	E14: donor matcher	Golden explanation tests; performance at 50k synthetic donors
17	E15.1–15.5: mobilisation	J4 E2E: full mobilisation loop ← key milestone
18	E17, E18.1: dashboards & core reports	Role dashboards reviewed against master §30 questions
19	E2.6–2.7, E4.2, E5.4, E6.4: admin hardening features	MFA/step-up enforced on sensitive actions
20	E16: real providers (sandbox), webhooks, escalation	Delivery tracking with sandbox providers
21	E9.3, E13.3, E15.6, E18.2, E19: P1 completions	—
22	E20: security, accessibility, performance hardening	ASVS checklist signed off; zero serious axe issues; k6 targets met

Step	Work	Gate / demo
23	E21: staging, IaC, monitoring, backups	Restore drill passes
24	E22: pilot readiness (runs in parallel from step 1 for the non-code work: stakeholder outreach and legal questions)	Pilot go/no-go review

Rough effort: ~26–32 developer-weeks for steps 1–23 (one experienced developer). This is an estimate for planning, not a commitment.

AF. Definition of Done

A backlog item is **Done** only when every applicable box is ticked:

Functionality

- ☐ Backend use case, API and frontend implemented as specified; `allowed_actions` correct
- ☐ Works end-to-end in the local docker stack

Data integrity

- ☐ DB constraints enforce invariants (not only application code)
- ☐ State transitions go through the state machine only; invalid transitions tested
- ☐ Transactions wrap multi-row changes; concurrency considered and tested where relevant
- ☐ Migration included, tested upgrade-from-empty and on a seeded DB; backward-compatible

Security & privacy

- ☐ Permission declared on every new route; authz matrix entries added and passing
- ☐ Out-of-scope access returns 404; no sensitive fields in responses (schema reviewed)
- ☐ Input validation with `unknown=RAISE`; free text length-limited
- ☐ No PHI/secrets in logs or notifications (redaction tests where relevant)
- ☐ Data minimisation question answered: "Do we need to store this?"

Clinical safety

- ☐ No clinical value hardcoded; reads the ACTIVE rule set only through `clinical_config`
- ☐ Behaviour with a missing/unvalidated rule set defined and tested (fail closed)
- ☐ Any new clinical/operational assumption added to SAA

Auditability

- ☐ Audit events written for significant actions (Q.3); unit events for unit changes

Error handling & resilience

- ☐ Error envelope with codes; frontend shows meaningful error and empty states
- ☐ Idempotency on critical POSTs; retries safe
- ☐ Failure of external providers handled (tests with fakes)

UX & accessibility

- ☐ Answers the role's key questions (master §30) and is readable by the intended user in seconds
- ☐ Keyboard-operable, labelled, focus managed, status not conveyed by colour alone; axe clean
- ☐ Works at 360 px width (donor) / tablet (staff); strings externalised

Testing

- ☐ Unit tests for domain logic; service/API tests; dangerous cases from §S.3 that apply

- ☐ CI green (lint, types, tests, security scans, OpenAPI breaking-change check)

Documentation

- ☐ OpenAPI descriptions complete; README/architecture/DB docs updated; ADR added if a decision was made
- ☐ Environment variables added to `.env.example`

Future compatibility

- ☐ No web-only assumptions in the API (usable by the future mobile client)
 - ☐ No business logic only in React
-

Decisions needed from you

The **[PROPOSED]** items with the most impact on implementation. Please approve, amend or reject each:

1. **ADR-002:** domain-first backend layout (differs from the suggested structure).
2. **ADR-003:** Postgres-backed jobs/outbox instead of Celery/Redis.
3. **ADR-005:** rule-set lifecycle with a separate CLINICAL_CONFIG_APPROVER role and MFA step-up.
4. **ADR-007:** JWT access + rotating refresh tokens (vs. server sessions).
5. **ADR-013:** no patient identifiers at all in the MVP, and dropping S3 prescription uploads.
6. **ADR-011:** reject blockchain in favour of a hash-chained audit log.
7. **ADR-018 / AS-20:** appeals are suggested by the system and launched by staff (no automatic donor SMS on low stock).
8. **Donor privacy:** donors masked to staff until they accept; hospitals see only banded counts.
9. **Admin "verify donors" (PDF):** replaced by phone OTP + lab confirmation of blood group by blood-bank staff, rather than manual admin review of every donor.
10. **Scope:** stock-take reconciliation (E19) and Kiswahili (20.4) as P1 before the pilot.
11. **Hosting:** defer the AWS-vs-in-Kenya decision until legal input on data residency (AS-40); build portable.